Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 3. Divide and Conquer Algorithms
Session 1

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 3. Divide and Conquer Algorithms

8 hours

3.1 Binary search, min max finding algorithm
3.2 Analysis of sorting algorithms
- 3.2.1. Merge sort
- 3.2.2. Heap sort
- 3.2.3. Quick sort
- 3.2.4. Randomized Quick sort

3.3. Order Statistics
- 3.3.1. Selection in expected linear time
- 3.3.2. Selection in worst case linear time

BINARY SEARCH ALGORITHM

→ This algorithm looks for a value in a non decreasing sequence and returns the index of the value if it is found or -1 if it is not found.

→ **Input:** A sequence A_i, A_{i+1}, …. …., A_j, $i \geq 0$ sorted in non-decreasing order, a value *key* and *j*.

→ **Output:** The output is an index *k* for which A_k = key or if key is not in the sequence, the output is the value 0.

```
Algorithm binary_search(A,i,j,key)
{
if(i>j) //not found
     return -1
k = [(i+j)/2]
if(key==Ak) //found
     return k
if(key<Ak) //search left half
     j=k-1
else //search right half
     i=k+1
return binary_search(A,i,j,key)
}
```

Note: The index of array is assumed to start from 0.

EXAMPLE 1:

Let,

Sequence = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91} and

key = ***23***

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

i=0 1 2 3 ***k***=4 5 6 7 8 ***j***=9

2	5	8	12	**16**	23	38	56	72	91

Since, key>16, take right half i.e. put i=k+1

0 1 2 3 4 *i*=5 6 *k*=7 8 ***j***=9

2	5	8	12	16	23	38	**56**	72	91

Since, key<56, take left half i.e. put j=k-1

0 1 2 3 4 *i*,*k*=5 *j*=6 7 9

2	5	8	12	16	**23**	38	56	72	91

Since, key=23, value found at index 5

EXAMPLE 2:

Let,

Sequence = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91} and

key = ***5***

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

i=0 1 2 3 ***k***=4 5 6 7 8 ***j***=9

2	5	8	12	**16**	23	38	56	72	91

Since, key<16, take left half i.e. put j=k-1

i=0 *k*=1 2 *j*=3 4 5 6 7 8 9

2	**5**	8	12	16	23	38	56	72	91

Since, key=5, value is found at index 1

Time Complexity:

From the above algorithm, the running time of Algorithm is:

$T(n) = T(n/2) + O(1)$

In the best case, output is obtained at one run. i.e. $O(1)$ time if the key is at the middle. In the worst case, the output is at the end of the array so running time is $O(\log n)$ time. In the average case also running time is $O(\log n)$. For unsuccessful search best, worst and average time complexity is $O(\log n)$.

MIN MAX FINDING ALGORITHM

→ This algorithm is used to find the minimum and maximum items in a set of *n* elements.

→ **Input:** An array

→ **Output:** Maximum and Minimum values

→ **Iterative Algorithm:**

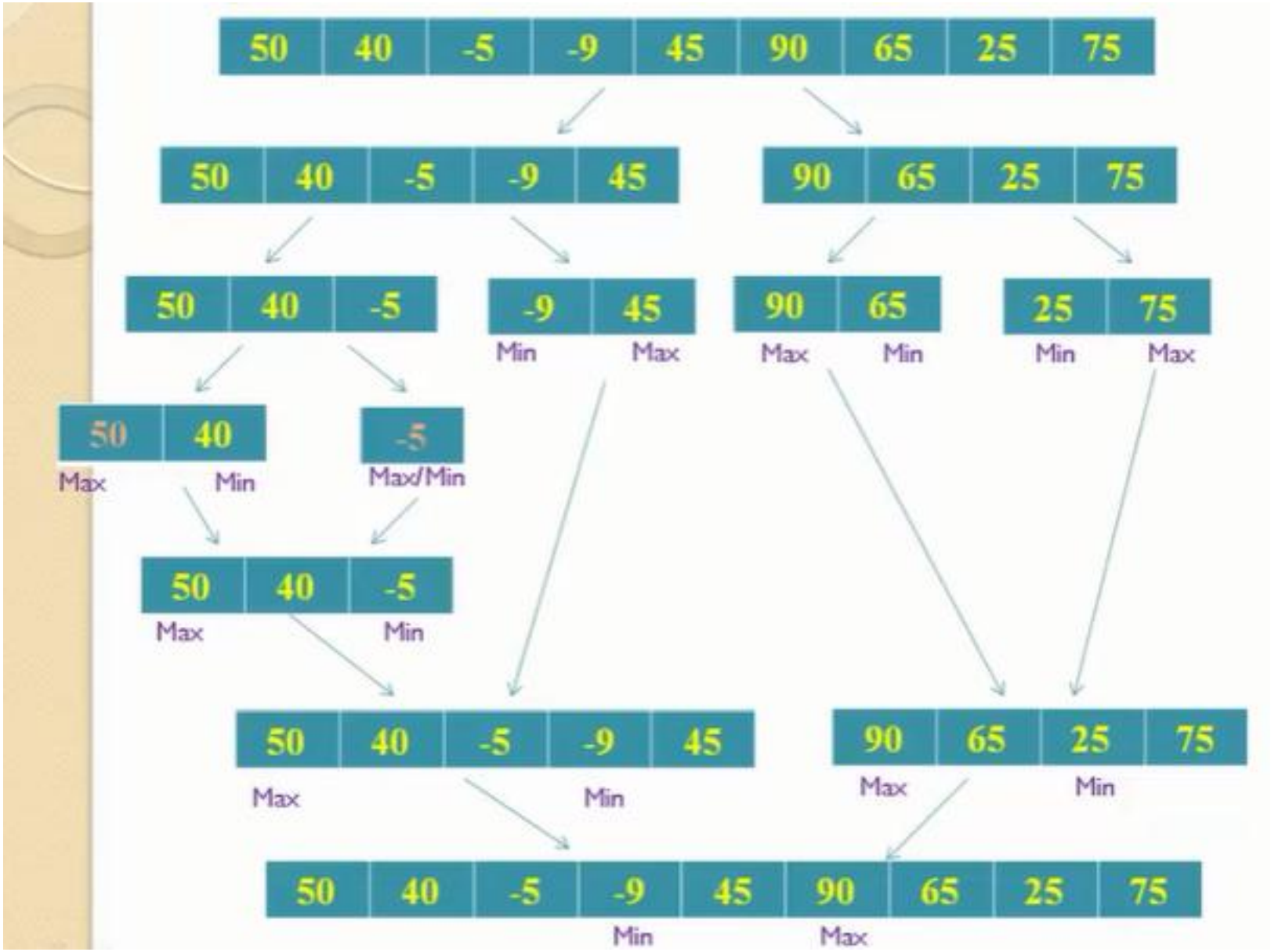
```
Algorithm MinMax(A,n)
{
max=min=A[0];
for(i=1;i<n;i++)
{
     if(A[i]>max)
         max=A[i];
     if(A[i]<min)
         min=A[i];
}
}

Time Complexity: O(n)
```

→ **Recursive Algorithm:**

```
void MinMax(int A[],int i,int j,int &min,int &max)
{
int mid,min1,max1;
if(i==j)
    max=min=A[i];
else if(i==j-1)
{
    if(A[i]<A[j])
    {
    max=A[j];
    min=A[i];
    }
    else
    {
    max=A[i];
    min=A[j];
    }
}
else
{
    mid=(i+j)/2;
    MinMax(A,i,mid,min,max);
    MinMax(A,mid+1,j,min1,max1);
    if(max1>max) max=max1;
    if(min1<min) min=min1;
}
}
```

EXAMPLE OF FINDING MINM AND MAXM



→ The Recurrence relation for the MinMax algorithm is:

$$T(n) = 2T(n/2) + 1 \text{ if } n > 2$$

$$T(n) = 1 \text{ if } n \leq 2$$

→ Solving it by master method, we get the complexity as $O(n)$

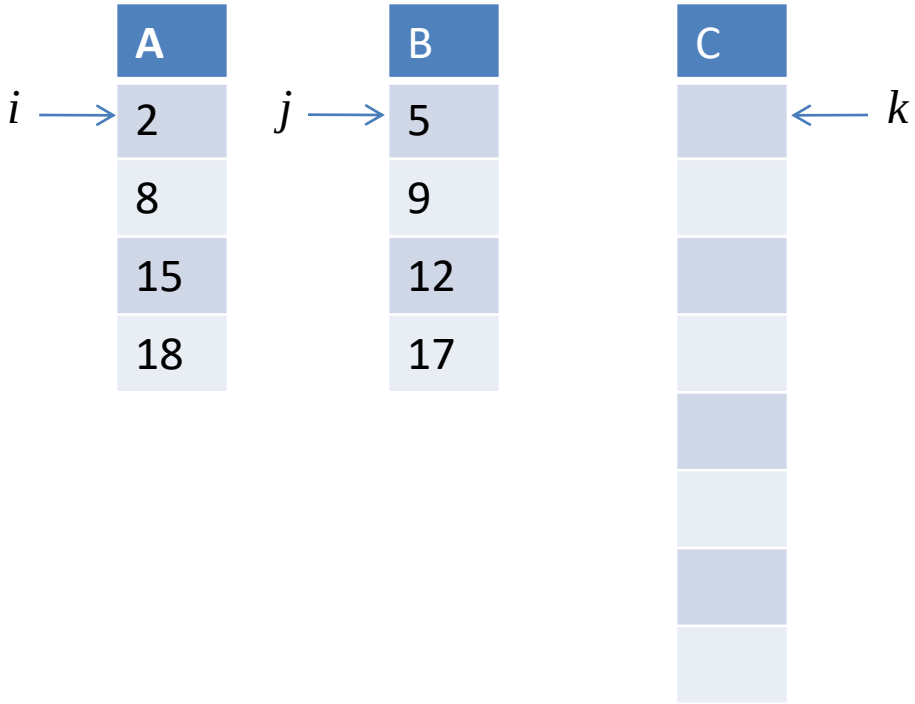
MERGING OF TWO LISTS

Merge operation is used to combine two sorted arrays or lists to get resultant.

Algorithm:

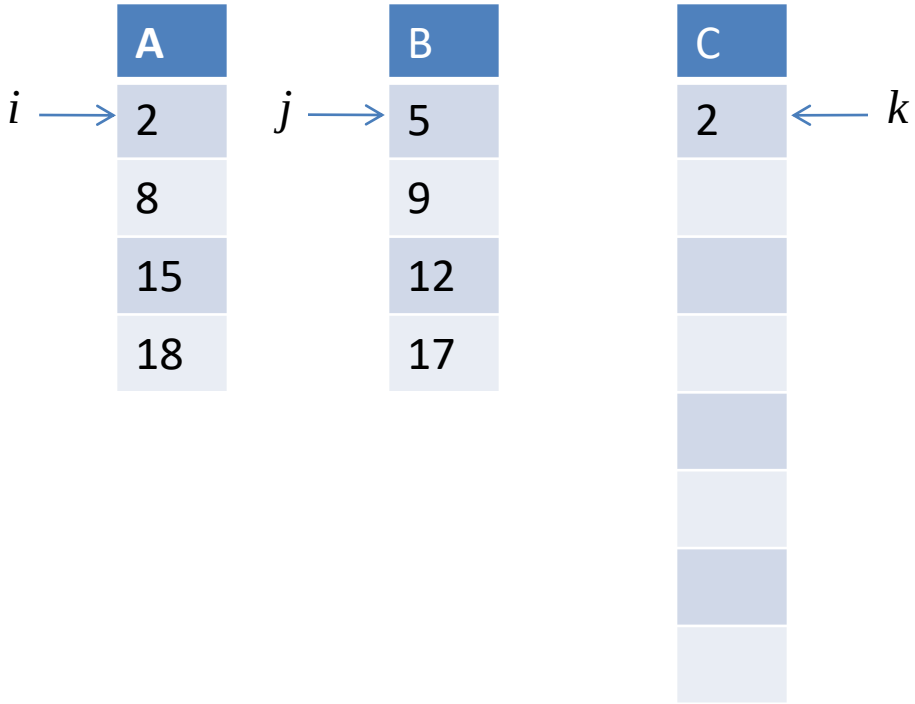
```
Algorithm Merge(A,B,m,n)
{
int i=0,j=0,k=0;//Initialization
while(i<m && j<n)
{
    if(A[i]<B[j])
        C[k++]=A[i++];
    else
        C[k++]=B[j++];
}
for(;i<m;i++)
    C[k++]=A[i];
for(;j<n;j++)
    C[k++]=B[j];
}
```

Example:
A={2,8,15,18}, B={5,9,12,17}



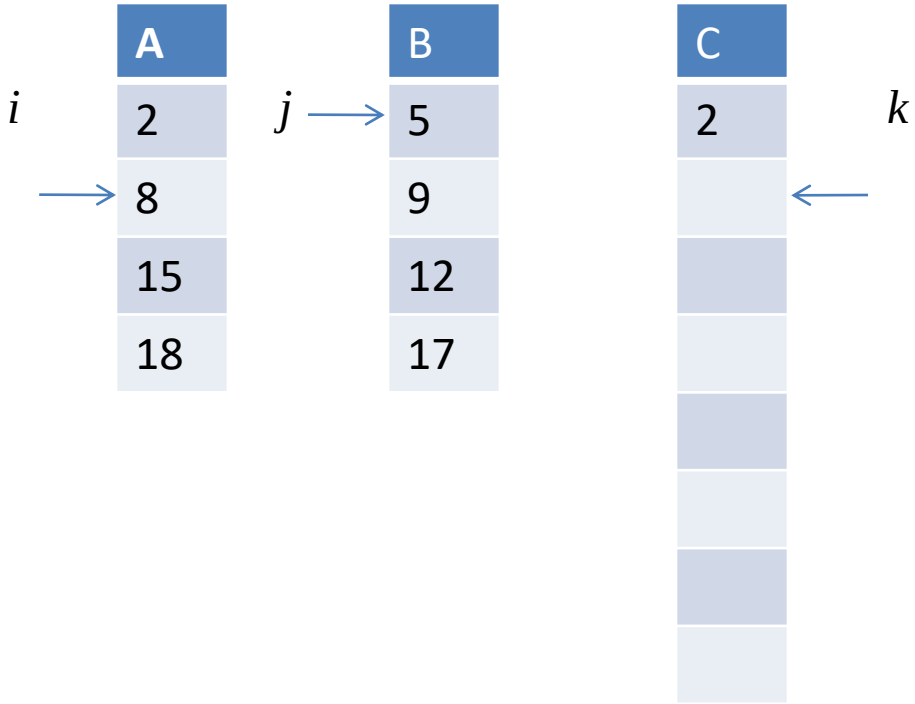

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



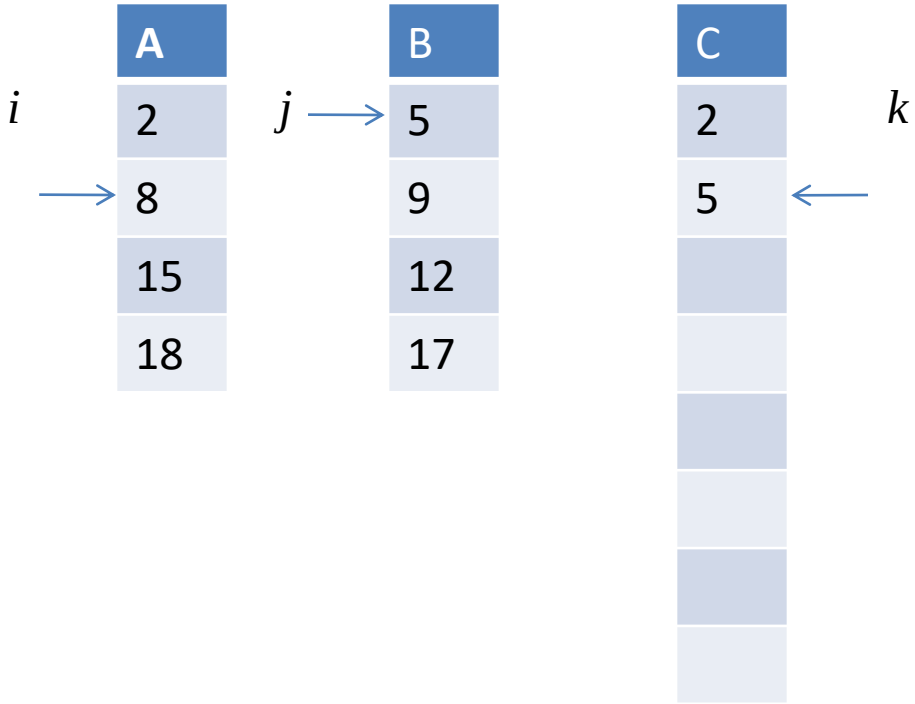

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



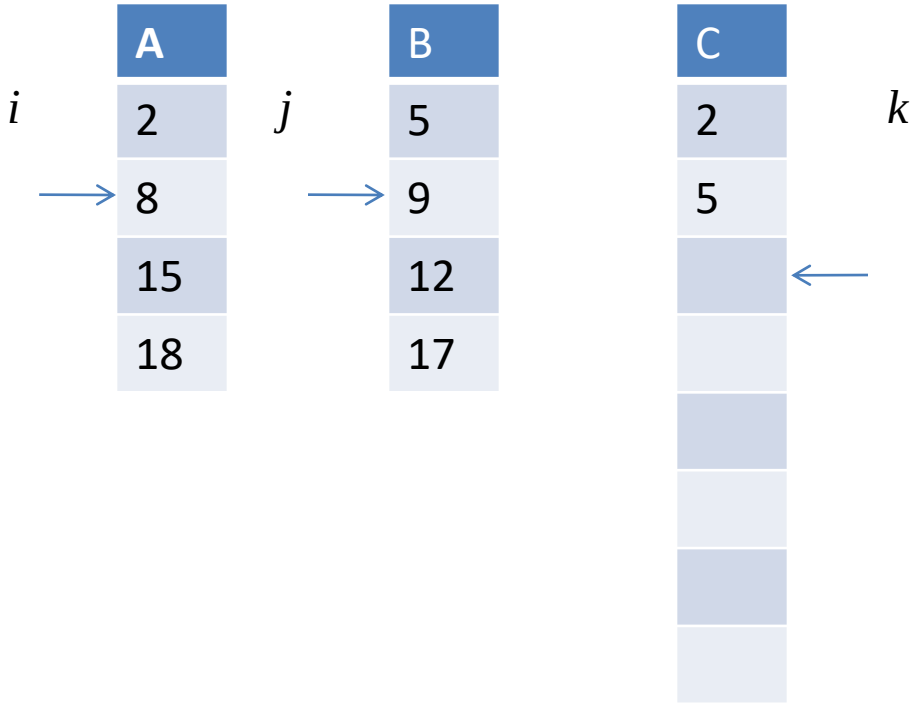

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



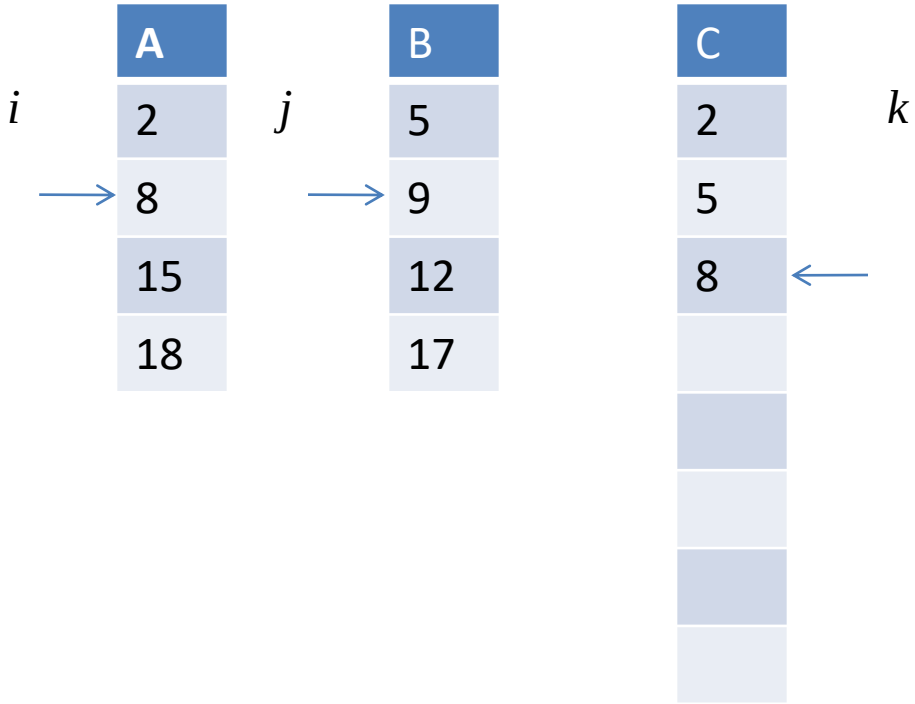

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



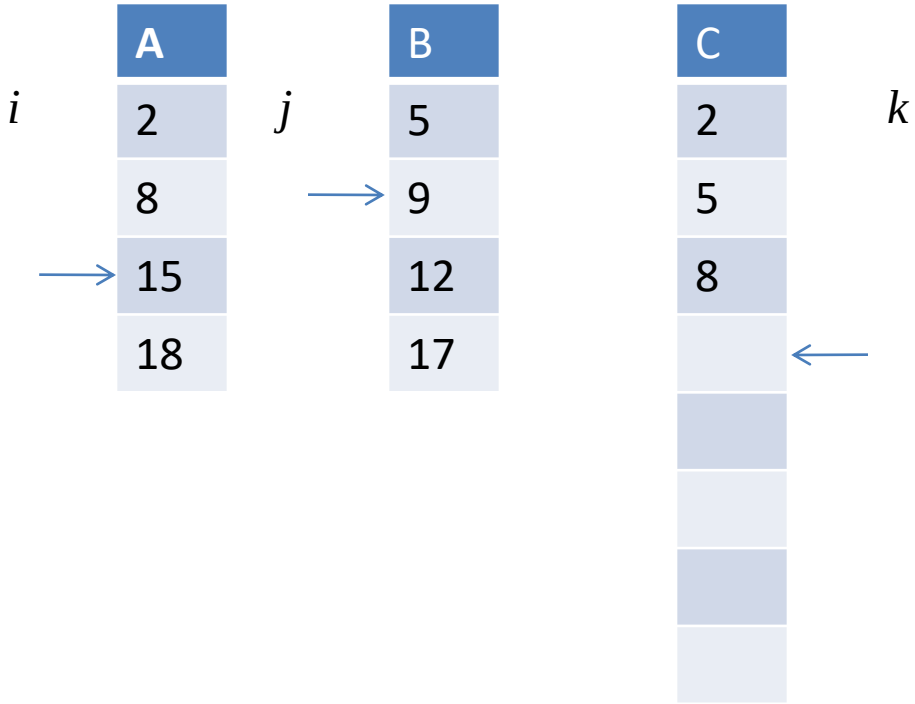

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



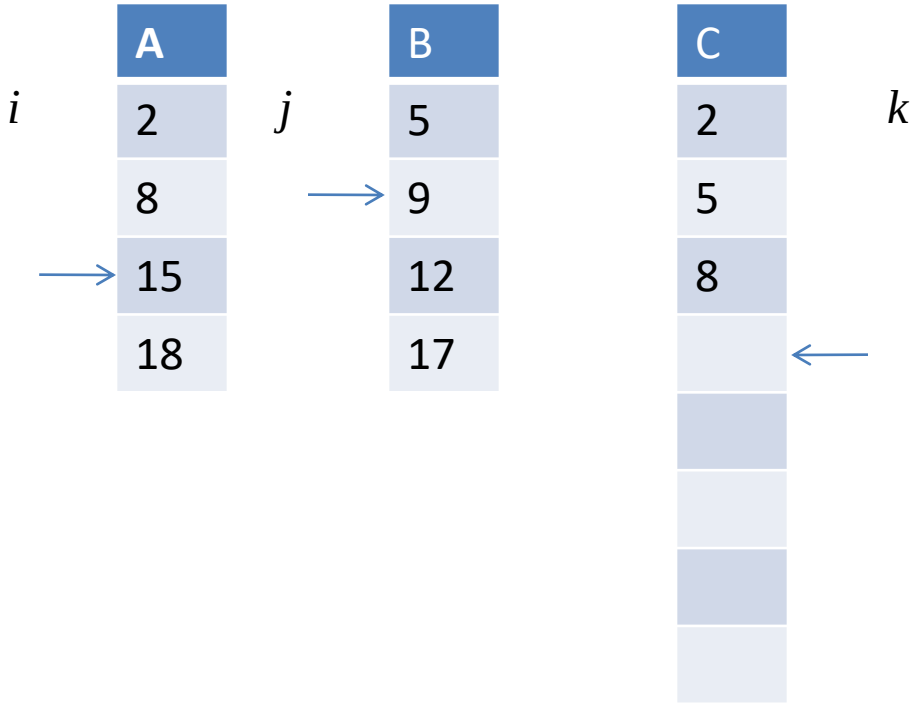

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



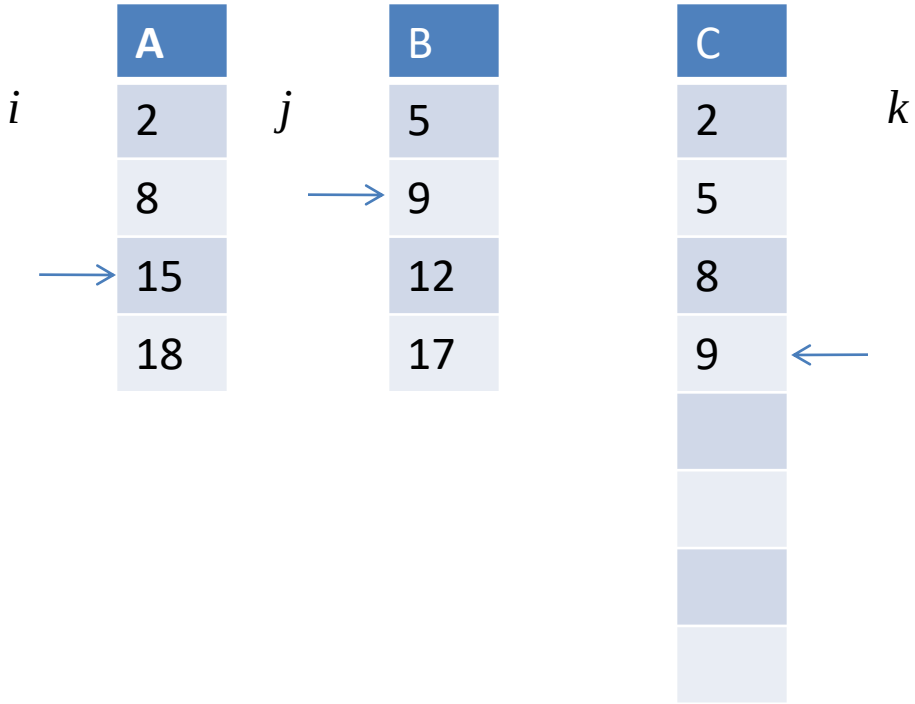

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



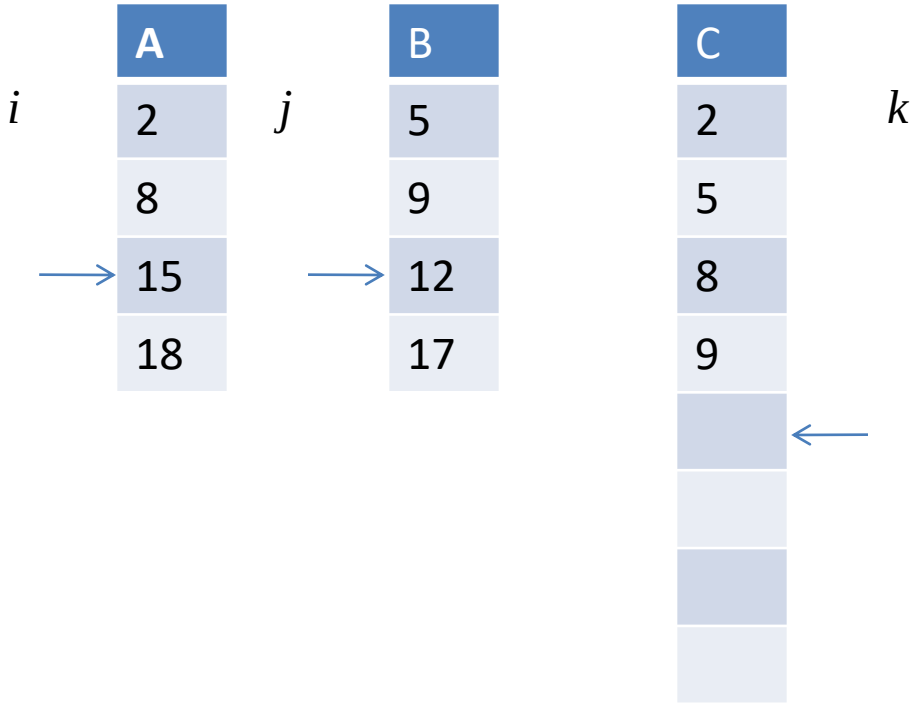

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



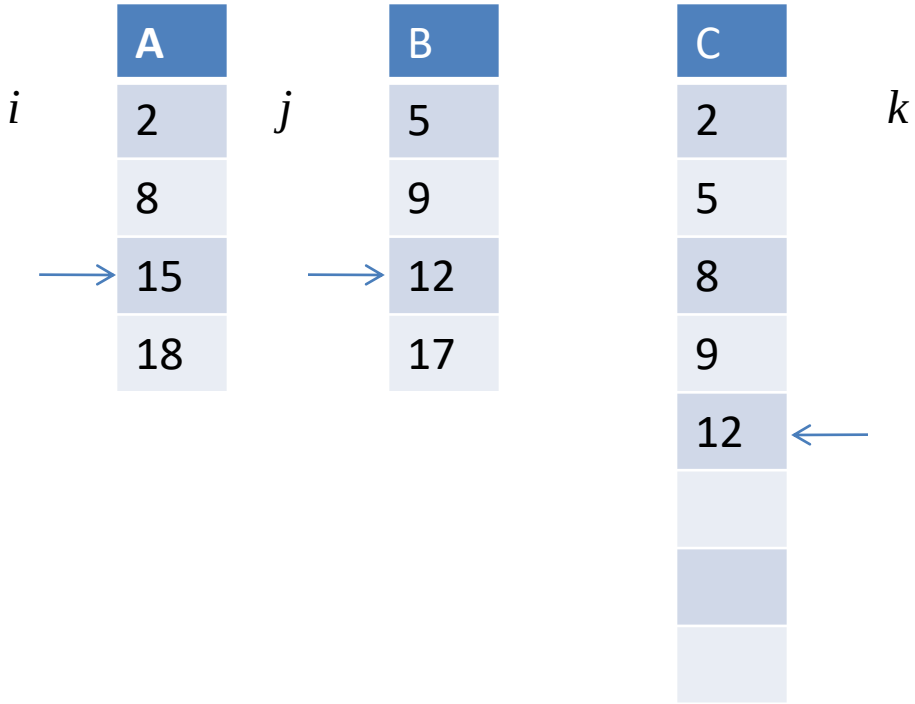

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



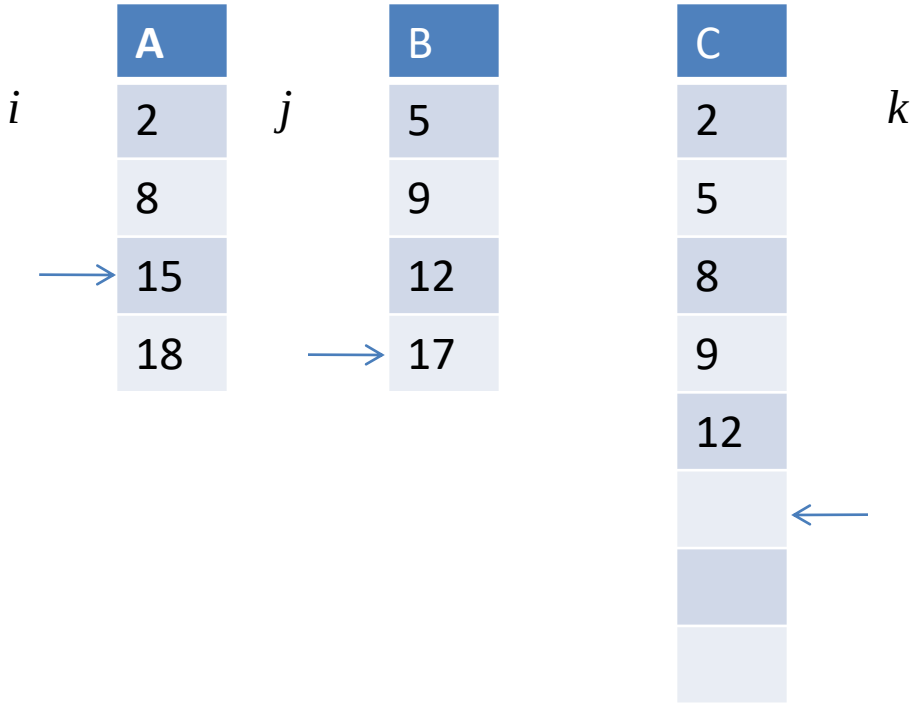

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



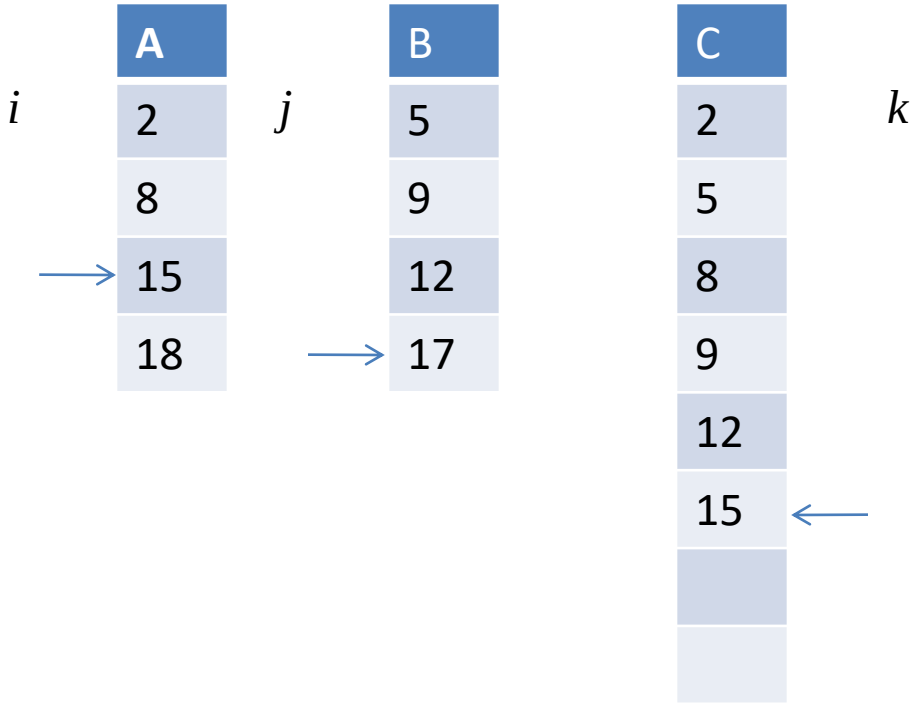

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



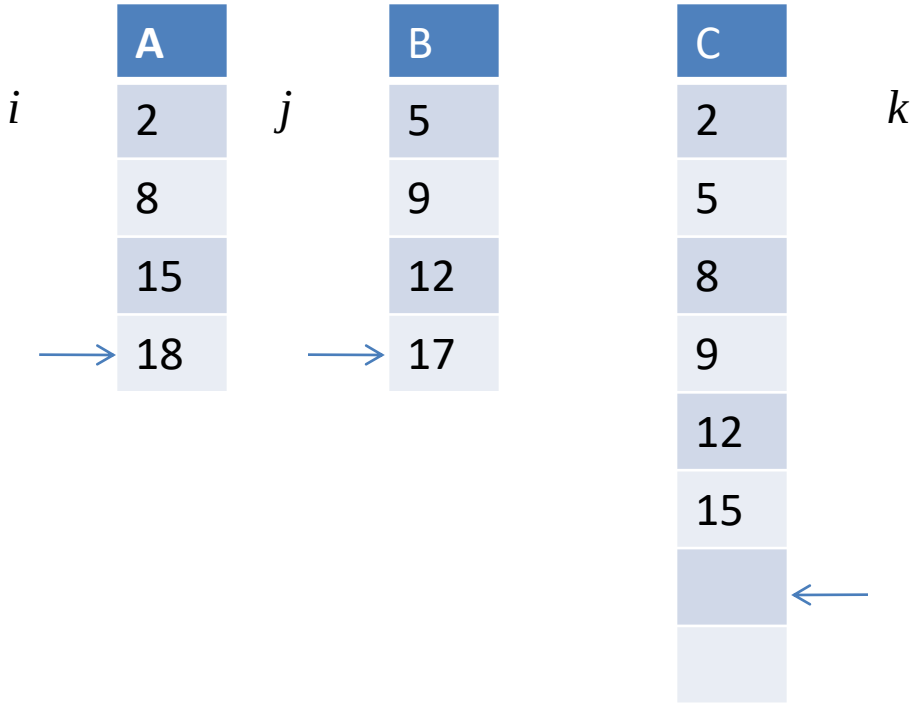

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



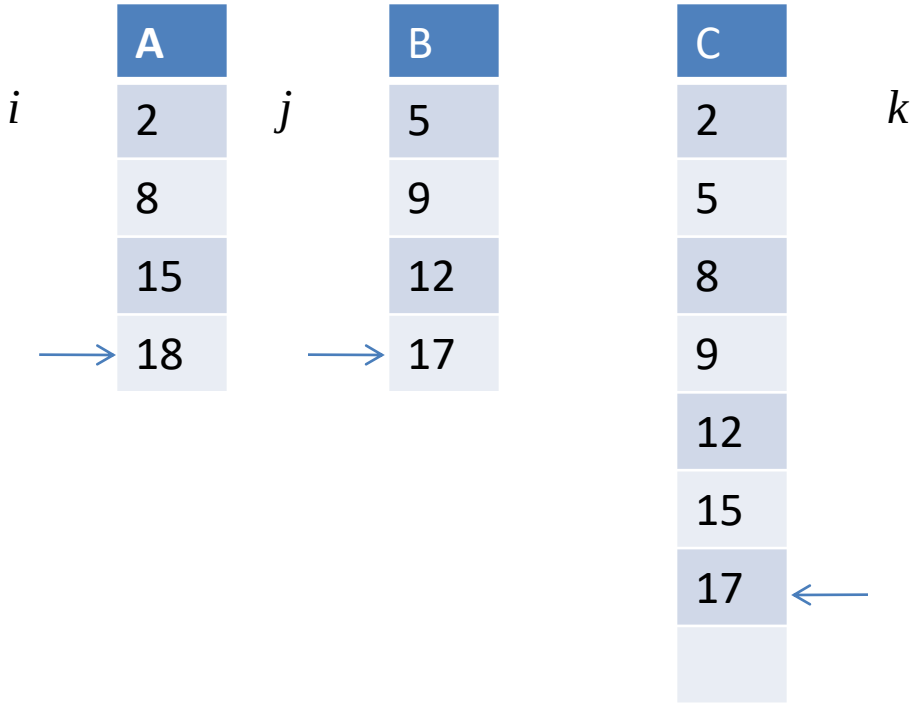

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



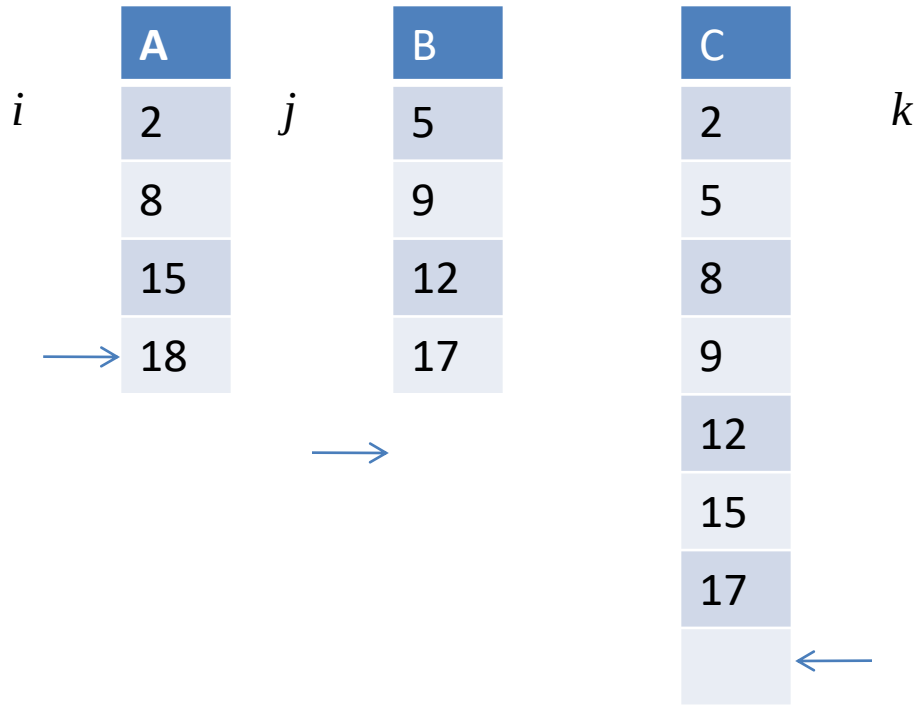

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



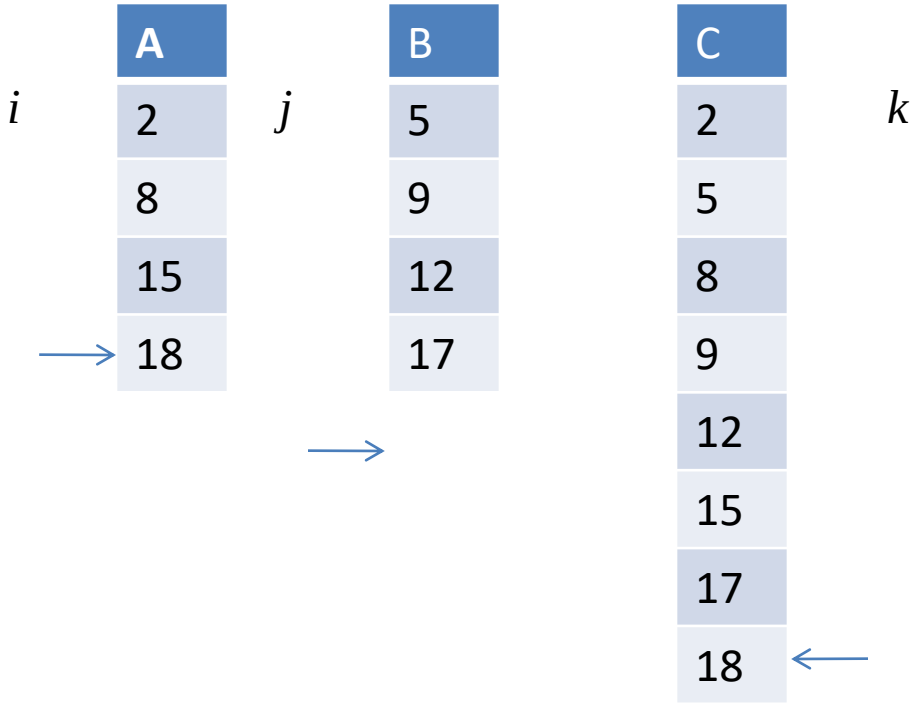

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}



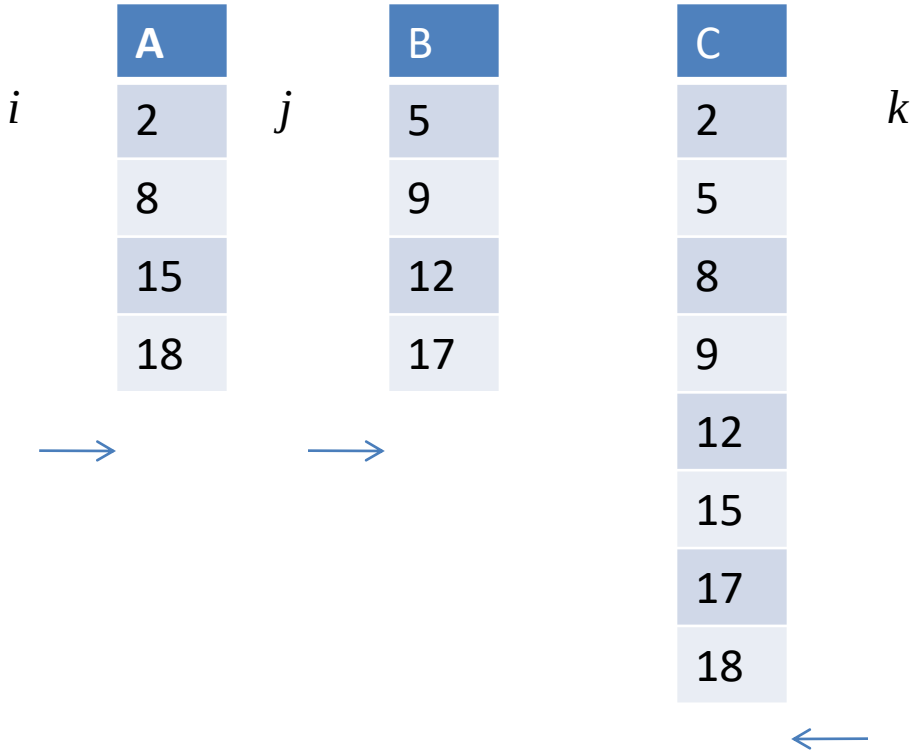

```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}




```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

Example:
A={2,8,15,18}, B={5,9,12,17}




```
if(A[i]<B[j])
      C[k++]=A[i++];
else
      C[k++]=B[j++];
```

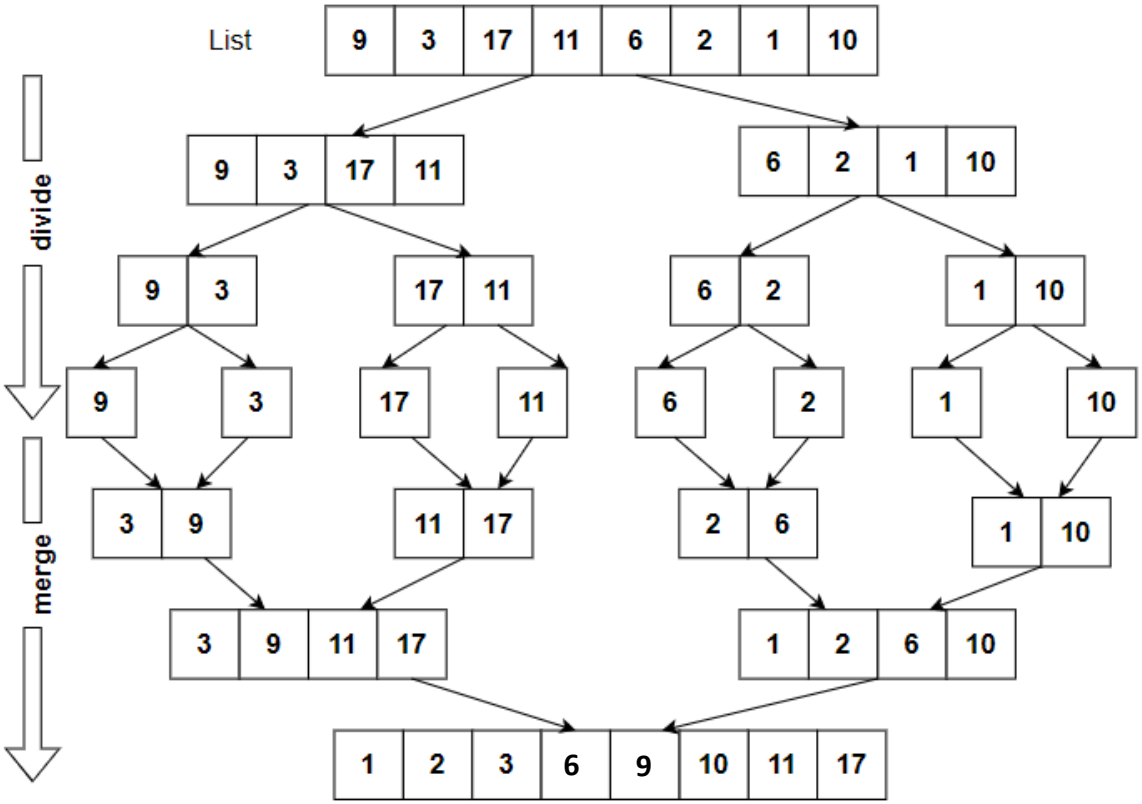
MERGE SORT ALGORITHM

To sort an array A[*l*….h] using merge sort algorithm:

i. **Divide:** Divide the n element sequence to be sorted into two subsequences of n/2 elements each.

ii. **Conquer:** Sort the subsequences recursively using merge sort. When the size of the sequences is *l* there is nothing more to do.

iii. **Combine:** Merge the two sorted subsequences.



Input: Unsorted list
Output: Sorted list
Algorithm:

```
Algorithm MergeSort(low,high)
{
        if(low<high)
        {
        mid=(low+high)/2;
        MergeSort(low,mid);
        MergeSort(mid+1,high);
        Merge(low,mid,high);
        }
}
```

```
Algorithm Merge(low,mid,high)
{
i=low;
j=mid+1;
k=0;
if(A[i]<=A[j])
        temp[k++]=A[i++];
else
        temp[k++]=A[j++];
for(;i<=mid;i++)
        temp[k++]=A[i];
for(;j<=high;j++)
        temp[k++]=A[j];
for(i=low;i<=high;i++)
        A[i]=temp[i];
}
```

Time Complexity:

Recurrence relation for merge sort is :

$$T(n) = \begin{cases} 1 & ,if\ n = 1 \\ 2T(n/2) + O(n) & ,if\ n > 1 \end{cases}$$

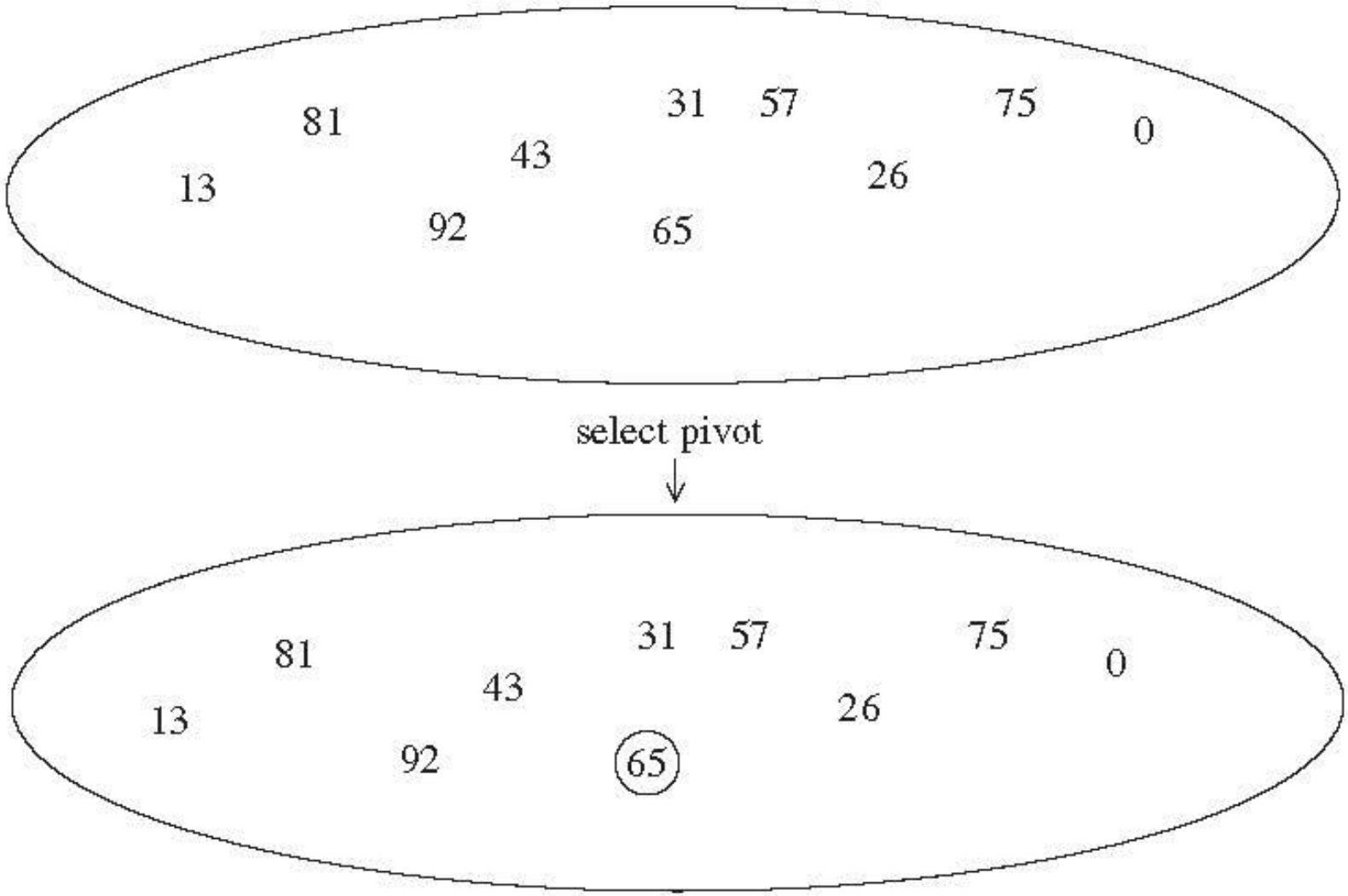
Upon solving the above RR, we get,

Time Complexity $= O(n \lg n)$

Space Complexity:

It uses one extra array and some extra variables during sorting, therefore Space complexity = $2n$ + c = $O(n)$

QUICK SORT



QUICK SORT

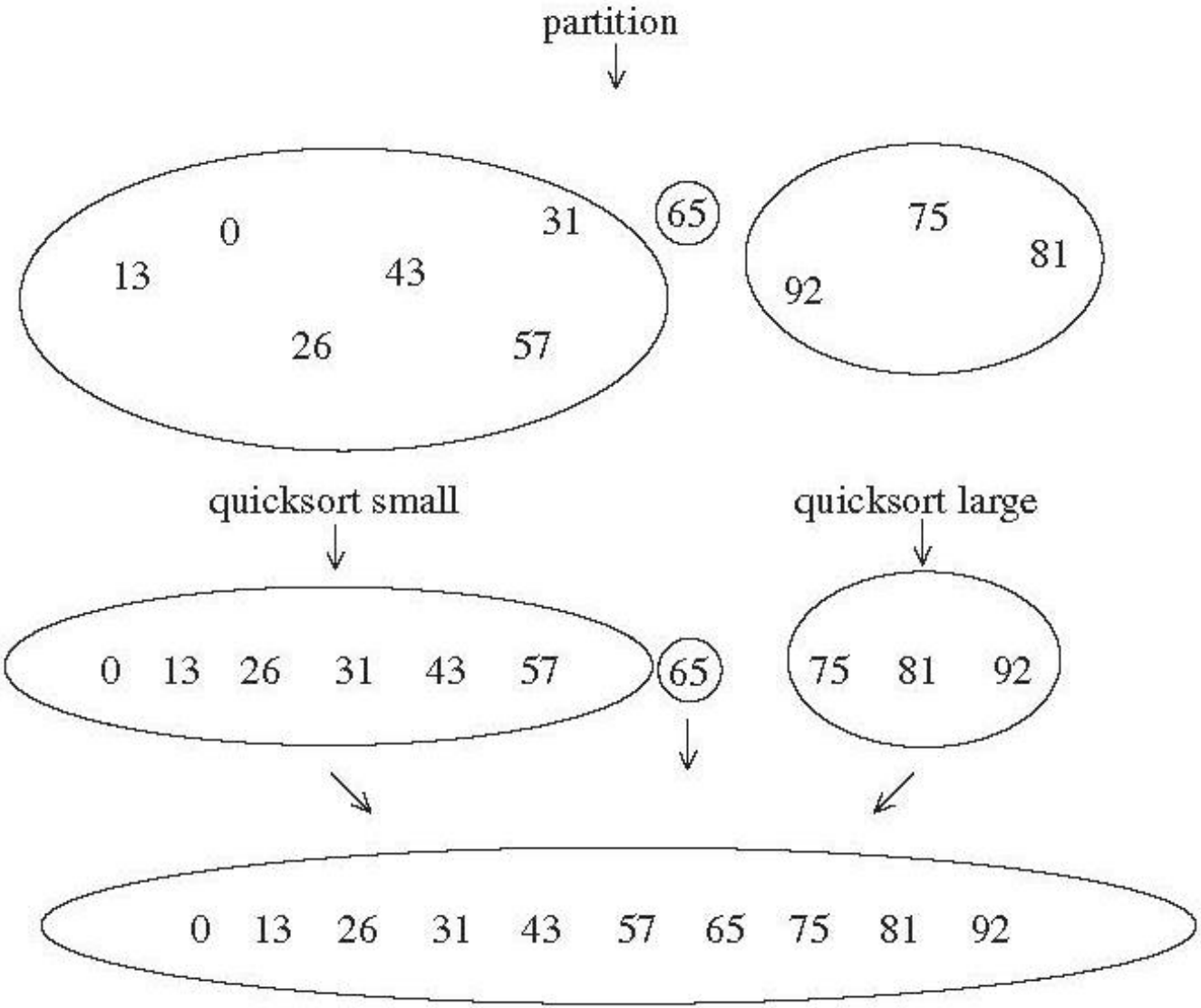
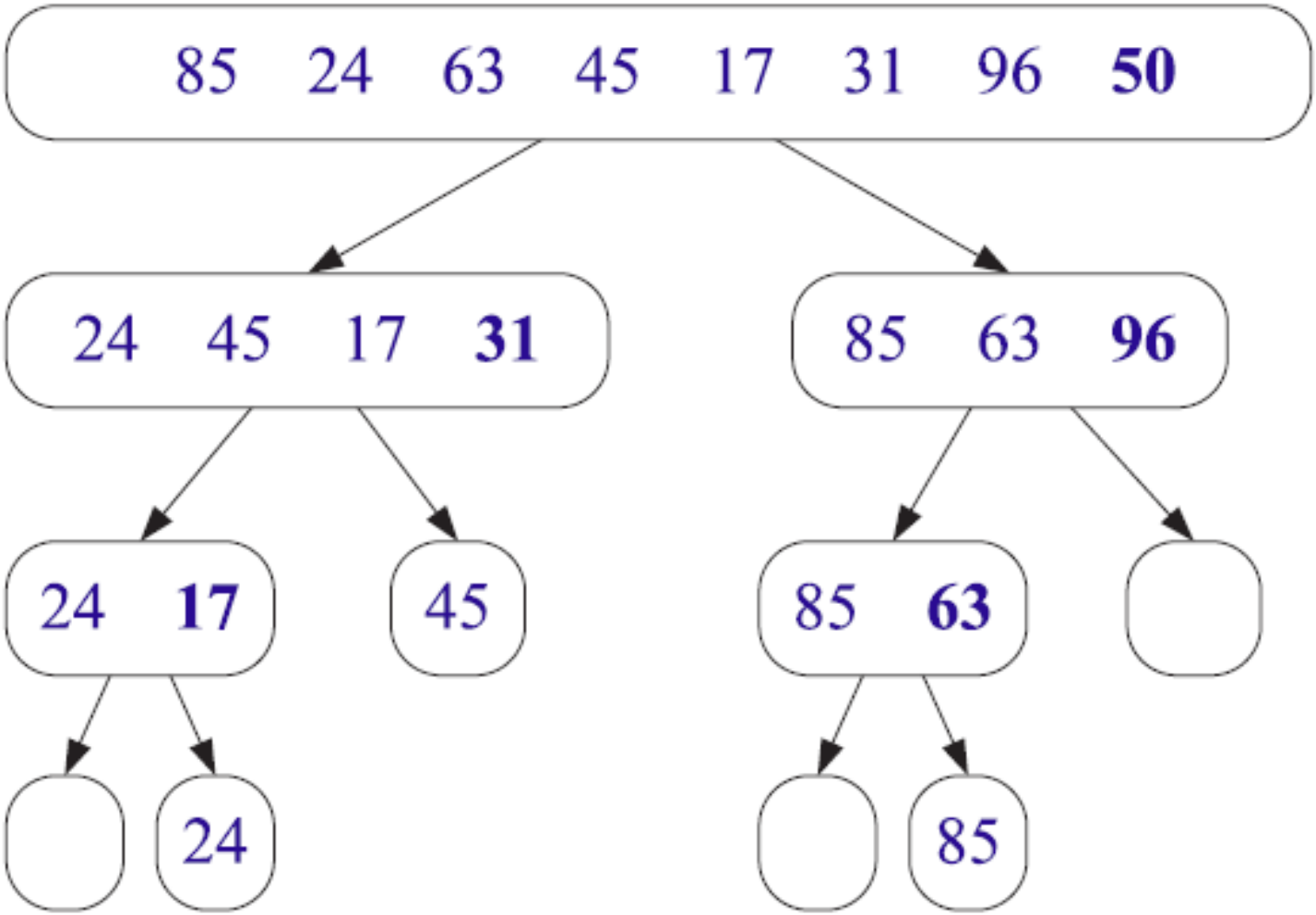


Figure The steps of quicksort illustrated by example

QUICK SORT

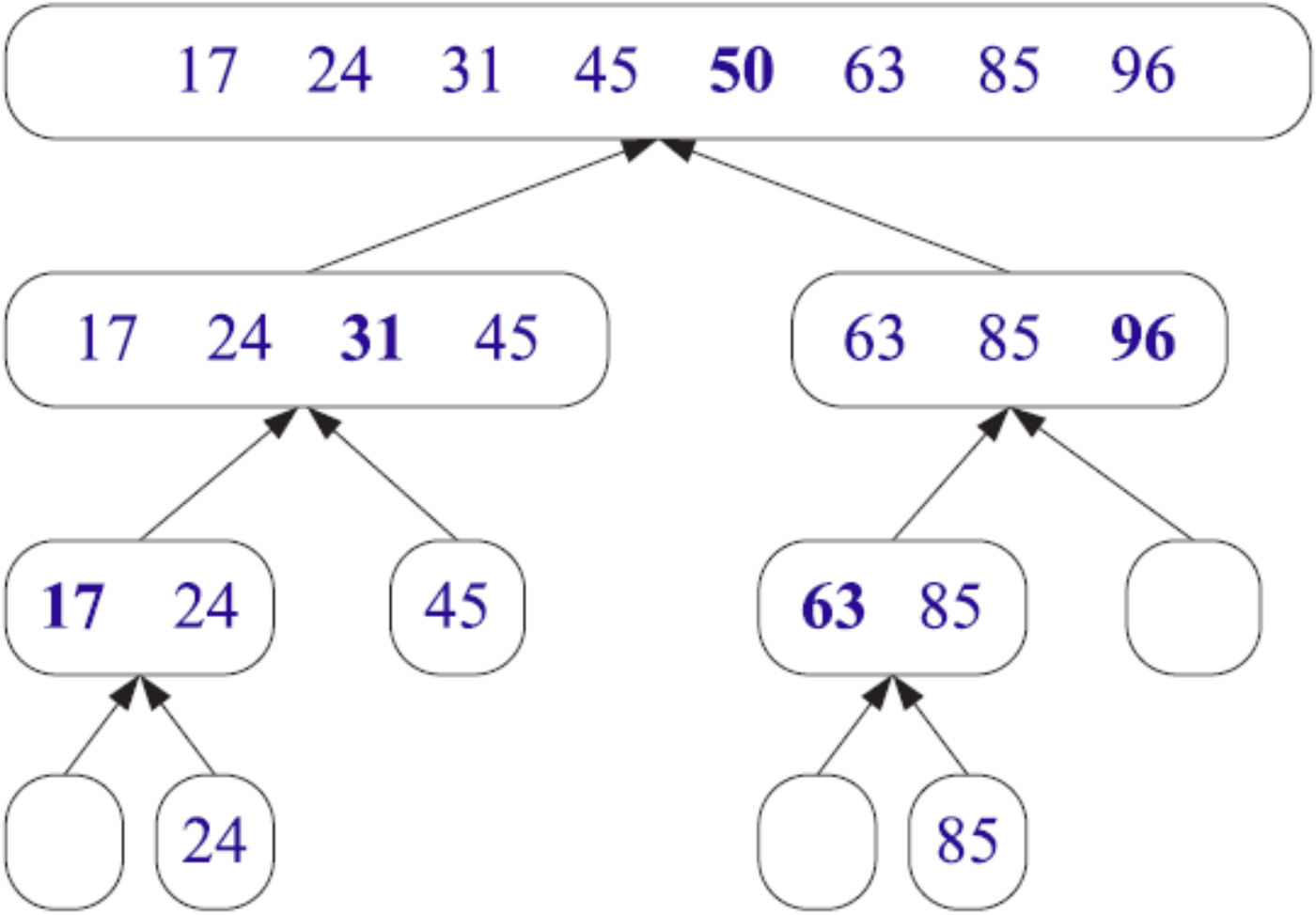
Data = {85, 24, 63, 45, 17, 31, 96, 50}



Sorted Data = {17, 24, 31, 45, 50, 63, 85, 96}

QUICK SORT

Data = {17, 24, 31, 45, 50, 63, 85, 96



Sorted Data = {17, 24, 31, 45, 50, 63, 85, 96}

QUICK SORT

Quick Sort is a divide and conquer type sorting algorithm that uses following steps for sorting:

Divide:

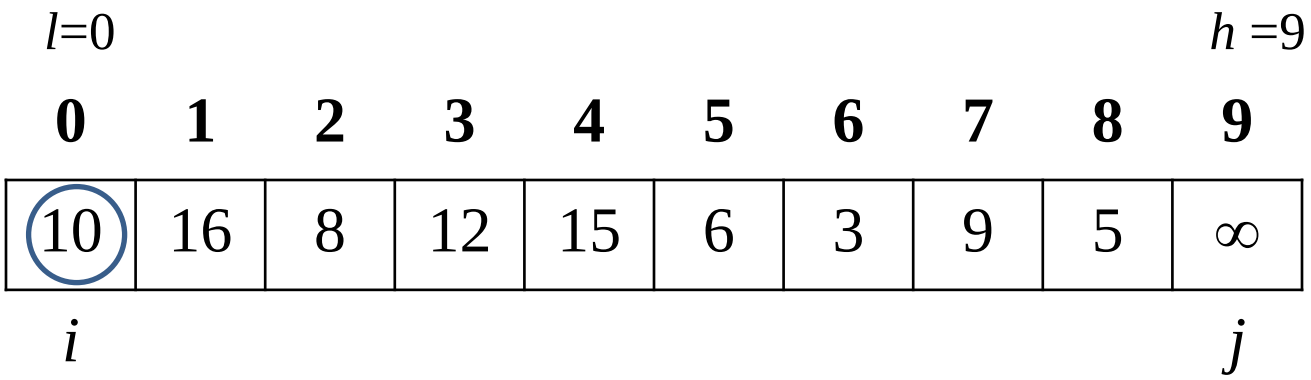
Partition the array A[*l*….h] into 2 subarrays A[*l*..*p*-1] and A[*p*+1..*h*], such that each element of A[*l*….*p*-1] is smaller than or equal to each element in A[*p*+1…h]. Need to find index *p* to partition the array.

Conquer:

Recursively sort A[*l*…*p*-1] and A[*p*+1 ….*h*] using quick sort

Combine:

Trivial: The arrays are sorted in place. No additional work is required to combine them.



Pivot Element = A[*l*]=10

Start *i* from *i* = *l*
Start *j* from *j* = *h*

0	1	2	3	4	5	6	7	8	9
10	16	8	12	15	6	3	9	5	∞
	i							*j*	

Pivot Element = 10

Repeat until i<j is true
{
Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]
}

0	1	2	3	4	5	6	7	8	9
10	5	8	12	15	6	3	9	16	∞
	i							*j*	

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	12	15	6	3	9	16	∞
		i					*j*		

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	12	15	6	3	9	16	∞
			i				*j*		

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	9	15	6	3	12	16	∞
			i				*j*		

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	9	15	6	3	12	16	∞
				i		*j*			

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	9	3	6	15	12	16	∞
				i		*j*			

Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	9	3	6	15	12	16	∞

i *j*

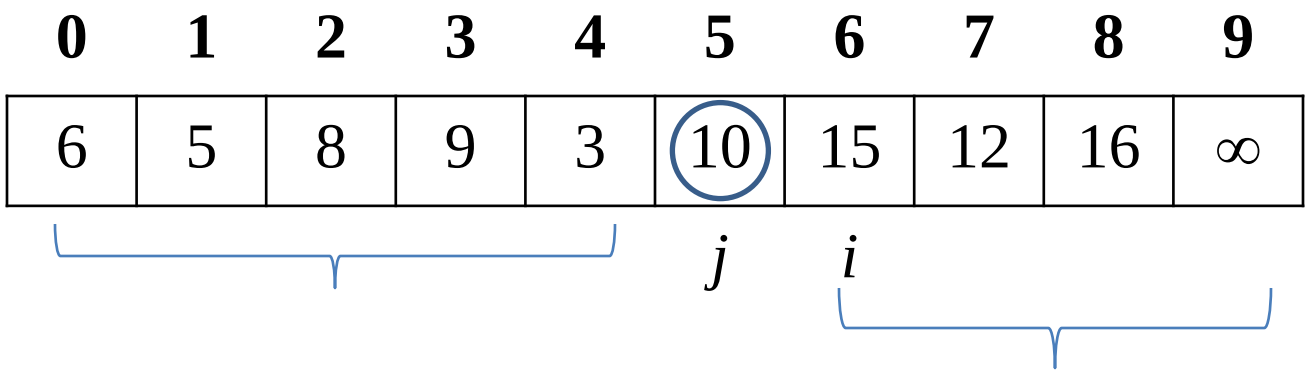
Pivot Element = 10

Increment *i* until you find the element greater than the pivot element.
Decrement *j* until you find the element less than the pivot element.
Swap the value of A[i] and A[j]

0	1	2	3	4	5	6	7	8	9
10	5	8	9	3	6	15	12	16	∞
					j	*i*			

Pivot Element = 10

i<j is false
Swap A[*l*] and A[*j*]



Pivot Element = 10

i<j is false

So

New pivot position = j = 5

Algorithm:

```
Algorithm QuickSort(A, l, h)
{
   if(l < h)
   {
   p=Partition(A,l,h);
   QuickSort(A,l,p-1);
   QuickSort(A,p+1,h);
   }
}
```

```
Algorithm Partition(A, l, h)
{
i=l;
j=h+1;
p=A[l];
   while(i<j)
   {
      do
      {
      i++;
      }while(A[i]<=p);
      do
      {
      j--;
      }while(A[j]>p);
      if(i<j)
         swap(A[i],A[j]);
   }
   A[l]=A[j];A[j]=p;
   return j; //position of the pivot
}
```

Time Complexity:

We can notice that the complexity of the partitioning is $O(n)$ because the outer while loop executes cn times.

Thus, recurrence relation for the quick sort is:

$T(n) = T(k) + T(n\text{-}k\text{-}1) + O(n)$

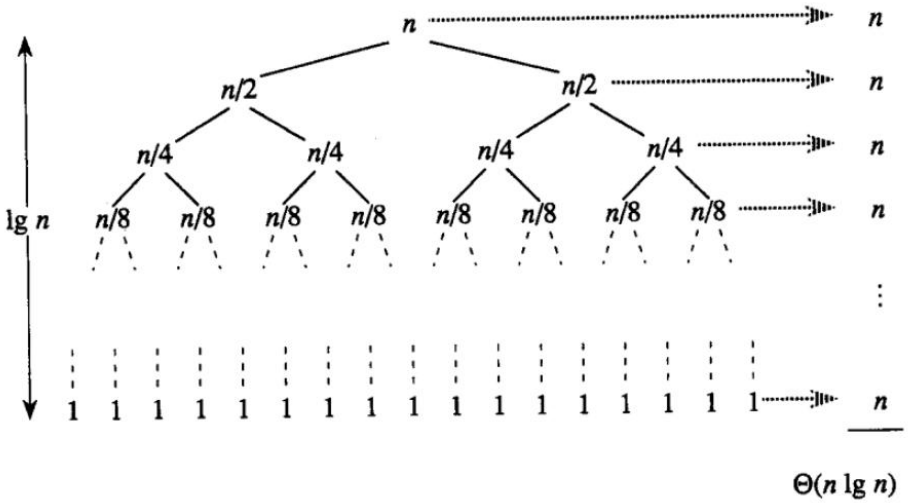
Best Case:

Divides the array into two partitions of equal size, therefore,

$T(n) = 2T(n/2) + O(n)$

Solving this recurrence relation, we get,

Time Complexity = $O(n \lg n)$



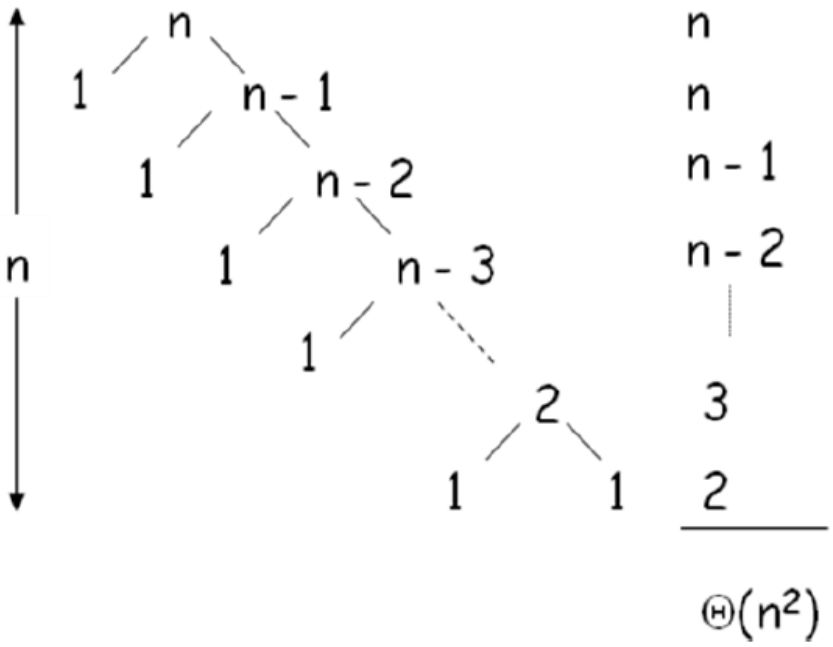
Worst Case:

When array is already sorted or sorted in reverse order, one partition contains n-1 items and another contains zero items, therefore,

$T(n) = T(n\text{-}1) + O(n)$

Solving this recurrence relation ,we get,

Time Complexity = $O(n^2)$



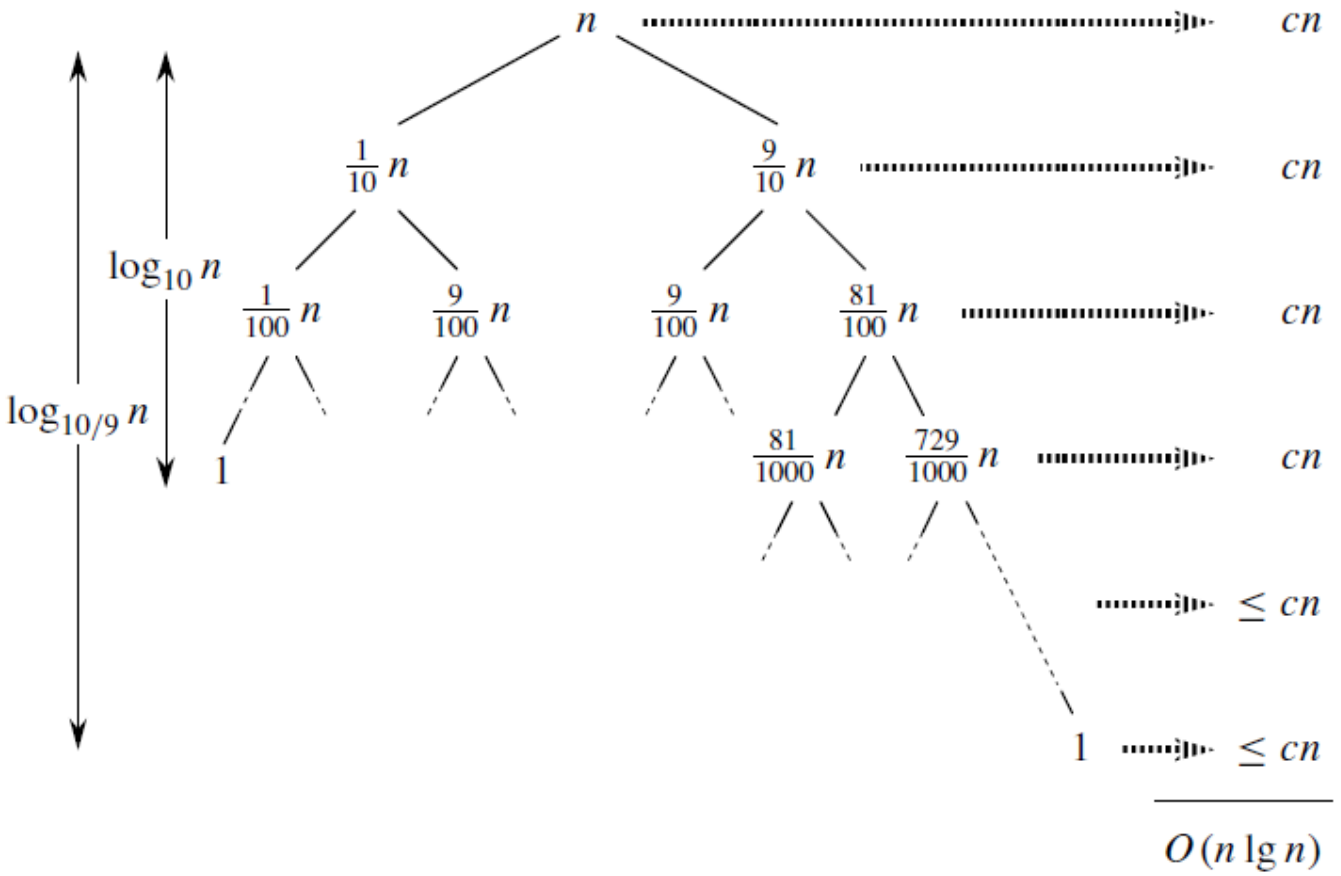
Case in between Best Case and Worst Case:

Suppose the array is partitioned in a 9-1 split.

$T(n) = T(9n/10) + T(n/10)$

Solving this recurrence relation, we get,

Time Complexity = $O(n \lg n)$



Average Case:

All permutations of the input numbers are equally likely. On a random input array, we will have a mix of well balanced and unbalanced splits. Good and bad splits are randomly distributed across throughout the tree

Suppose we are alternate: Balanced, Unbalanced, Balanced, ….

$B(n) = 2UB(n/2) + O(n)$ Balanced

$UB(n) = B(n\text{-}1) + O(n)$ Unbalanced

Solving,

$B(n) = 2\ B(n/2\text{-}1) + O(n)$

$= O(n \log n)$

RANDOMIZED QUICK SORT

- The algorithm is called randomized if its behavior depends on input as well as random value generated by random number generator. The beauty of the randomized algorithm is that no particular input can produce worst-case behavior of an algorithm.
- IDEA: Partition around a random element. Running time is independent of the input order. No assumptions need to be made about the input distribution. No specific input elicits the worst-case behavior.
- The worst case is determined only by the output of a random-number generator.
- Randomization cannot eliminate the worst-case but it can make it less likely!

Algorithm:

```
Algorithm RandQuickSort(A,l,h)
{
     if(l<h)
     {
     p=RandPartition(A,l,h);
     RandQuickSort(A,l,p-1);
     RandQuickSort(A,p+1,h);
     }
}

Algorithm RandPartition(A,l,h)
{
r = random(l,h);
swap(A[l],A[r]);
return Partition(A,l,h);
}
```

Note: Partition() function same as the one used in Quick Sort.

ORDER STATISTIC

The ith order statistic of a set of n elements is the ith smallest element in that set.

If $i = 1$, we are taking the minimum.

If $i = n$, we are taking the maximum,

When n is an odd number, position of the median i is defined by $(n+1)/2$

Selection After Sorting

Using an $O(n \log n)$ Algorithm

Assumption

Suppose n elements are sorted by an $O(n \log n)$ algorithm, e.g., MERGE-SORT.

Then the following properties hold for the ordered set

- Minimum: The first element.
- Maximum: The last element.
- The ith order statistic corresponds to the ith element.
- For the median:
  - If n is odd, then the median is equal to the $\frac{n+1}{2}$ th element.
  - If n is even:
    - ★ The lower median is equal to the $\lfloor \frac{n+1}{2} \rfloor$ th element.
    - ★ The upper median is equal to the $\lceil \frac{n+1}{2} \rceil$ th element.

CAN THE SELECTION BE BETTER THAN O(N LOG N) ?

SELECTION IN EXPECTED LINEAR TIME

Algorithm:

```
Algorithm RandSelect(A, l, h, i)
{
   if (l == h)
      return A[l];
   p = RandPartition(A, l, h);  // pivot position
   k = p - l + 1;               // rank of pivot in A[l..h]
   if (i == k)
      return A[p];
   else if (i < k)
      return RandSelect(A,  l, p-1, i);
   else
      return RandSelect(A, p+1, h, i - k);
}
```

Note: RandPartition() and Partition() function same as the one used in randomized Quick Sort.

EXPLANATION

Base case:

If the array has only one element (l == h), return it that's the answer.

Pick a random pivot:

p = RandPartition(A, l, h)

chooses a random pivot and partitions the array around it.

Find pivot's rank

$k = p - l + 1$ gives you the pivot's position (rank) in the current subarray.

Compare with desired rank i

If i == k

It means the pivot itself is the ith smallest, return it.

If $i < k$

the ith smallest lies in the left half, search the left.

If $i > k$

the ith smallest lies in the right half, search the right.

Selection Problem

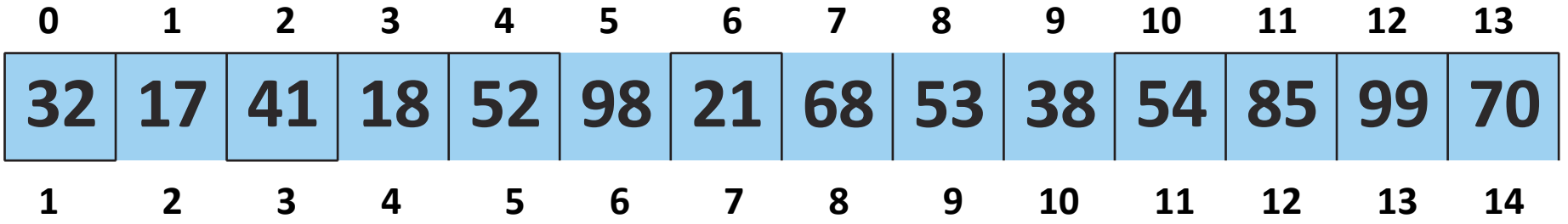
Input:

A set A of n (distinct) numbers and an integer i, with $1 \leq i \leq n$.

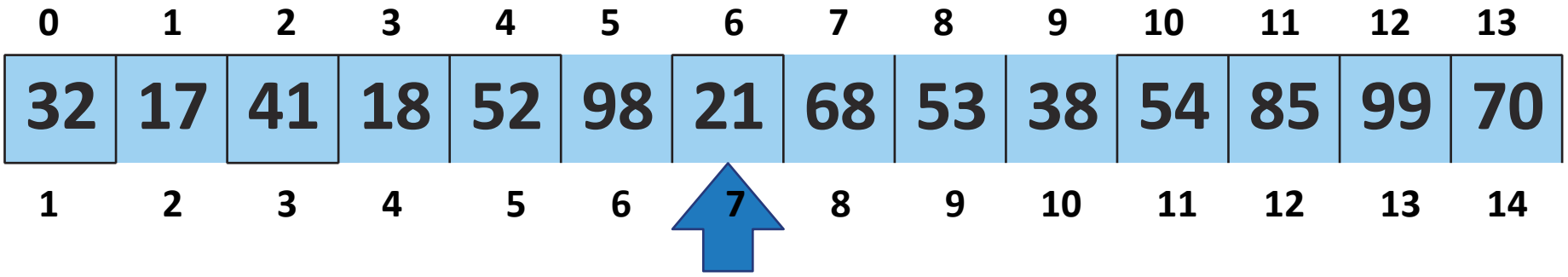
Output:

The element $x \in A$ that is larger than exactly $i - 1$ other elements of A.

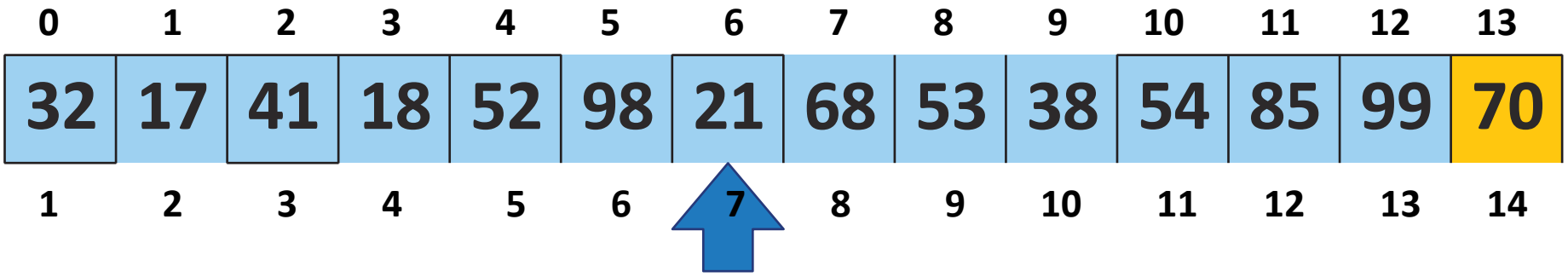
Partition-Based Selection



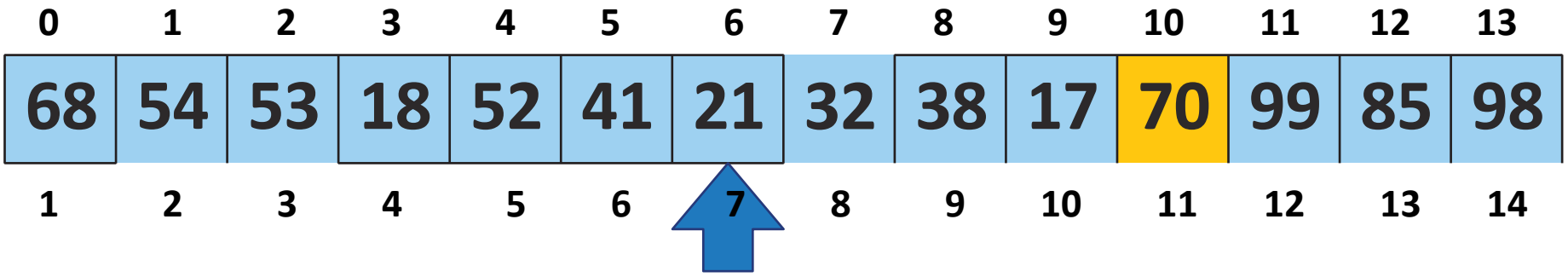
Partition-Based Selection



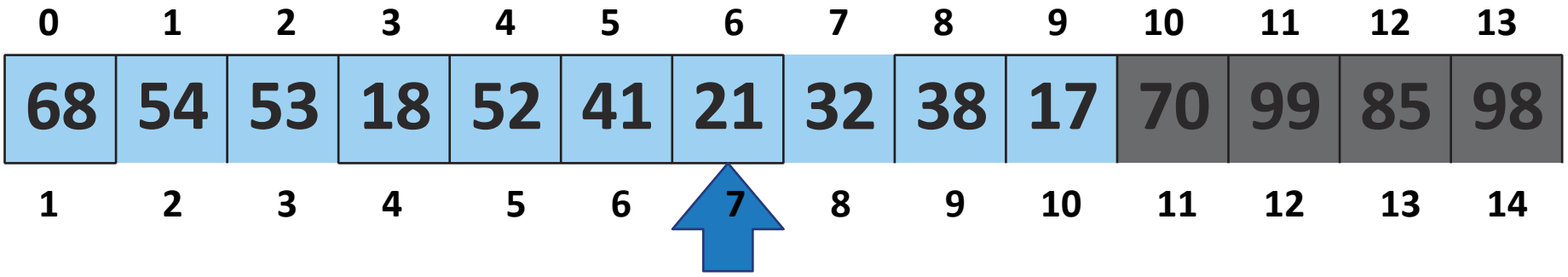
Partition-Based Selection



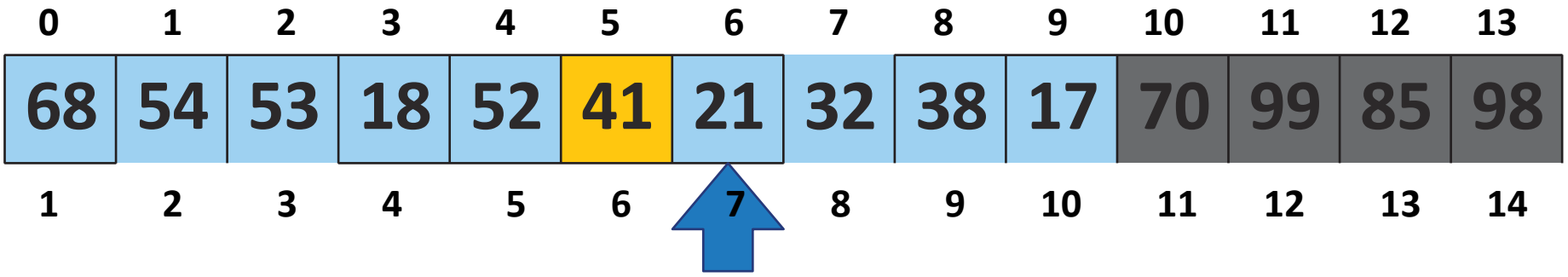
Partition-Based Selection



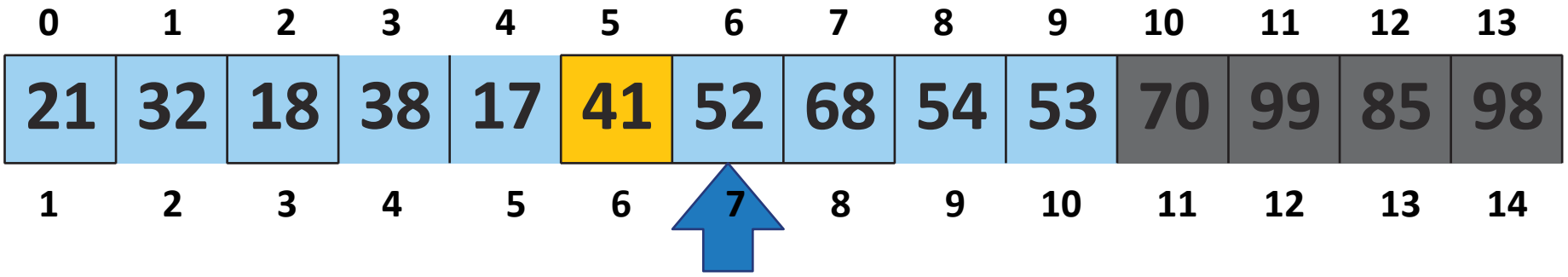
Partition-Based Selection



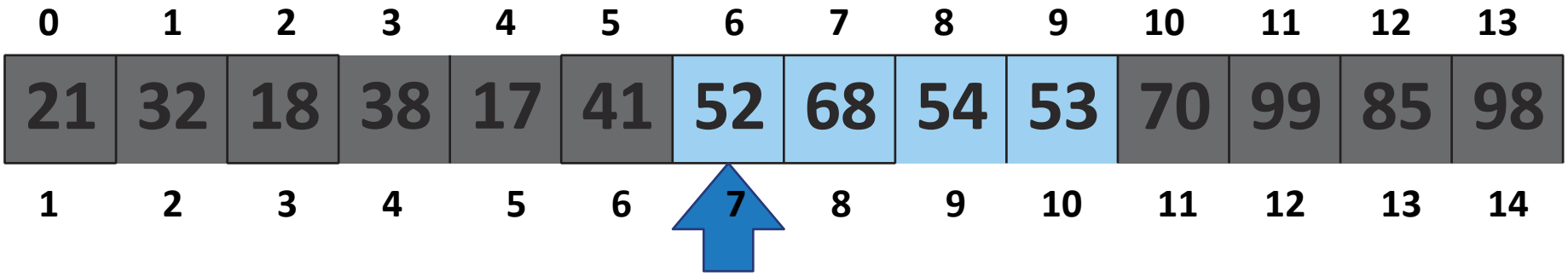
Partition-Based Selection



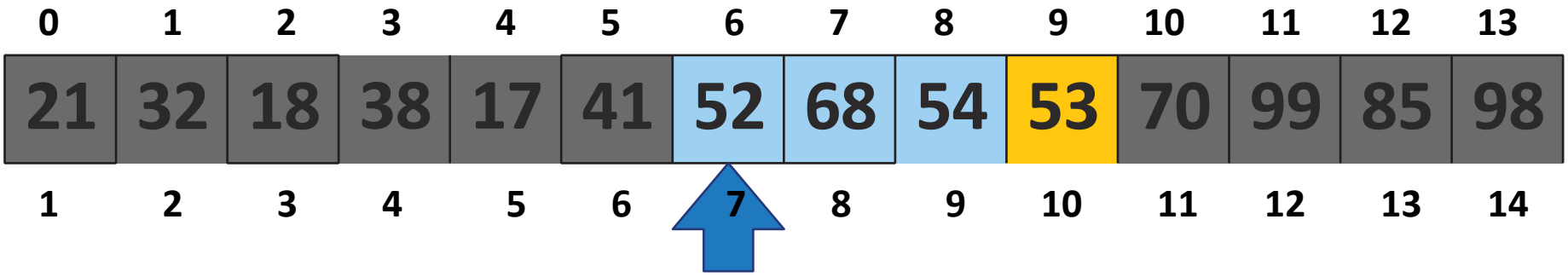
Partition-Based Selection



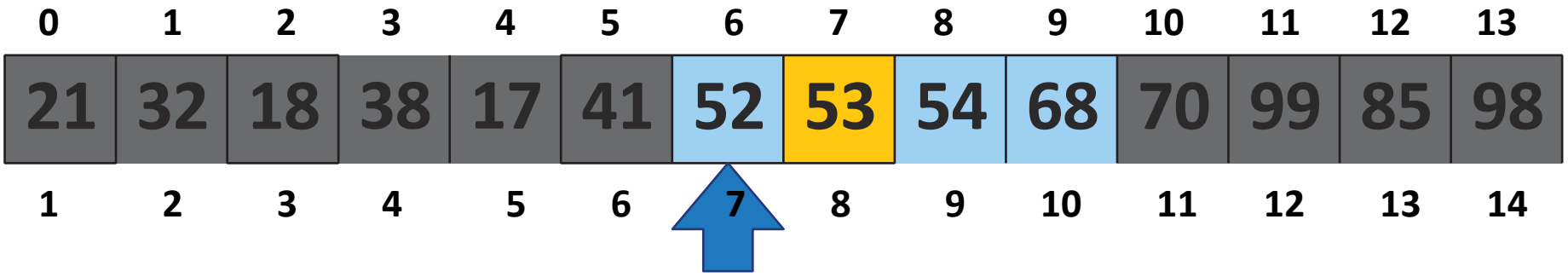
Partition-Based Selection



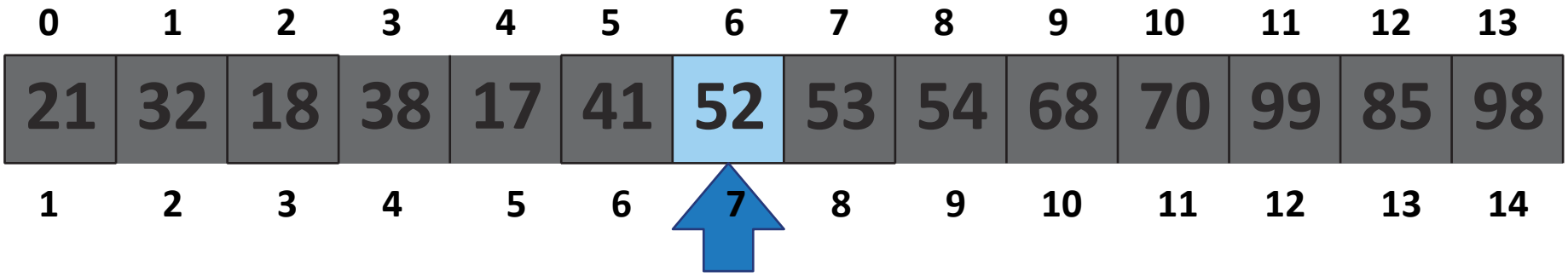
Partition-Based Selection



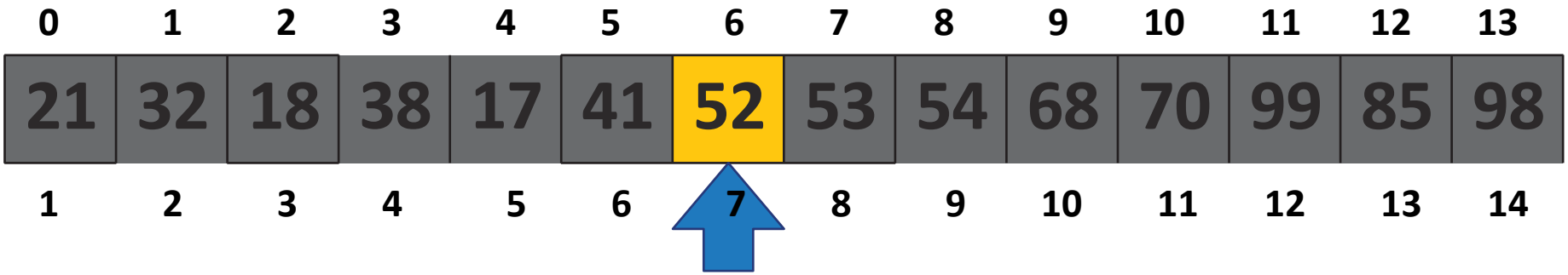
Partition-Based Selection



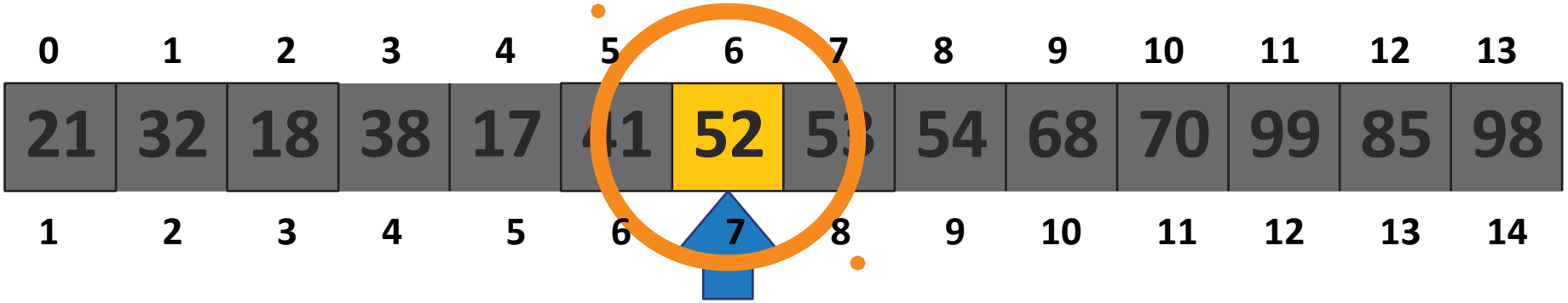
Partition-Based Selection



Partition-Based Selection



Partition-Based Selection



Selection in Expected Linear Time
Time Complexity

Recurrence Relation:

$T(n) = T(n/2) + O(n)$

Solution:

$T(n) = O(n)$

Three-Subset Method Example

- Input: Array A = {24, 63, -5, 8, 1}
- Objective: Find the 3rd smallest element using the Three-Subset Method.
- Approach:
  - Randomly select a pivot.
  - Partition the array into three subsets.
  - Analyze subsets to prune and search recursively.

Key Steps of the Three-Subset Method

1. Step 1: Choose Pivot (Randomly select 8 as pivot).
   - Partition:
     - S1 (less than pivot): {-5, 1}
     - S2 (equal to pivot): {8}
     - S3 (greater than pivot): {24, 63}

2. Step 2: Analyze Subsets.
   - Sizes: |S1| = 2, |S2| = 1, |S3| = 2.
   - Rank of pivot: Rank = |S1| + 1 = 3.

3. Step 3: Compare Rank.
   - Since Rank = 3 (matches desired position), 8 is the 3rd smallest element.

- Final Answer: The 3rd smallest element is 8.

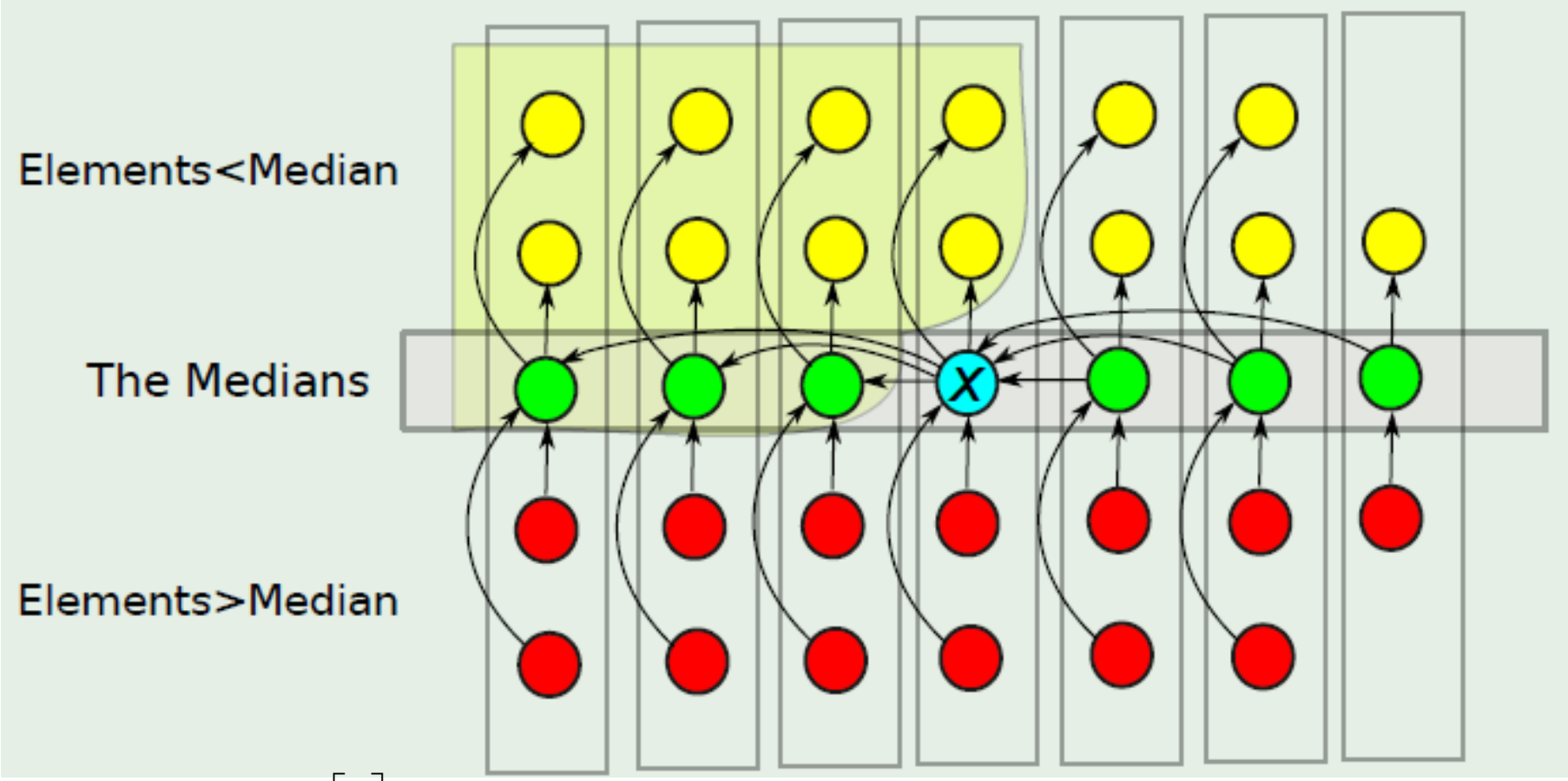
Verification:

- Sorted array: {-5, 1, 8, 24, 63}.
- 3rd smallest element in sorted array is 8.

SELECTION IN WORST CASE LINEAR TIME

Step 1. Start
Step 2. Divide the array into groups of 5 elements.
Step 3. Sort each group and find the median of each group.
Step 4. Collect all medians into a new array M.
Step 5. Find the median of M (this is the median of medians).
Step 6. Partition the original array using this median as pivot.
Step 7. Find the rank k of the pivot after partitioning.
Step 8. If $k = i$, the pivot is the required element.
Step 9. If $i < k$, repeat the algorithm on the left subarray.
Step 10. If $i > k$, repeat the algorithm on the right subarray.
Step 11. Output the required element.
Step 12. Stop

Selection in Worst Case Linear Time



There are at least $\left\lceil \frac{n}{5} \right\rceil$ no. of groups.

There are at least $\left\lfloor \frac{n}{5} \right\rfloor$ no. of groups having 5 elements.

SELECTION IN WORST CASE LINEAR TIME

Thus, at least half of the $\left\lceil \frac{n}{5} \right\rceil$ groups contribute at least 3 elements that are greater than x, except for the one group that has fewer than 5 elements if 5 does not divide n exactly, and the one group containing x itself. Discounting these two groups, it follows that the number of elements greater than x is at least:

$$3\left(\left\lceil \frac{1}{2}\left\lceil \frac{n}{5} \right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6$$

Similarly at least $\frac{3n}{10} - 6$ elements are less than x.

Thus, in the worst case, RandSelect will be called on at most $\frac{7n}{10} + 6$ elements.

Therefore, the recurrence relation is:

$$T(n) \leq \begin{cases} O(1) & \text{if } n < 140 \\ T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{10} + 6\right) + O(n) & \text{if } n \geq 140 \end{cases}$$

Its solution is: $O(n)$ in worst case.

SOLUTION

We show that the running time is linear by substitution. More specifically, we will show that $T(n) \leq cn$ for some suitably large constant c and all $n > 0$. We begin by assuming that $T(n) \leq cn$ for some suitably large constant c and all $n < 140$; this assumption holds if c is large enough. We also pick a constant a such that the function described by the $O(n)$ term above (which describes the non-recursive component of the running time of the algorithm) is bounded above by an for all $n > 0$. Substituting this inductive hypothesis into the right-hand side of the recurrence yields

$$\begin{aligned} T(n) &\leq c\lceil n/5 \rceil + c(7n/10 + 6) + an \\ &\leq cn/5 + c + 7cn/10 + 6c + an \\ &= 9cn/10 + 7c + an \\ &= cn + (-cn/10 + 7c + an) , \end{aligned}$$

Source:
(Cormen et al., 2022)

which is at most cn if

$$-cn/10 + 7c + an \leq 0 . \tag{9.2}$$

Inequality (9.2) is equivalent to the inequality $c \geq 10a(n/(n-70))$ when $n > 70$. Because we assume that $n \geq 140$, we have $n/(n-70) \leq 2$, and so choosing $c \geq 20a$ will satisfy inequality (9.2). (Note that there is nothing special about the constant 140; we could replace it by any integer strictly greater than 70 and then choose c accordingly.) The worst-case running time of SELECT is therefore linear.

SELECTION IN WORST CASE LINEAR TIME

Example: Find the 7th smallest element in

A = {12, 3, 5, 7, 4, 19, 26, 23, 2, 15, 9, 21, 30, 11, 6}

Solution:

Step 1: Divide into groups of 5

G1 = {12, 3, 5, 7, 4}, median of G1 = 5

G2 = {19, 26, 23, 2, 15}, median of G2 = 19

G3 = {9, 21, 30, 11, 6}, median of G3 = 11

Medians array = {5, 19, 11}

Step 2: Median of medians

Median({5, 19, 11}) = 11 → pivot = 11

Step 3: Partition around pivot

Left (<11) = {3, 5, 7, 4, 2, 9, 6} (7 elements)

Pivot = 11 (rank = 8 overall)

Right (>11) = {12, 19, 26, 23, 15, 21, 30}

Step 4: Decide direction

We want the 7th smallest.

Pivot is 8th smallest → target lies in left side.

SELECTION IN WORST CASE LINEAR TIME

Step 5: Search on left side

Left side = {3, 5, 7, 4, 2, 9, 6}

Sorted left side = {2, 3, 4, 5, 6, 7, 9}

7th smallest = 9

Final Answer: The 7th smallest element is 9

Thank You!

Any Queries?

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 3. Divide and Conquer Algorithms

Session 2

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 3. Divide and Conquer Algorithms

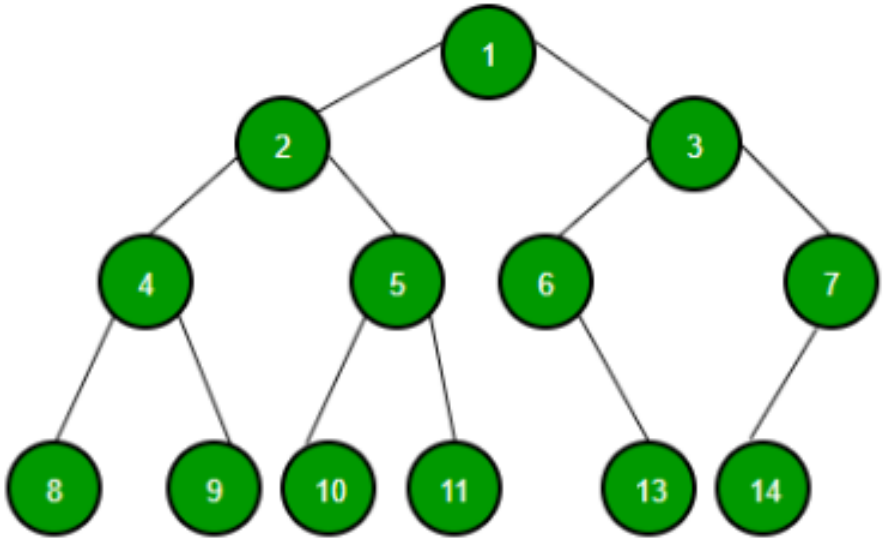
8 hours

3.1 Binary search, min max finding algorithm
3.2 Analysis of sorting algorithms
- 3.2.1. Merge sort
- 3.2.2. Heap sort
- 3.2.3. Quick sort
- 3.2.4. Randomized Quick sort

3.3. Order Statistics
- 3.3.1. Selection in expected linear time
- 3.3.2. Selection in worst case linear time

BINARY TREES

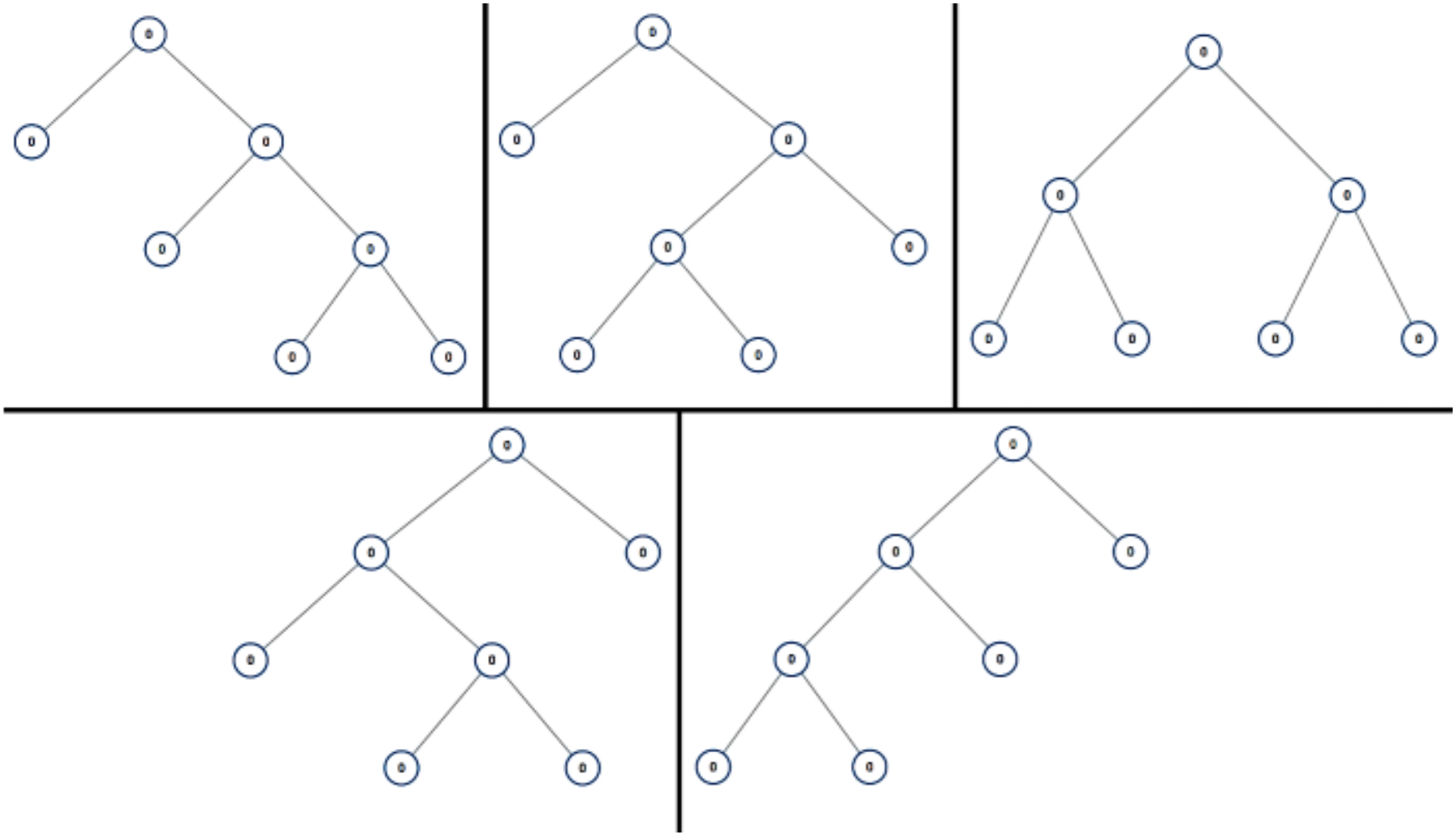
Binary Tree is defined as a Tree data structure with at most 2 children. Since each element in a binary tree can have only 2 children, we typically name them the left and right child.



Properties of a binary heap:

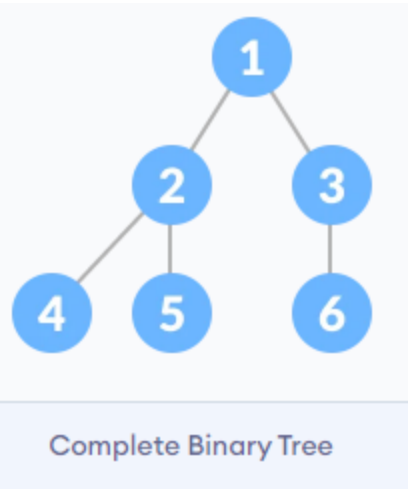
- If a node is at index i
- Its left child is at $2*i$
- Its right child is at $2*i+1$
- Its parent is at floor(i/2) i.e. $\lfloor i/2 \rfloor$

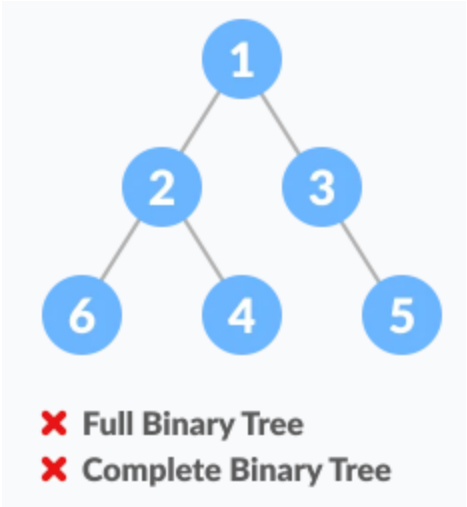
Full Binary Tree: A full binary tree is a tree in which every node has either 0 or 2 children.

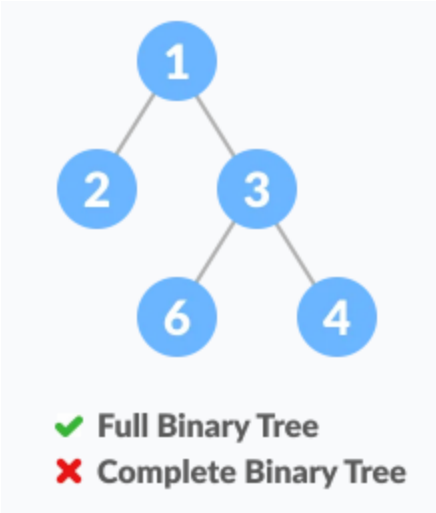


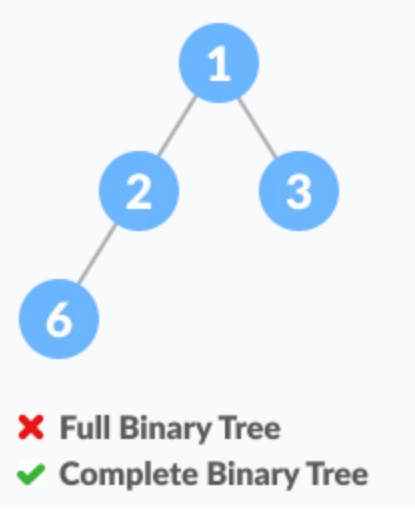
Complete Binary Trees:

- ✓ A complete binary tree is a binary tree in which all the levels are completely filled except possibly the lowest one, which is filled from the left.
- ✓ A complete binary tree is just like a full binary tree, but with two major differences:
  - All the leaf elements must lean towards the left.
  - The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



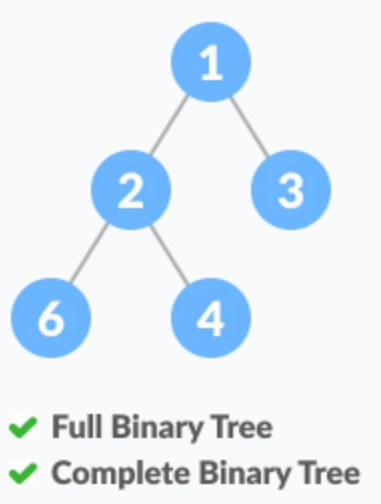






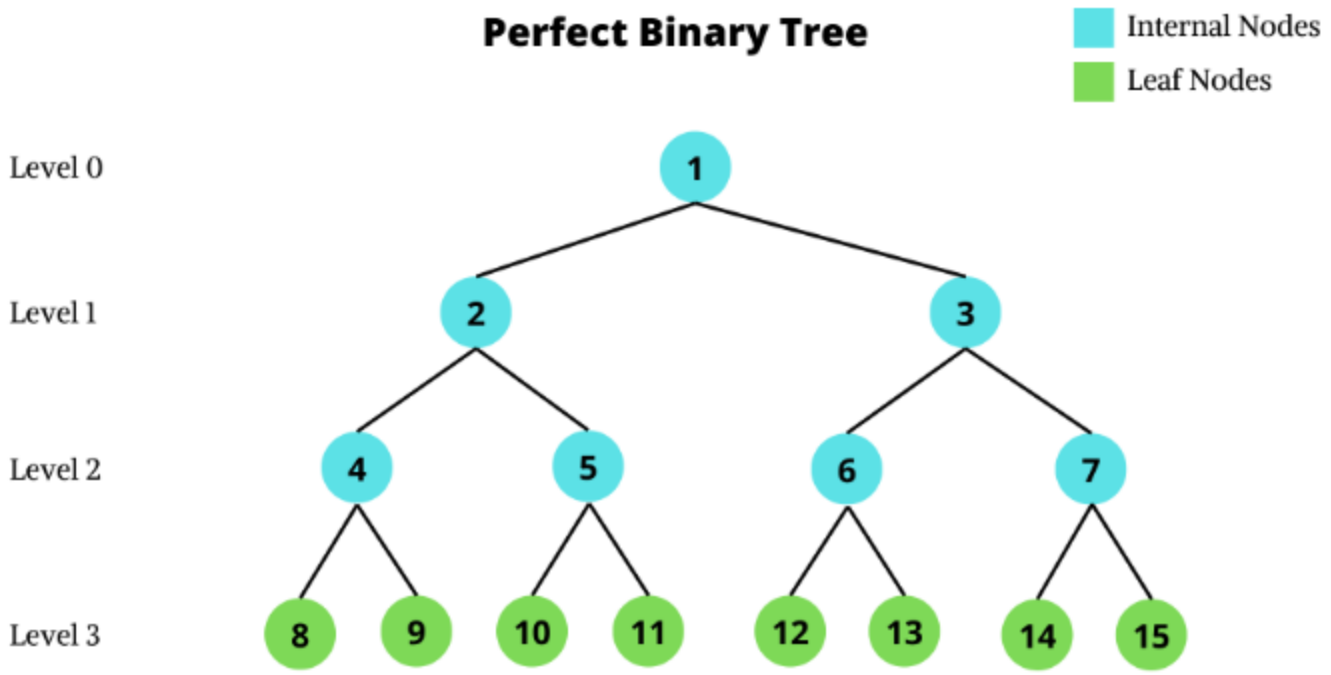
Complete Binary Trees:

- ✓ A complete binary tree is a binary tree in which all the levels are completely filled except possibly the lowest one, which is filled from the left.
- ✓ A complete binary tree is just like a full binary tree, but with two major differences:
  - All the leaf elements must lean towards the left.
  - The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.

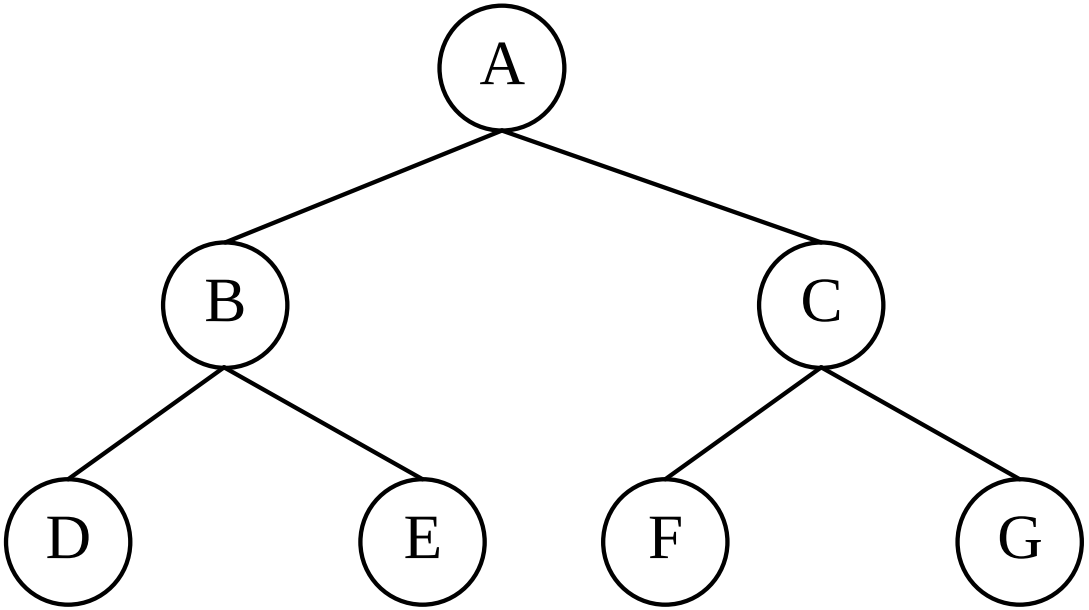


Perfect Binary Tree:

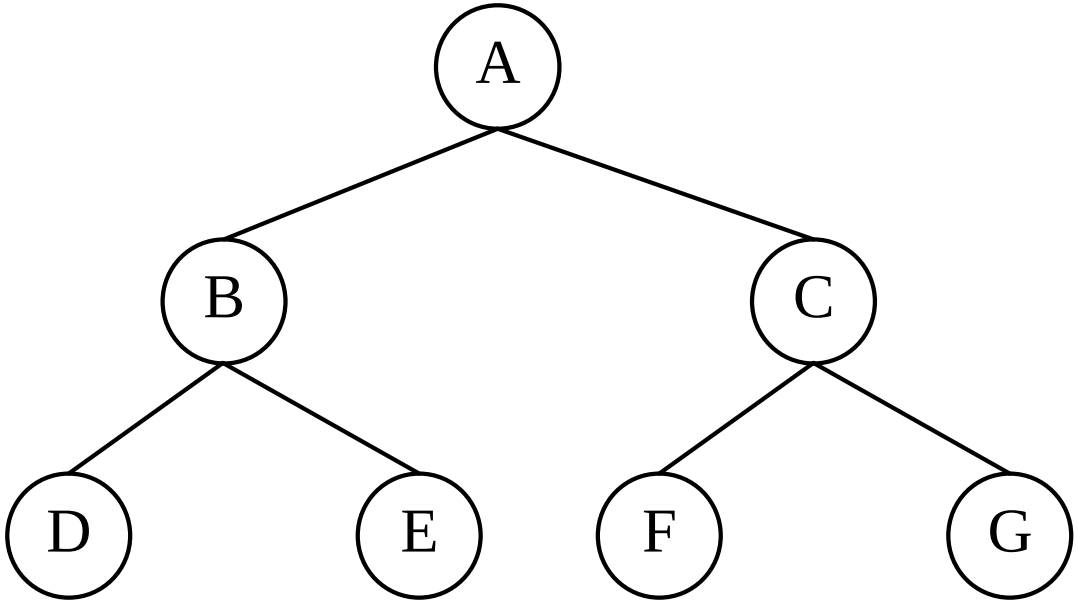
A Perfect Binary Tree is a binary tree in which all the internal nodes have 2 children and the leaf nodes are at the same level.



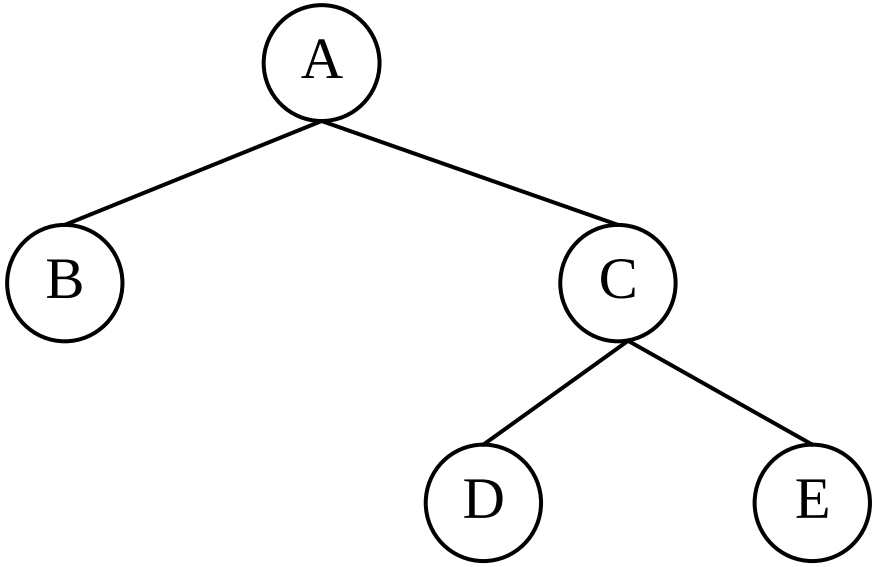
Array Representation of Binary Trees



1	2	3	4	5	6	7
A	B	C	D	E	F	G



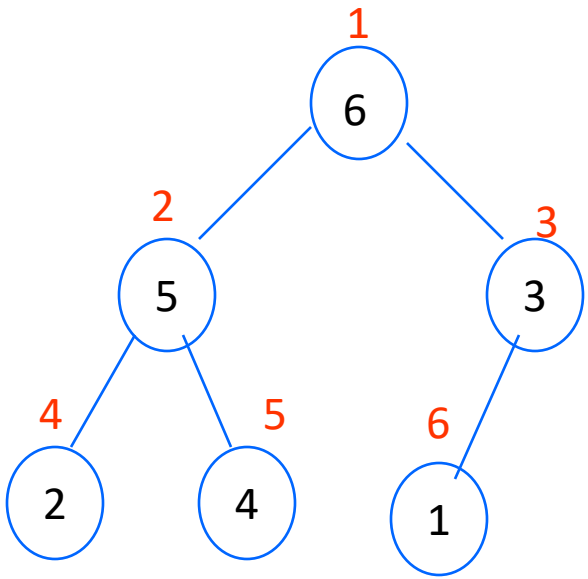
1	2	3	4	5	6	7
A	B	C	D	E	F	G



1	2	3	4	5	6	7
A	B	C			D	E

HEAP

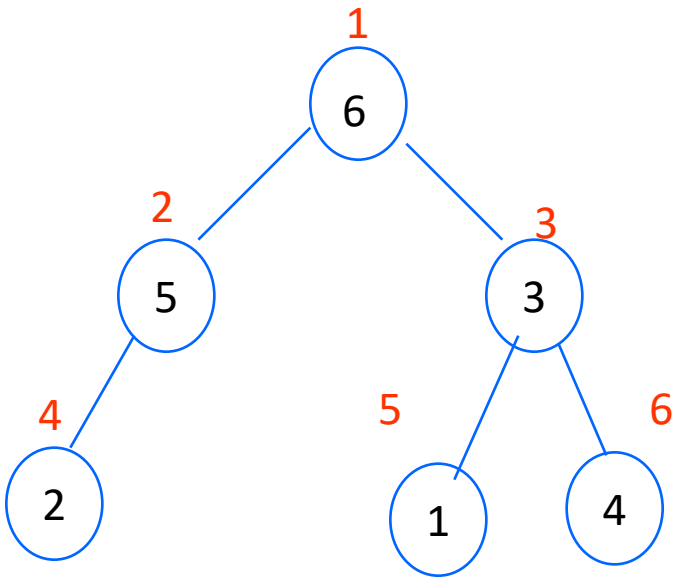
- ✓ A heap is a nearly complete binary tree which can be easily implemented on an array.



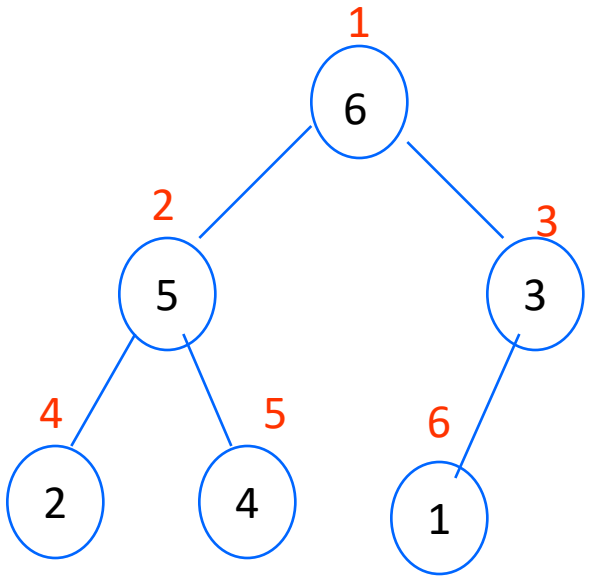
6	5	3	2	4	1

NEARLY COMPLETE

- ✓ Every level except bottom is complete.
- ✓ On the bottom, nodes are placed as left as possible.



No!



Yes!

ALGORITHMS ASSOCIATED WITH HEAP

Parent(i)

    return $\lfloor i/2 \rfloor$;

Left(i)

    return $2i$;

Right(i)

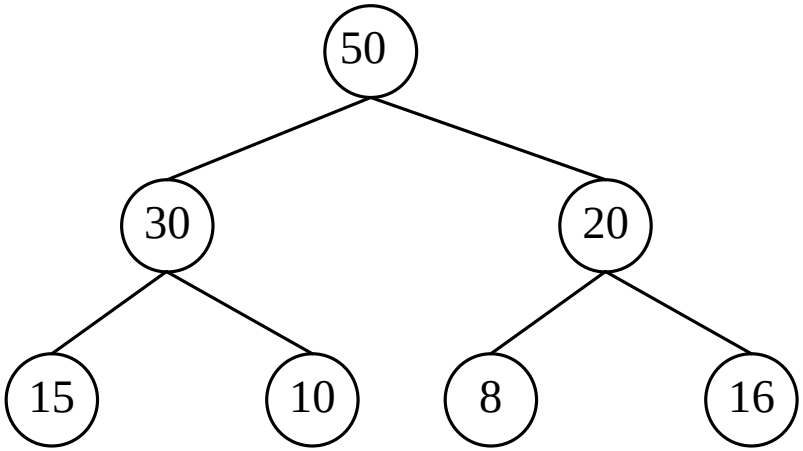
    return $2i+1$;

MAX HEAP

- ✓ A max heap is **a complete binary tree in which the value of a node is greater than or equal to the values of its children**.
- ✓ In a max-heap, every node ***i*** other than the root satisfies the following property:

$$A[\text{Parent}(i)] \geq A[i]$$

- ✓ Max Heap data structure is useful for sorting data using heap sort.



1	2	3	4	5	6	7
50	30	20	15	10	8	16

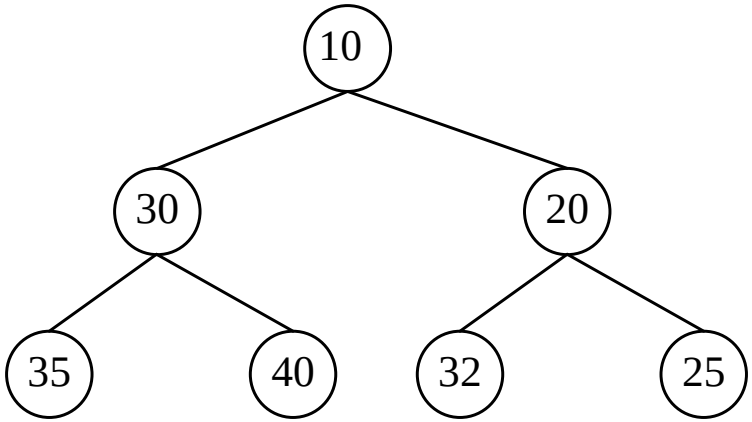
Figure: ***A Max Heap Viewed as a Binary Tree and as an array***

MIN HEAP

- ✓ A min heap is **a complete binary tree in which the value of a node is less than or equal to the values of its children**.
- ✓ In a min-heap, every node ***i*** other than the root satisfies the following property:

$$A[\text{Parent}(i)] \leq A[i]$$

- ✓ Min Heap data structure is useful for sorting data using heap sort.



1	2	3	4	5	6	7
10	30	20	35	40	32	25

Figure: ***A Min Heap Viewed as a Binary Tree and as an array***

BUILD A HEAP FROM GIVEN ARRAY

- ✓ Elements in a max heap should follow the **max heap property**. This means that the key at the parent node is always greater than the key at both child nodes. **To build a max heap, following steps should be followed:**
  - Create a new node at the beginning (root) of the heap.
  - Assign it a value.
  - Compare the value of the child node with the parent node.
  - Swap nodes if the value of the parent is less than that of either child (to the left or right).
  - Repeat until the largest element is at the root parent nodes (then you can say that the heap property holds).
- ✓ These steps can also be followed when inserting new elements into a heap. The key here is, whatever operation being carried out on a Max Heap, the heap property must be maintained.

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

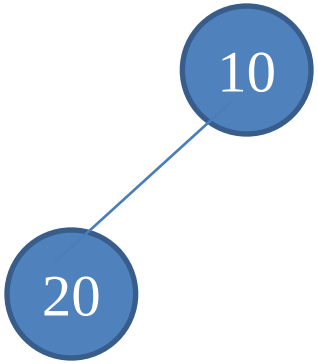


index	1	2	3	4	5
array	**10**				

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

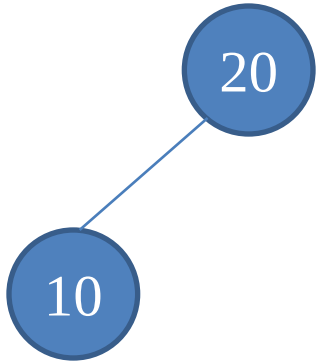


index	1	2	3	4	5
array	**10**	**20**			

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

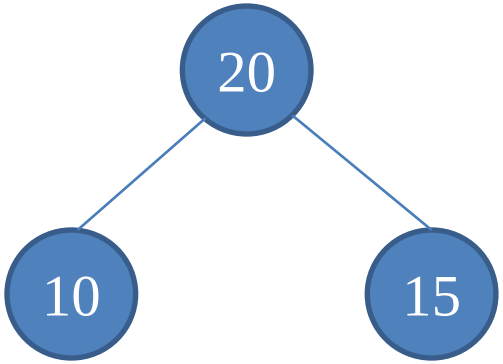


index	1	2	3	4	5
array	**20**	**10**			

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

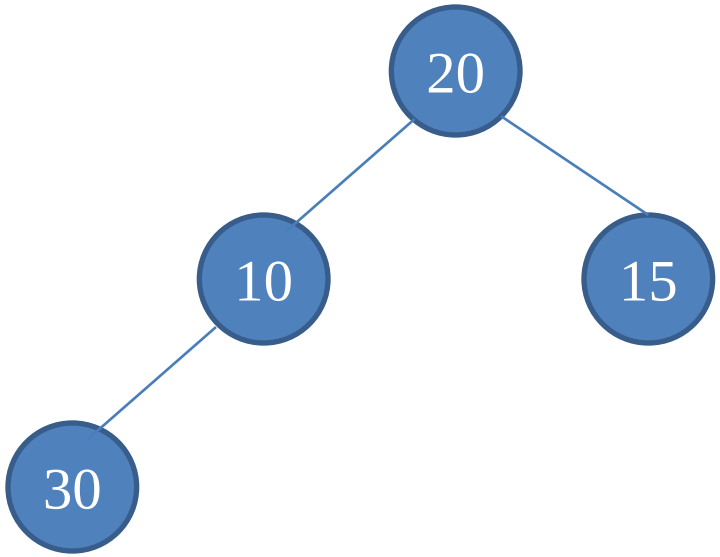


index	1	2	3	4	5
array	**20**	**10**	**15**		

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

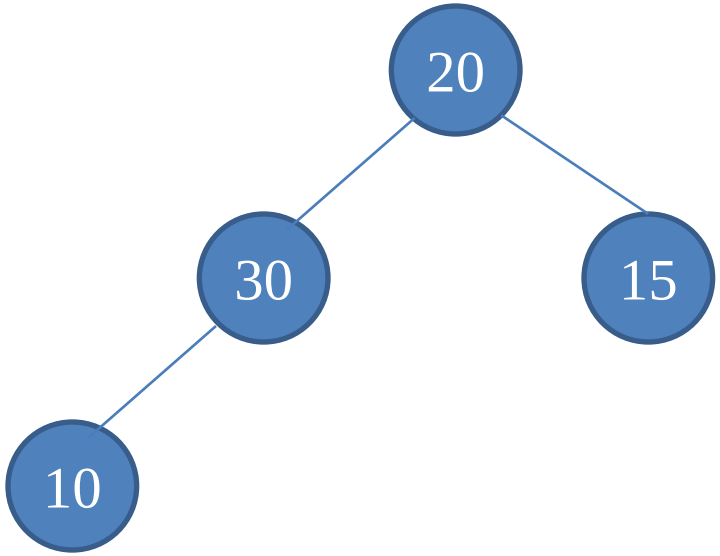


index	1	2	3	4	5
array	**20**	**10**	**15**	**30**	

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

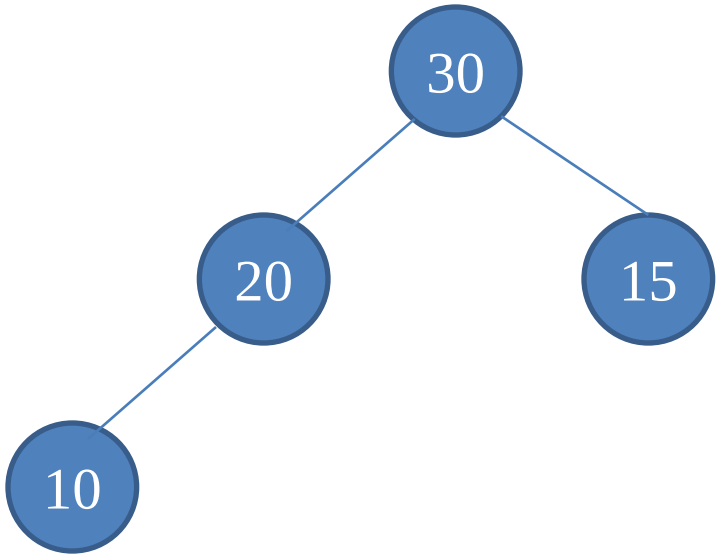


index	1	2	3	4	5
array	**20**	**30**	**15**	**10**	

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

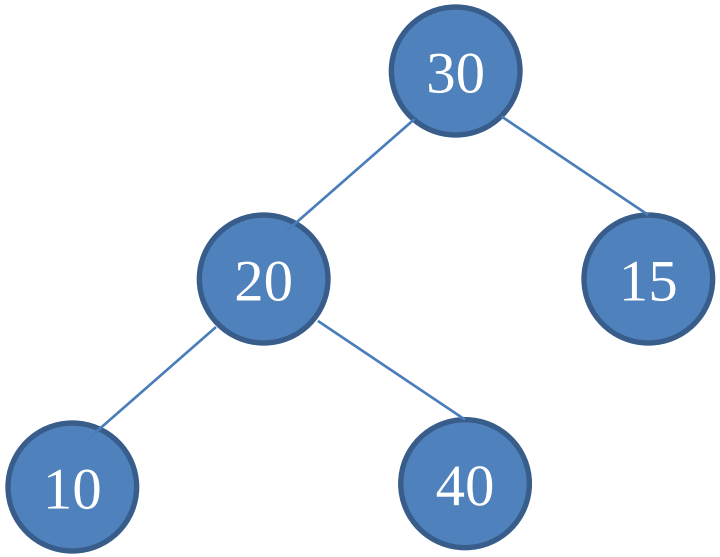


index	1	2	3	4	5
array	**30**	**20**	**15**	**10**	

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

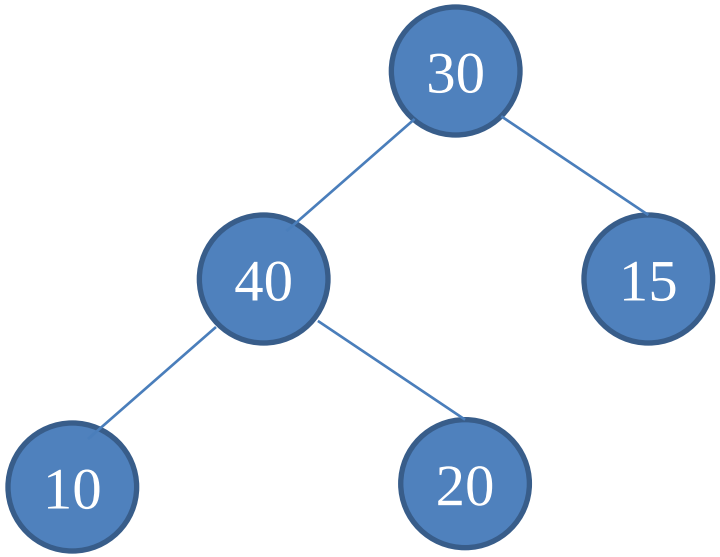


index	1	2	3	4	5
array	**30**	**20**	**15**	**10**	**40**

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

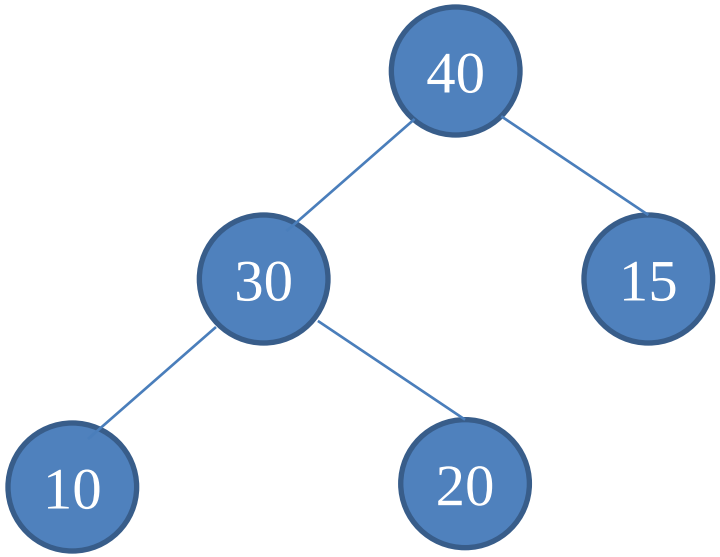


index	1	2	3	4	5
array	**30**	**40**	**15**	**10**	**20**

- ✓ Consider following array:

$$A = \{10,20,15,30,40\}$$

- ✓ Following animation illustrates the process of creation of the heap:

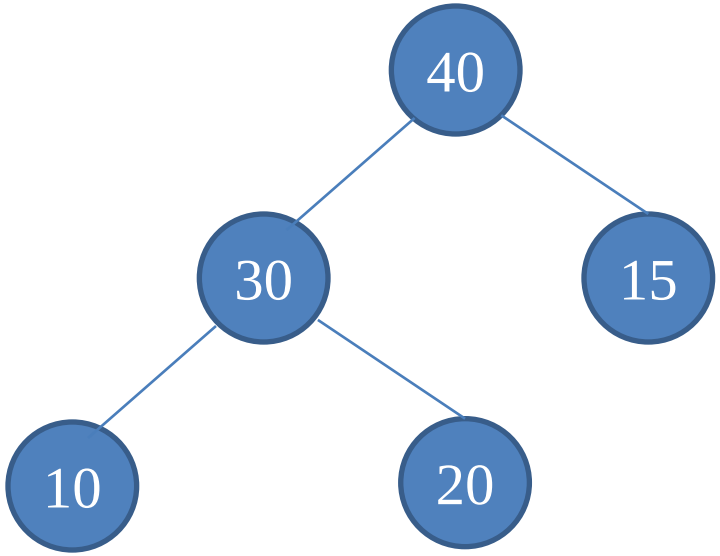


index	1	2	3	4	5
array	**40**	**30**	**15**	**10**	**20**

DELETING NODES FROM HEAP

- ✓ To delete nodes from a heap, following steps should be followed:
  - Delete the root node.
  - Move the last child node of the last level to root.
  - Compare the parent node with its children.
  - If the value of the parent is less than child nodes, swap them, and repeat until the heap property is satisfied.

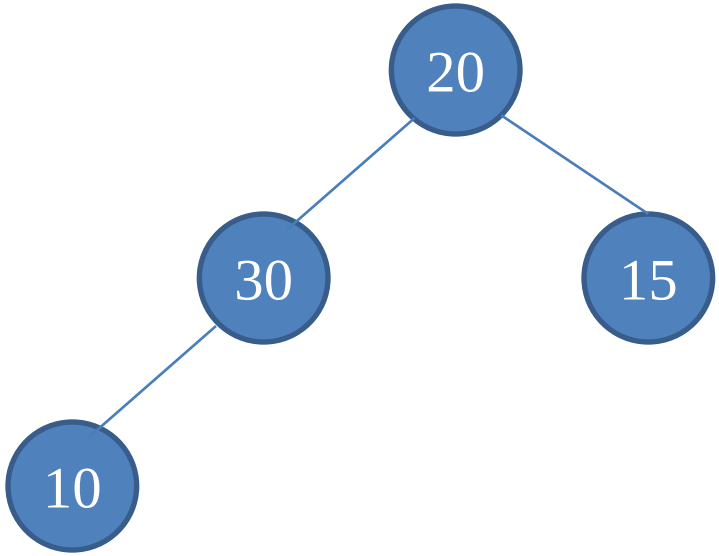
✓ Following animation shows how nodes are deleted from a Heap:



Heap Size = 5

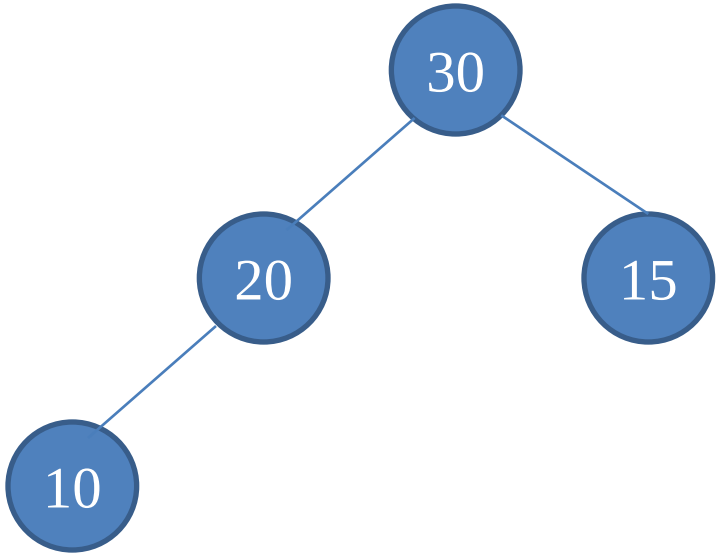
array	**40**	**30**	**15**	**10**	**20**
index	1	2	3	4	5

✓ Following animation shows how nodes are deleted from a Heap:



array	**20**	**30**	**15**	**10**	**40**
index	1	2	3	4	5

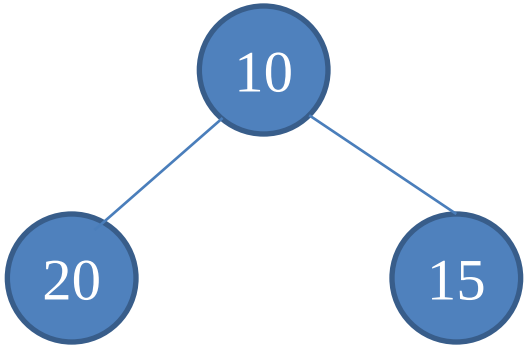
✓ Following animation shows how nodes are deleted from a Heap:



Heap Size = 4

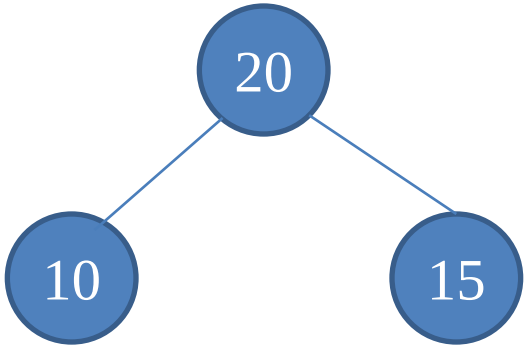
array	**30**	**20**	**15**	**10**	**40**
index	1	2	3	4	5

✓ Following animation shows how nodes are deleted from a Heap:



array	**10**	**20**	**15**	**30**	**40**
index	1	2	3	4	5

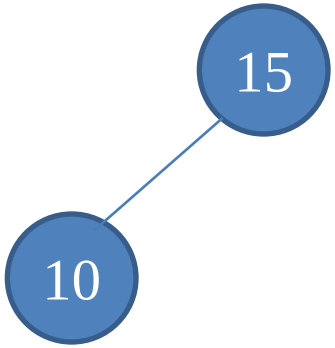
✓ Following animation shows how nodes are deleted from a Heap:



Heap Size = 3

array	**20**	**10**	**15**	**30**	**40**
index	1	2	3	4	5

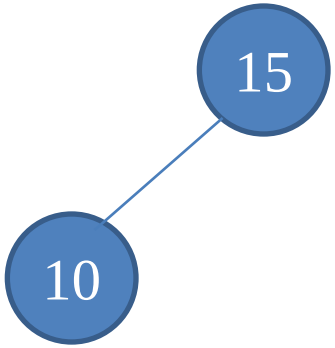
✓ Following animation shows how nodes are deleted from a Heap:



Heap Size = 2

array	**15**	**10**	**20**	**30**	**40**
index	1	2	3	4	5

✓ Following animation shows how nodes are deleted from a Heap:

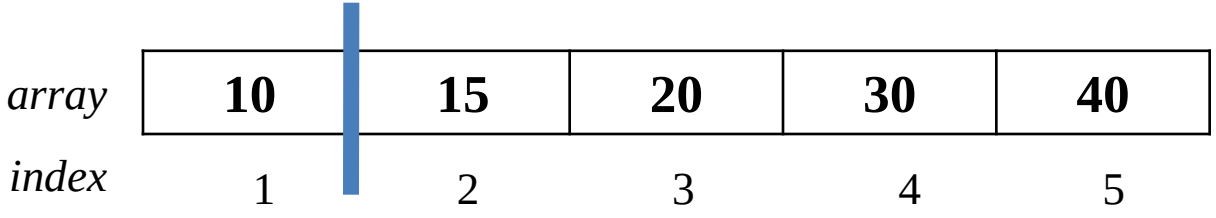


array	**15**	**10**	**20**	**30**	**40**
index	1	2	3	4	5

✓ Following animation shows how nodes are deleted from a Heap:

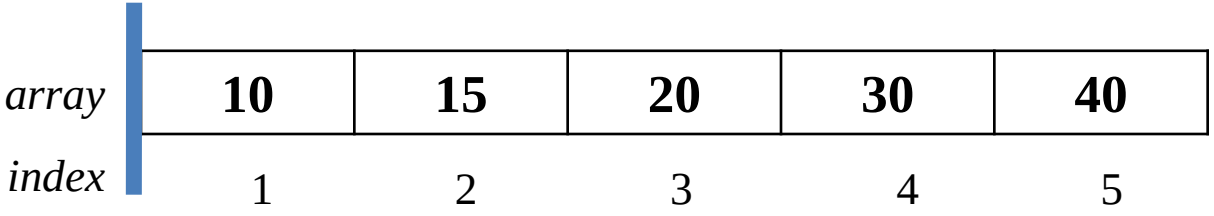


Heap Size = 1



:

Heap Size = 0

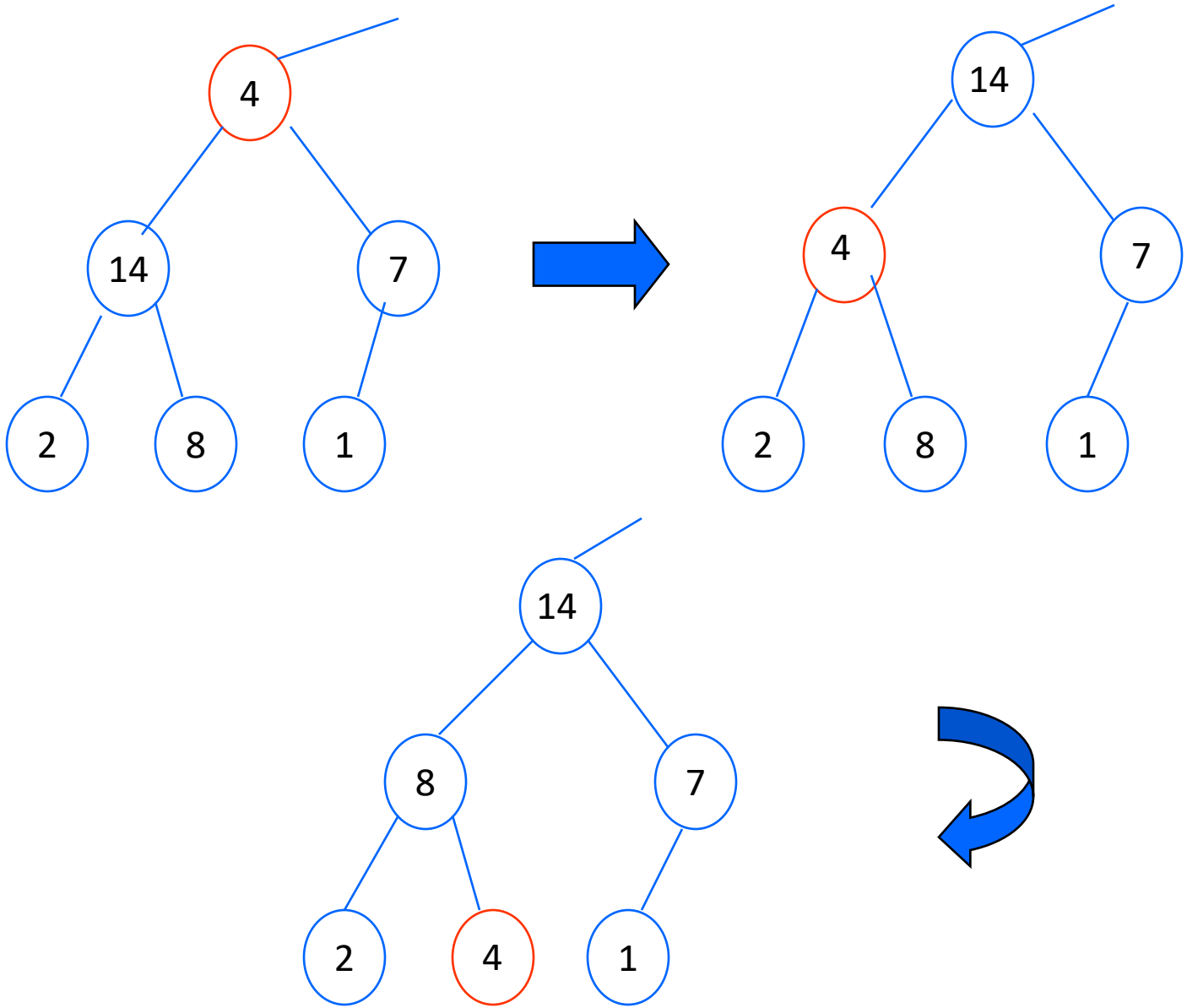


Algorithms associated with Max-Heap

- Three algorithms associated with heap.
- In addition, it is associated with two more algorithms: Max-Heapify and Build-Max-Heap.

Max-Heapify

- Max-Heapify(A,i) is a subroutine.
- When it is called, two subtrees rooted at Left(i) and Right(i) are max-heaps, but A[i] may not satisfy the max-heap property.
- Max-Heapify(A,i) makes the subtree rooted at A[i] become a max-heap by letting A[i] “float down”.


4
14
7
2
8
1
14
4
7
2
8
1
14
8
7
2
4
1

$\text{Max - Heapify}(A, i)$

$l \leftarrow \text{Left}(i);$

$r \leftarrow \text{Right}(i);$

if $l \leq heap-size[A]$ and $A[l] > A[i]$

then $largest \leftarrow l$

else $largest \leftarrow i;$

if $r \leq heap-size[A]$ and $A[r] > A[largest]$

then $largest \leftarrow r;$

if $largest \neq i$

then begin exchange $A[i] \leftrightarrow A[largest];$

$\text{Max - Heapify}(A, largest);$

end - if

Running time

$height = \lfloor \lg n \rfloor$ where $n = heap\text{-}size[A]$

Hence, Max - Heapify runs in time $O(\lg n)$.

Let $h = height$. Then h is the largest integer satisfying $n \geq 1 + 2 + \cdots + 2^{h-1} + 1$,

that is, $n \geq 2^h$.

Hence, $h = \lfloor \lg n \rfloor$.

Building a Heap

$$\text{Build-Max-Heap}(A)$$

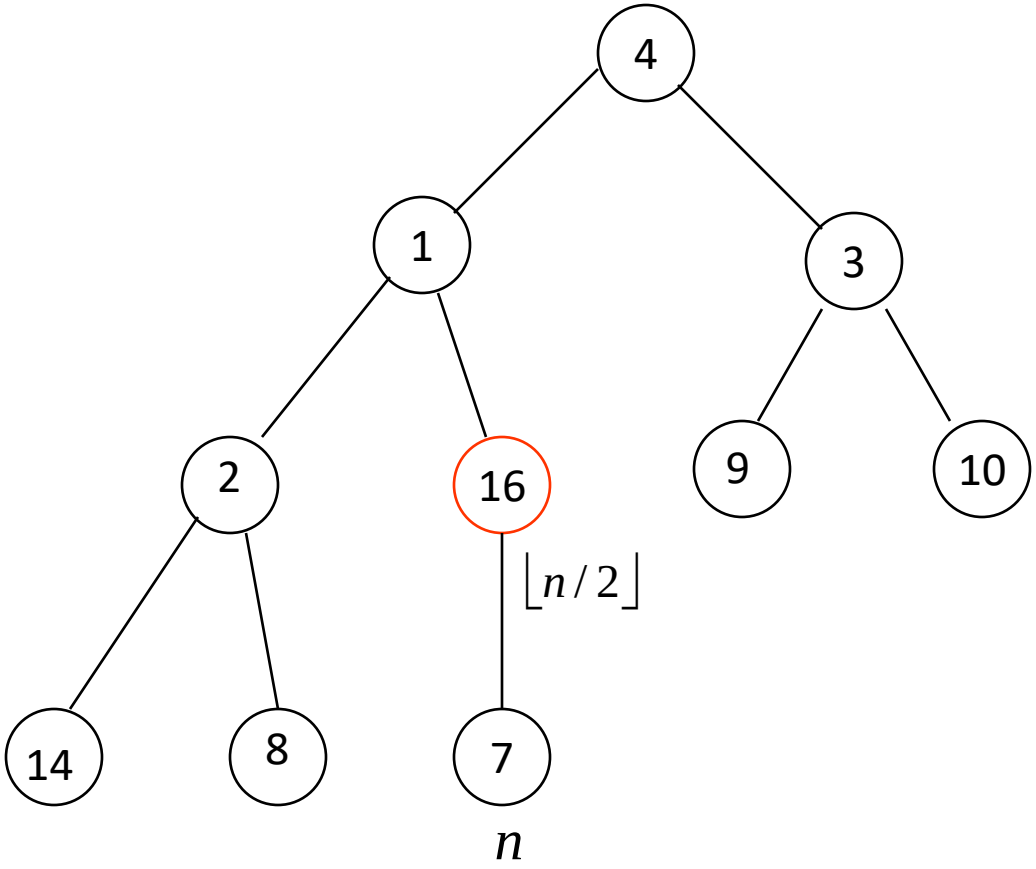
$$heap\text{-}size[A] \leftarrow length[A];$$

$$\text{for } i \leftarrow \lfloor length[A]/2 \rfloor \text{ downto } 1$$

$$\quad \text{do } \text{Max-Heapify}(A, i);$$

e.g., 4, 1, 3, 2, 16, 9, 10, 14, 8, 7.

The last location who has a child is $\lfloor n/2 \rfloor$.



Building a Max-Heap

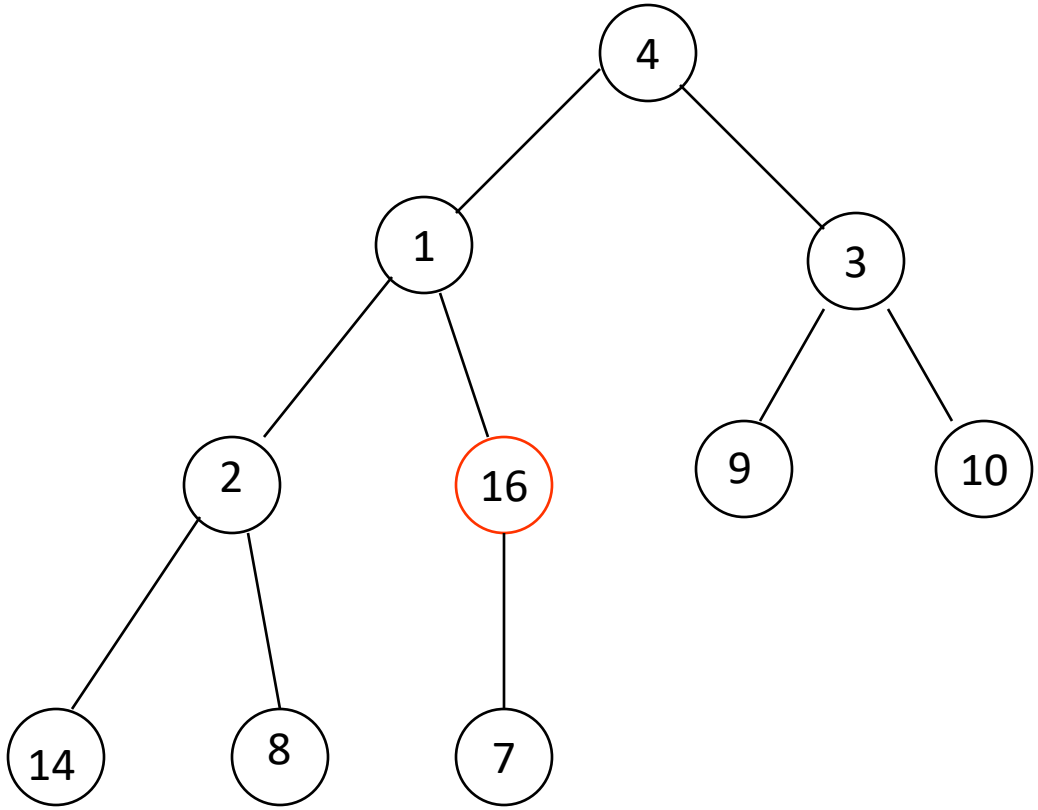
$$\text{Build-Max-Heap}(A)$$

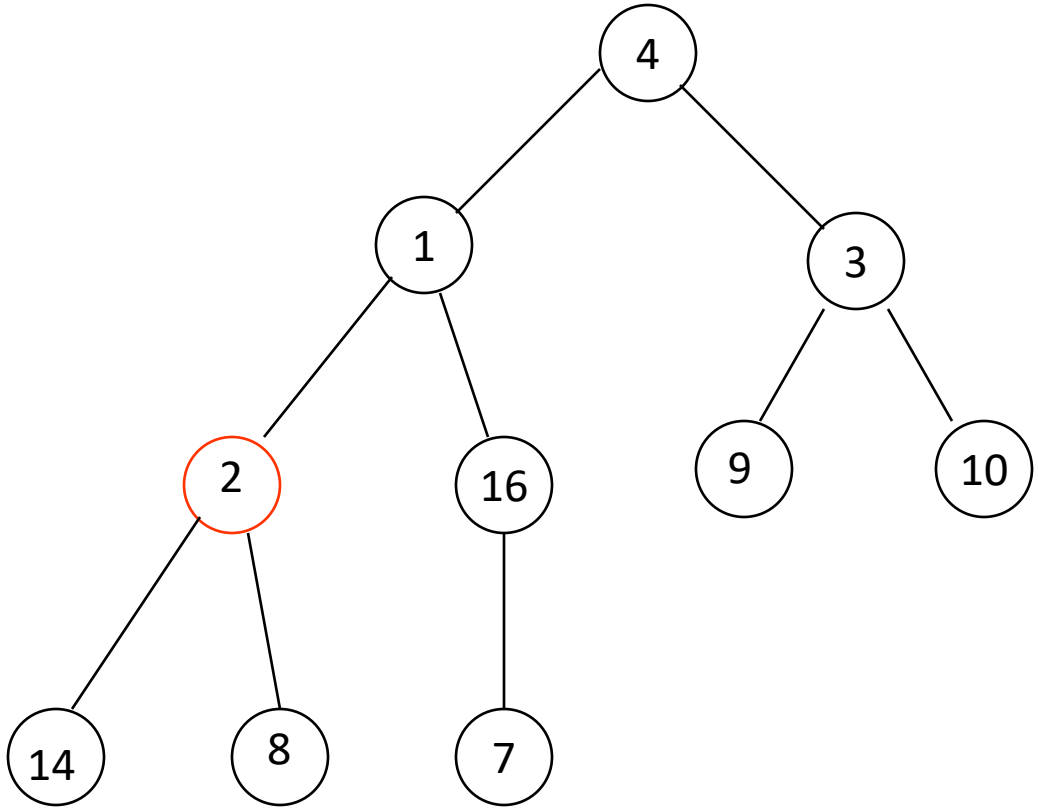
$$heap\text{-}size[A] \leftarrow length[A];$$

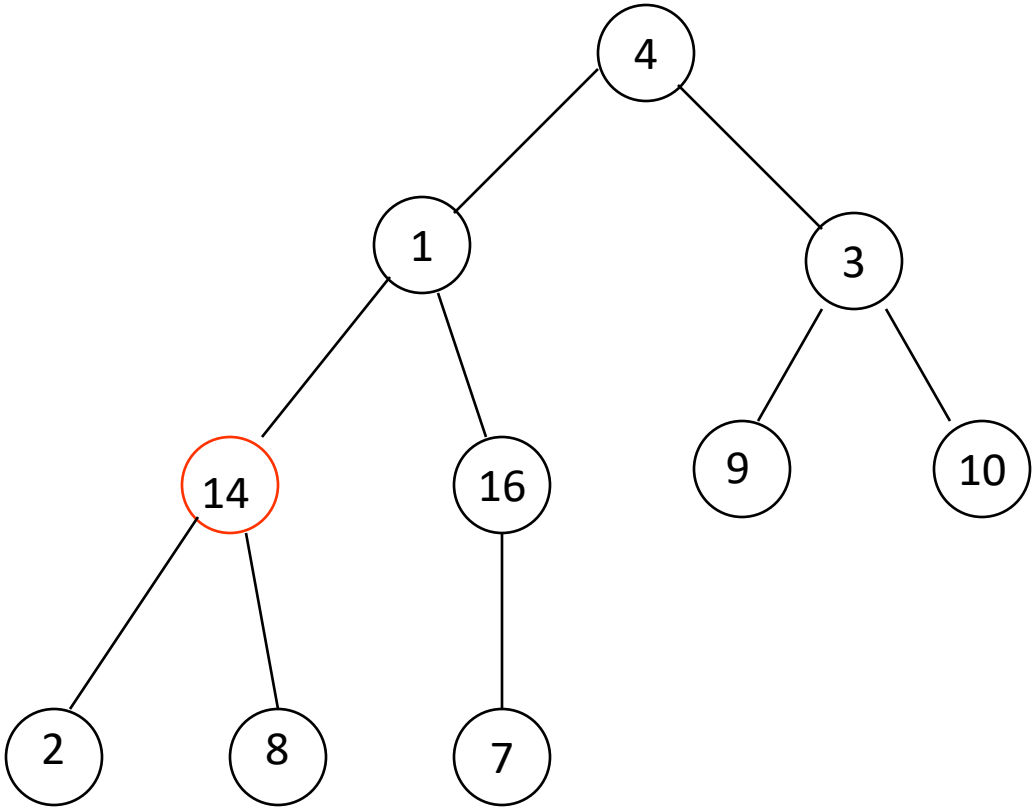
$$\text{for } i \leftarrow \lfloor length[A]/2 \rfloor \text{ downto } 1$$

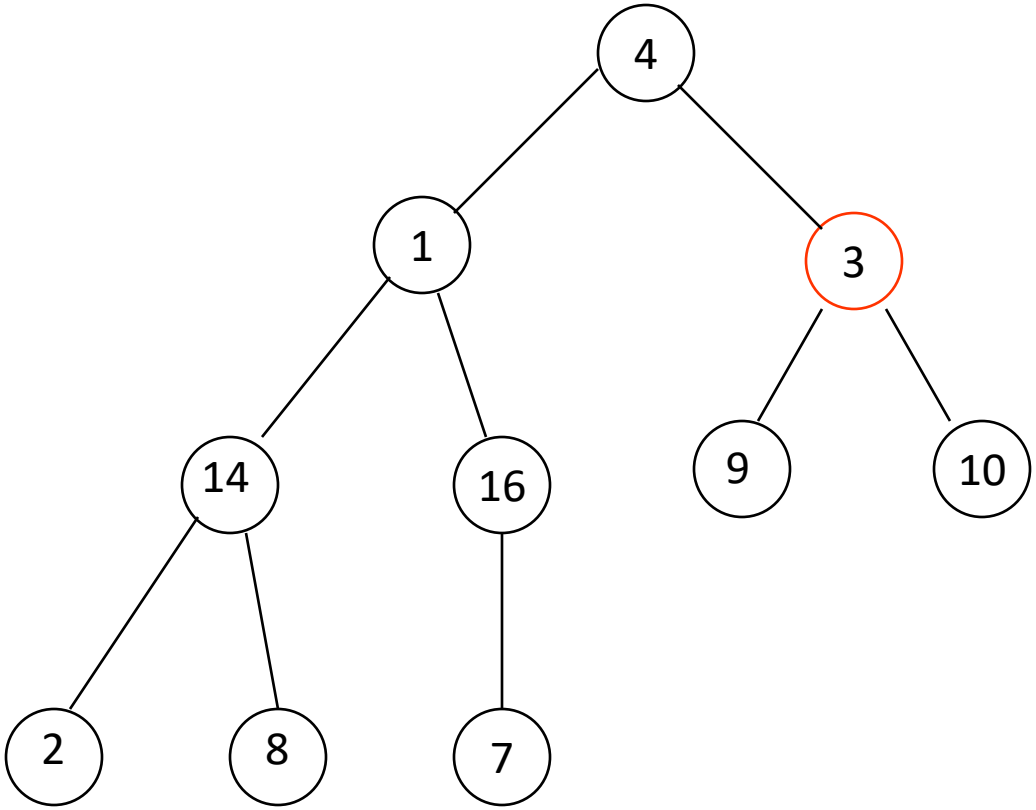
$$\quad \text{do } \text{Max-Heapify}(A, i);$$

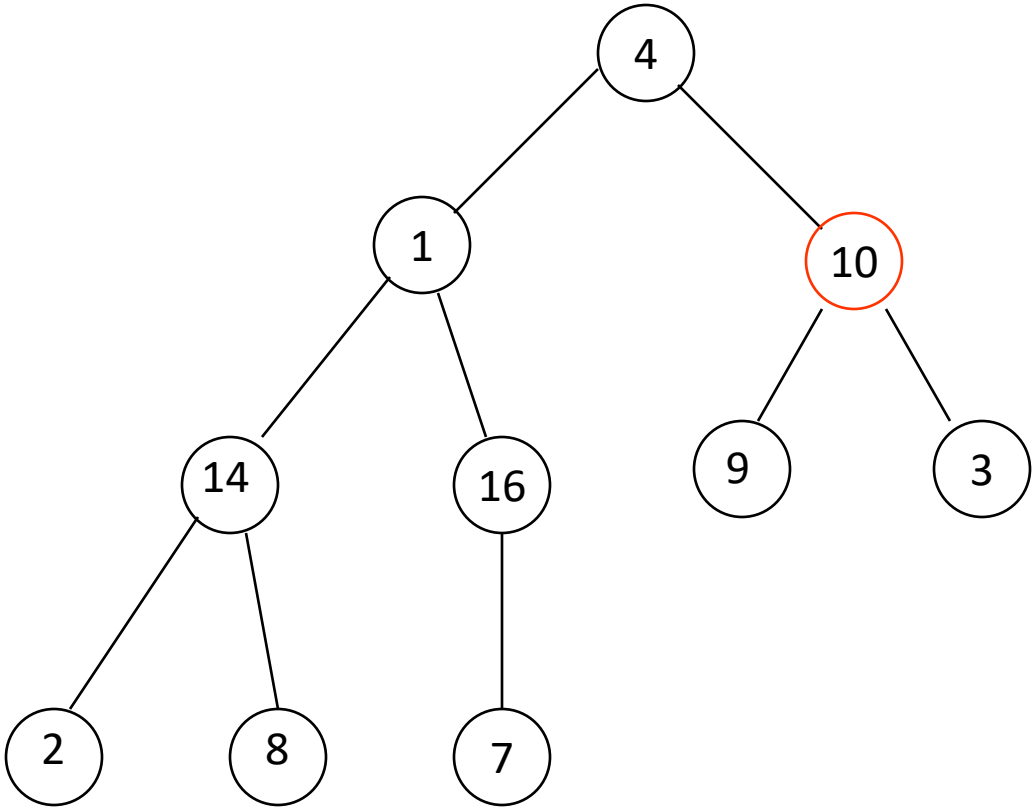
e.g., 4, 1, 3, 2, 16, 9, 10, 14, 8, 7.

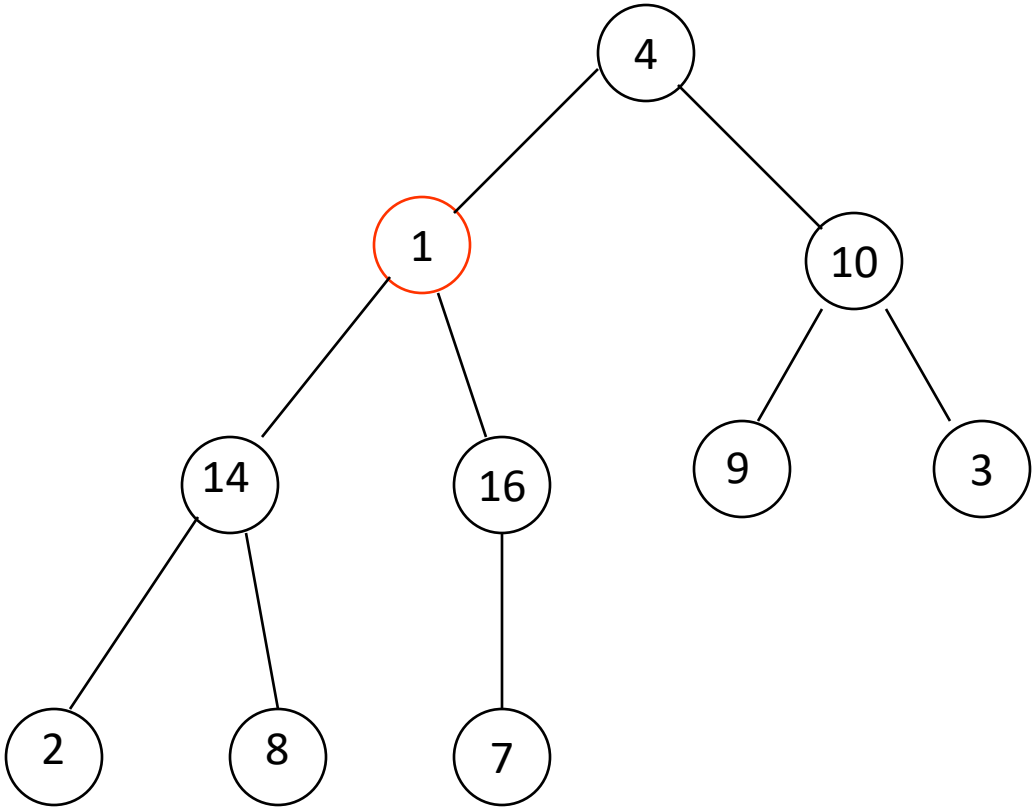

4
1
3
2
16
9
10
14
8
7

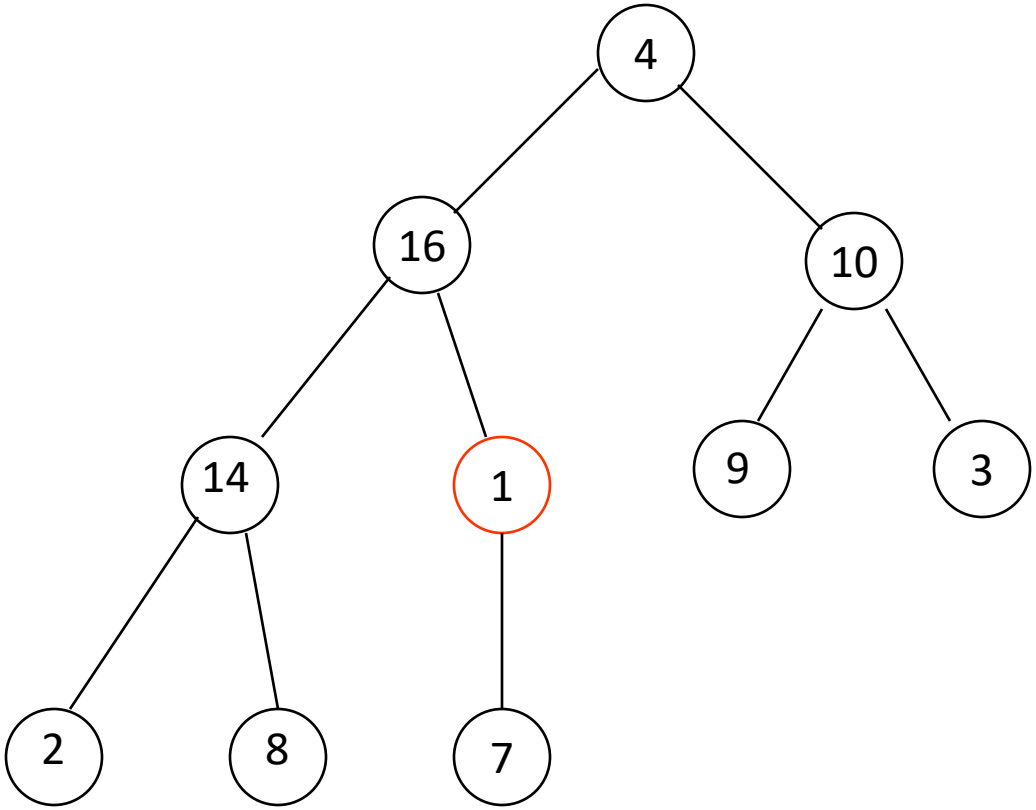

4
1
3
2
16
9
10
14
8
7

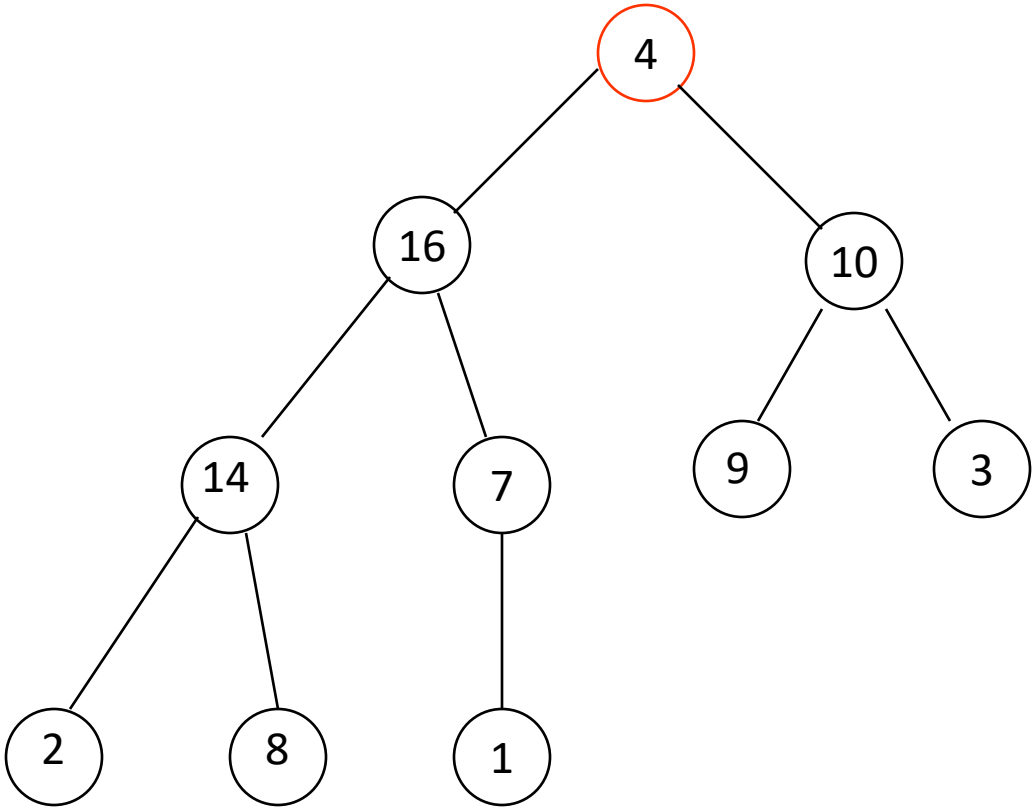

4
1
3
14
16
9
10
2
8
7

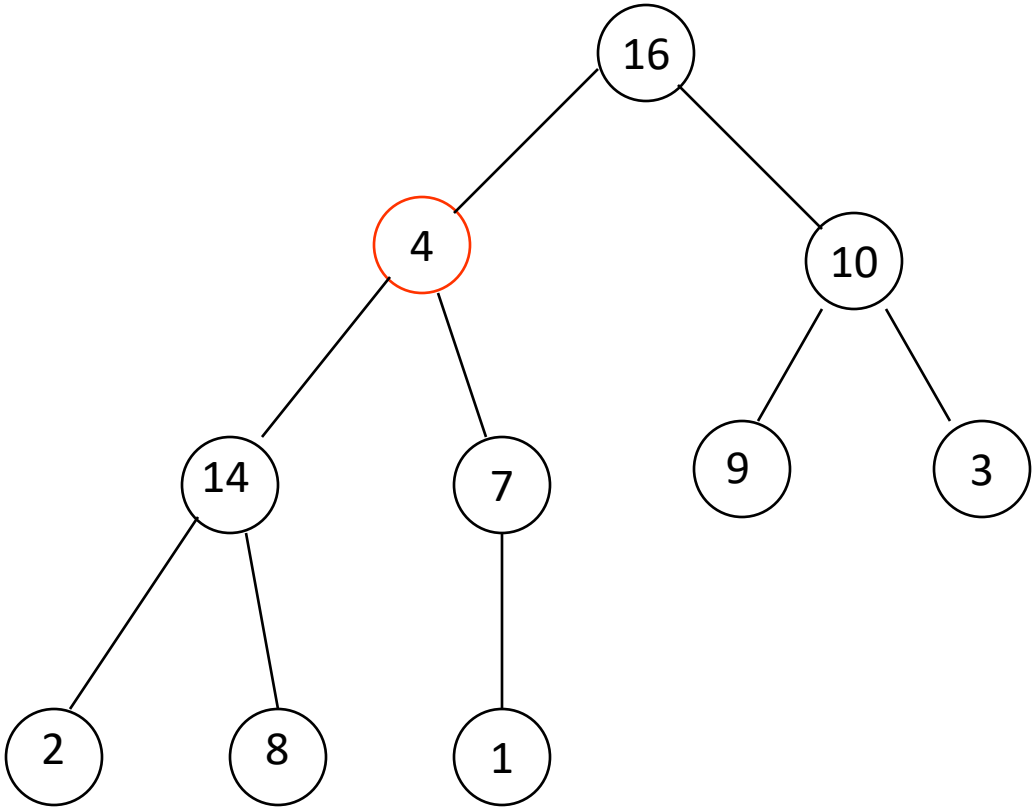

4
1
3
14
16
9
10
2
8
7

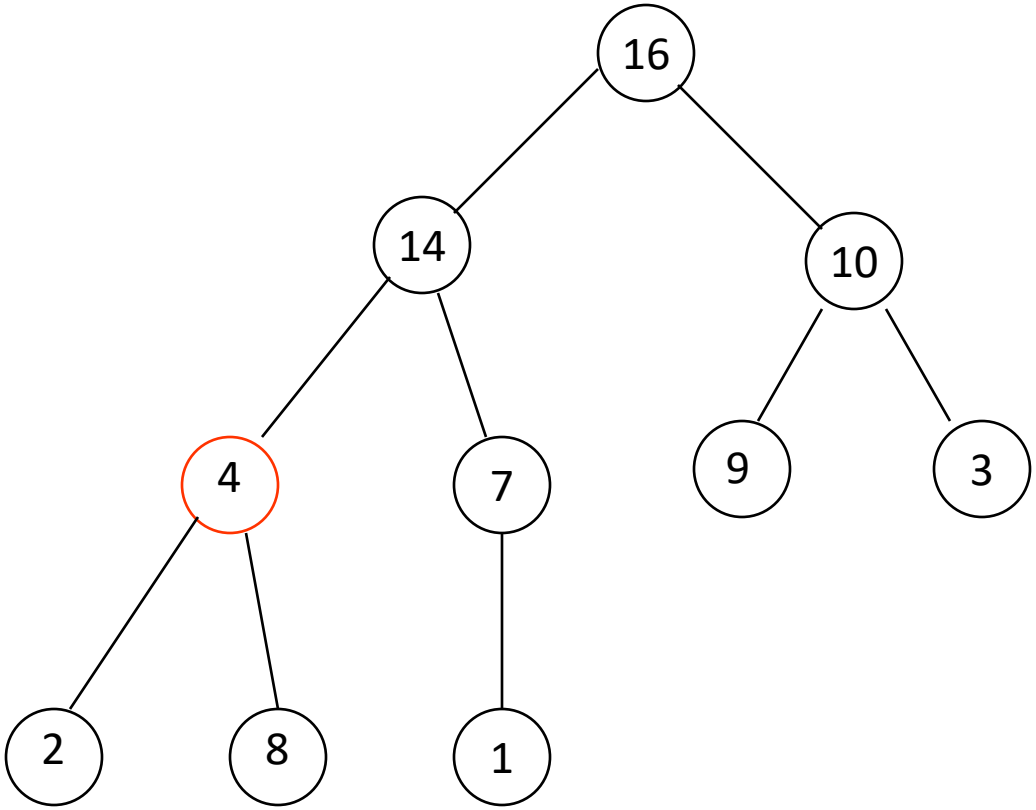

4
1
10
14
16
9
3
2
8
7

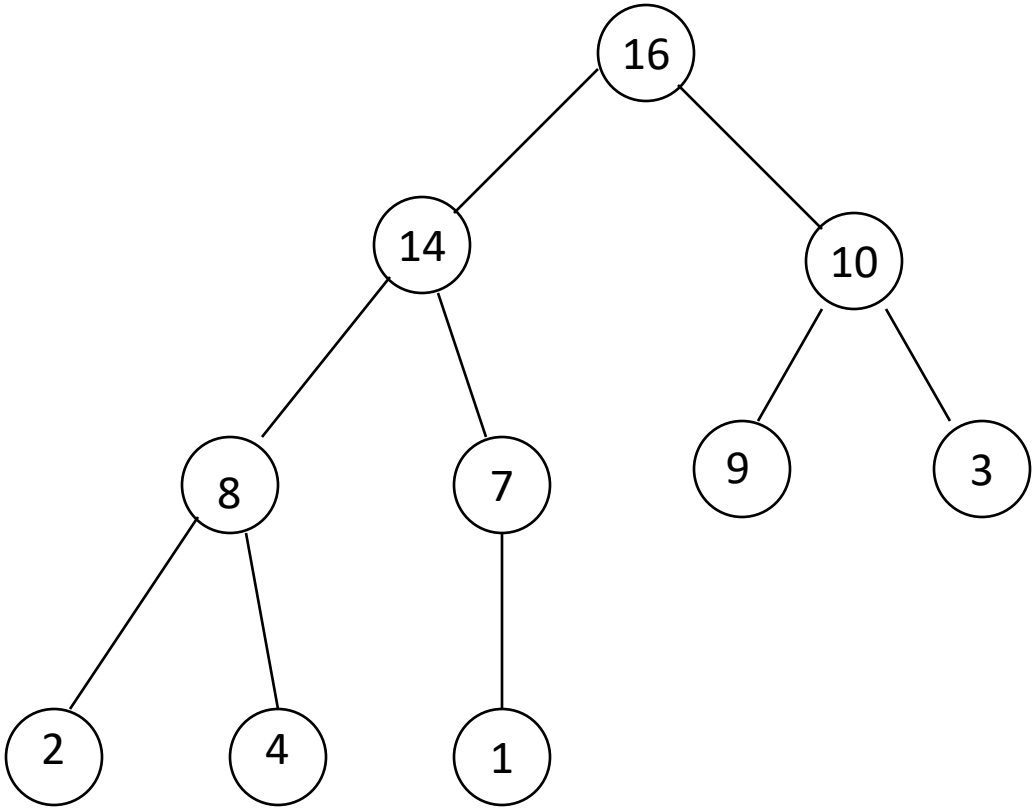

4
1
10
14
16
9
3
2
8
7


4
16
10
14
1
9
3
2
8
7


4
16
10
14
7
9
3
2
8
1


16
4
10
14
7
9
3
2
8
1


16
14
10
4
7
9
3
2
8
1


16
14
10
8
7
9
3
2
4
1

Analysis

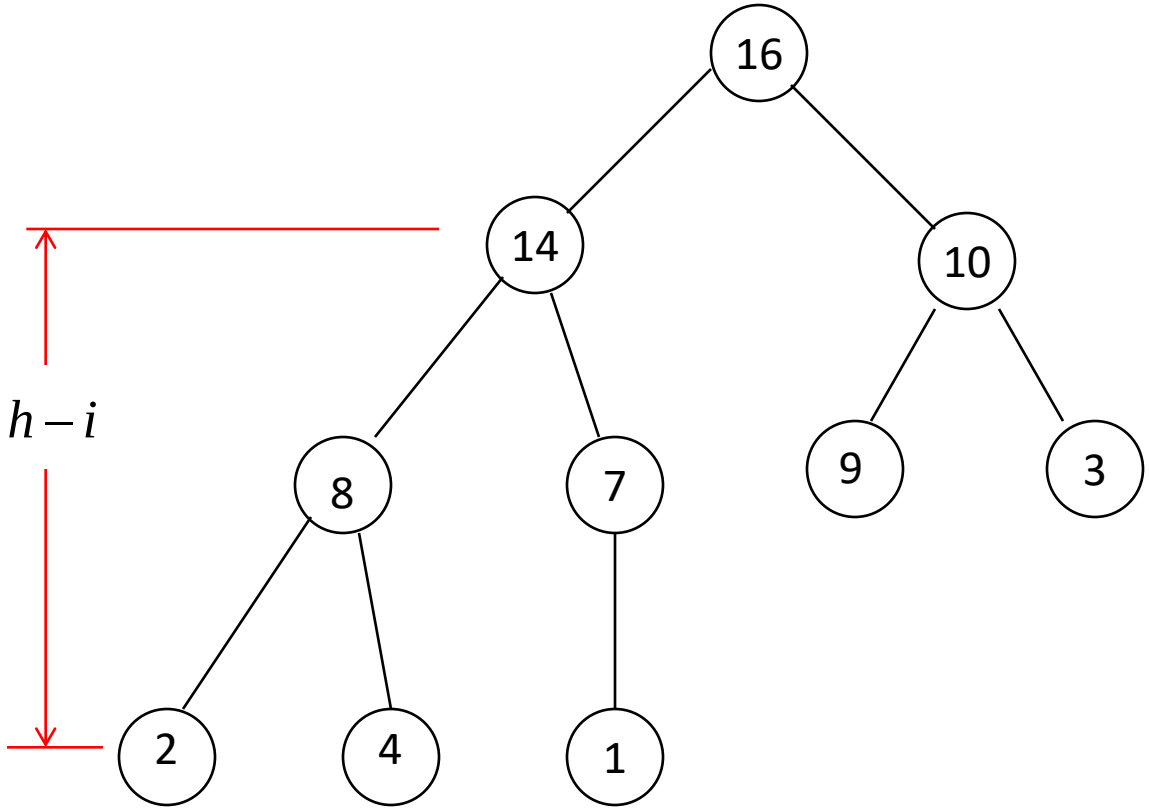
$$\text{running time} = \sum_{i=0}^{h} \#node(i) \cdot O(h-i)$$

where $\#node(i)$ is the number of nodes at level i and $h = height(A)$.

$$h = \lfloor \lg n \rfloor \text{ where } n = heap\text{-}size(A)$$

$$\#node(i) \le 2^i$$

Here i means level



$i = 0$

$i = 1$

$i = h$

Running time

$$\sum_{i=0}^{h} 2^i \cdot O(h-i) = O\left(\sum_{i=0}^{h} 2^i (h-i)\right) = O(2^h \sum_{i=0}^{h} \frac{(h-i)}{2^{h-i}}) = O(n)$$

because

$$\sum_{l=0}^{\infty} \frac{l}{2^l} = \frac{1/2}{(1-1/2)^2} = 2,$$

$$2^h = O(n).$$

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)f'(x) - f(x)g'(x)}{(g(x))^2}$$

$$\sum_{l=0}^{\infty} x^l = \frac{1}{1-x} \text{ for } x < 1$$

Differentiating both sides w.r.t x, we get,

$$or, \sum_{l=0}^{\infty} l.x^{l-1} = \frac{1}{(1-x)^2}$$

$$or, \sum_{l=0}^{\infty} l.x^{l} = \frac{x}{(1-x)^2}$$

Substituting $x = 1/2$, we get,

$$\sum_{l=0}^{\infty} \frac{l}{2^l} = \frac{1/2}{(1-1/2)^2} = 2$$

Heapsort

Heapsort(A)

Buid-Max-Heap(A);

for $i \leftarrow length[A]$ downto 2

do begin

exchange $A[1] \leftrightarrow A[i]$;

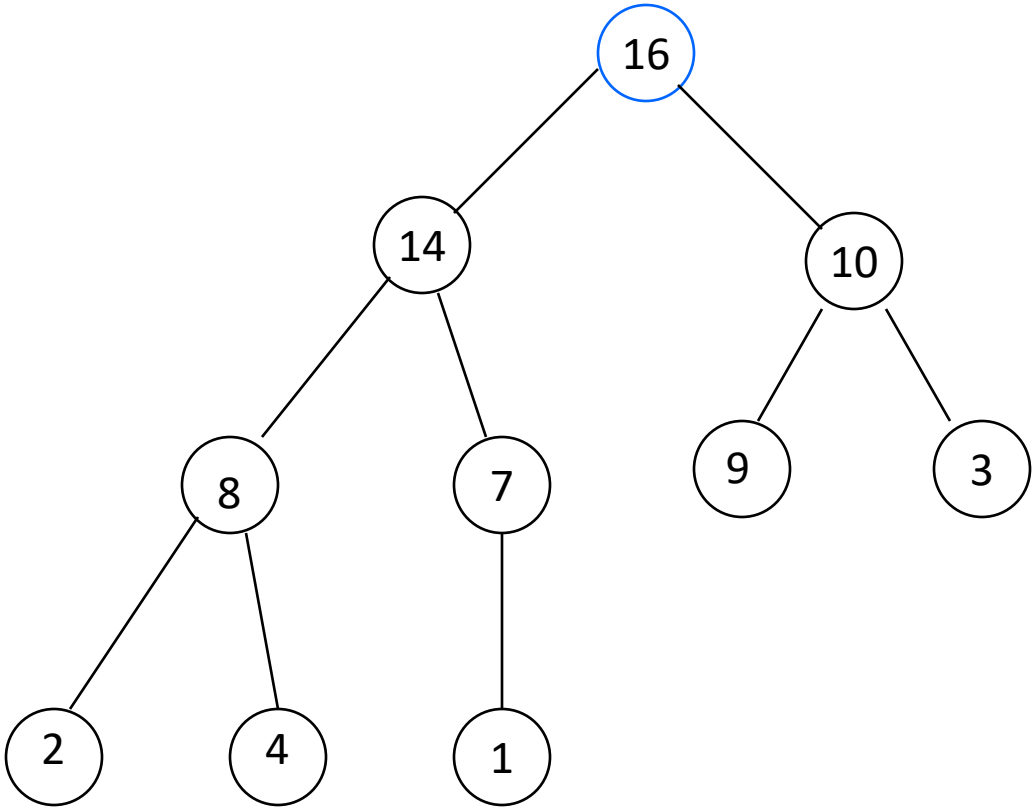
$heap\text{-}size[A] \leftarrow heap\text{-}size[A] - 1$;

Max-Heapify(A,1);

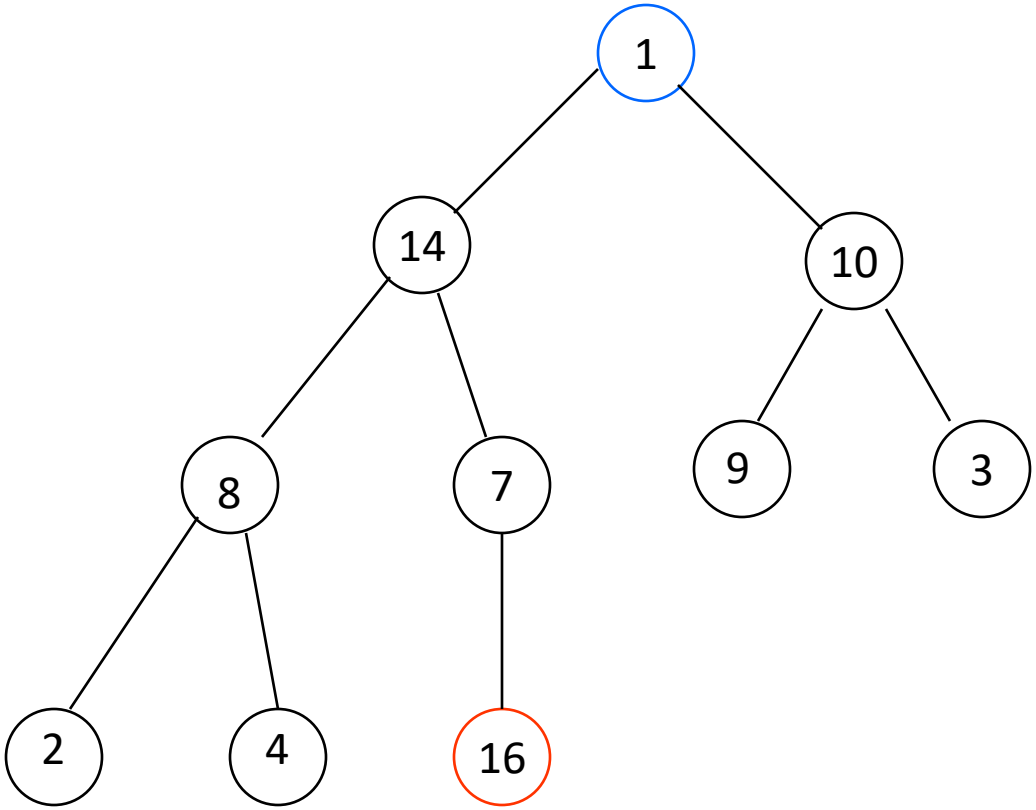
end-for

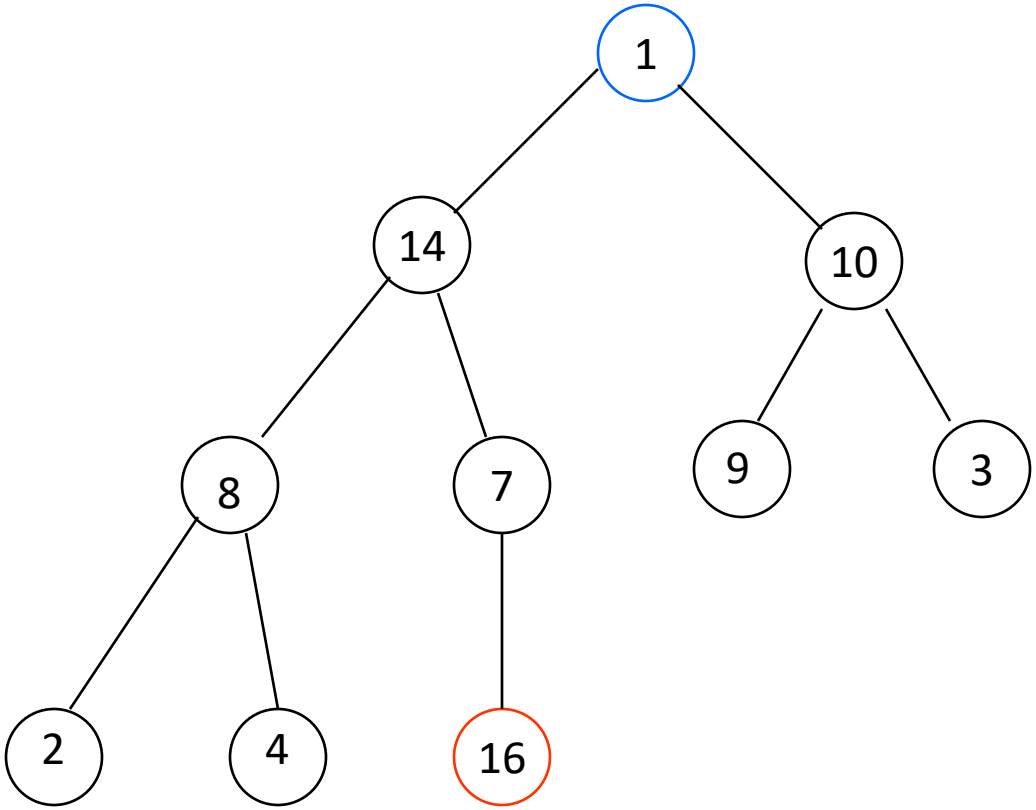
Input: 4, 1, 3, 2, 16, 9, 10, 14, 8, 7.

Build a max-heap

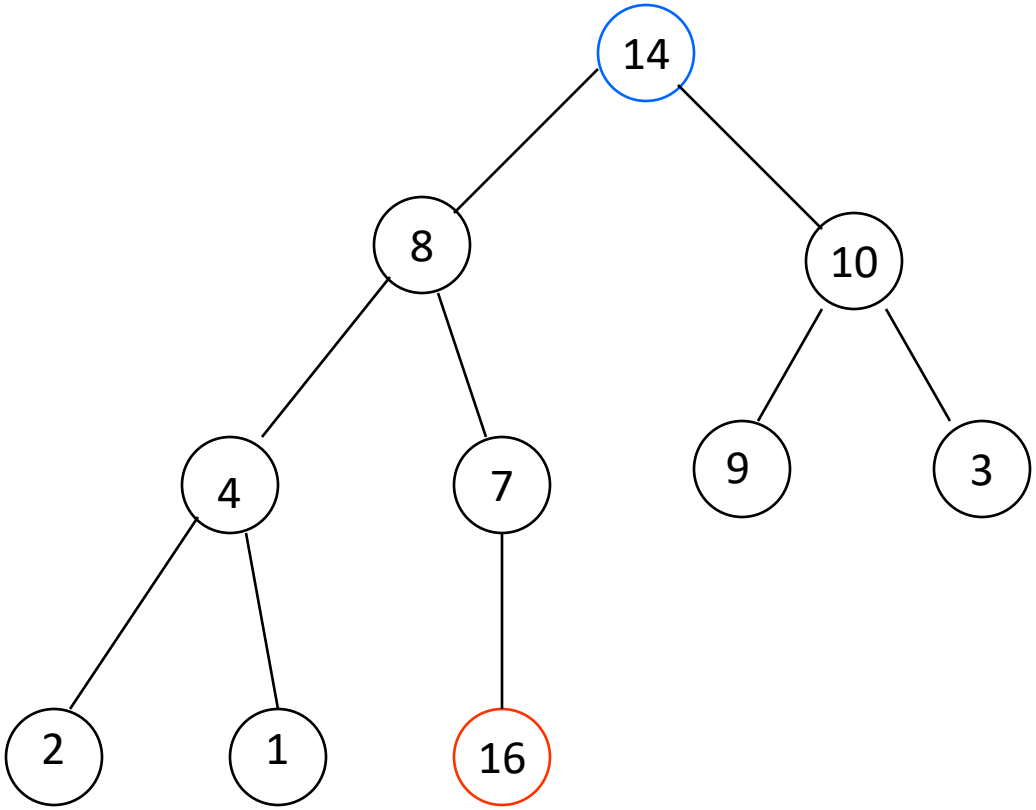


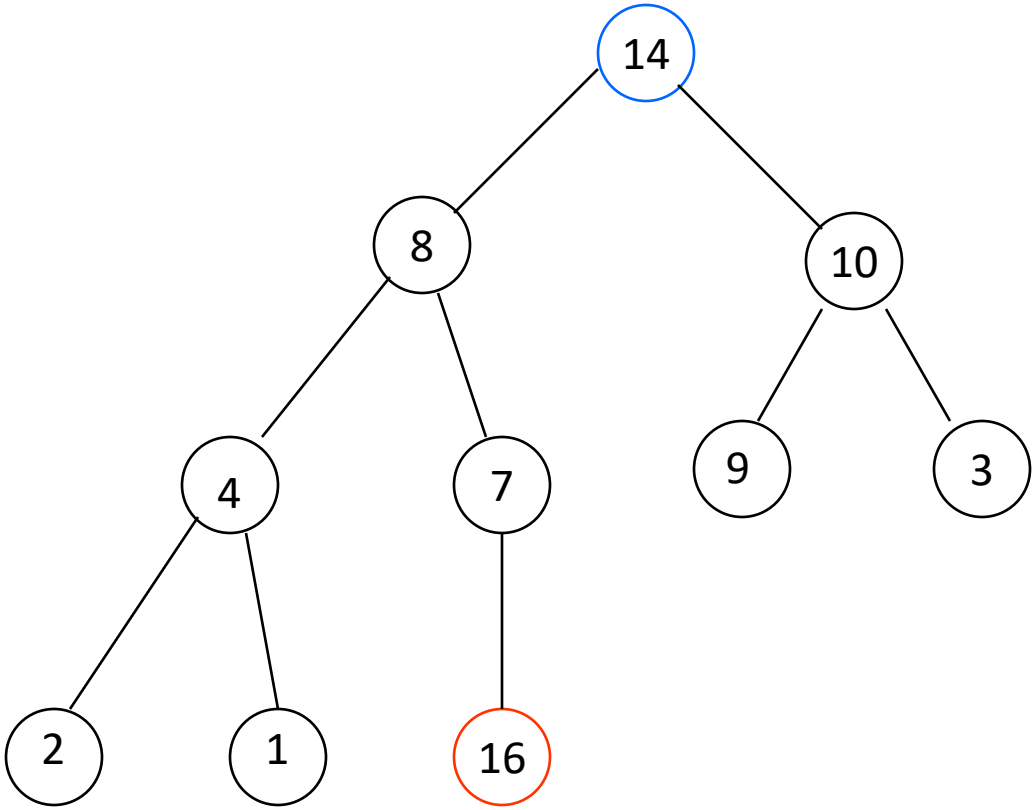
16, 14, 10, 8, 7, 9, 3, 2, 4, 1.


1
14
10
8
7
9
3
2
4
16

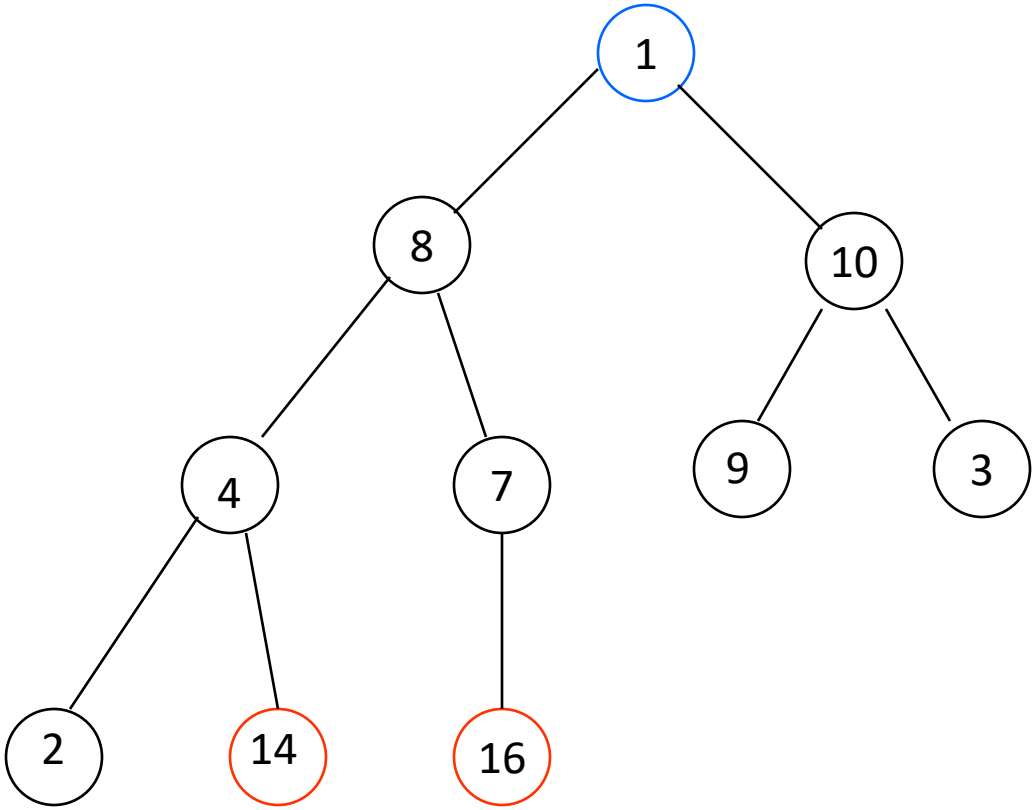


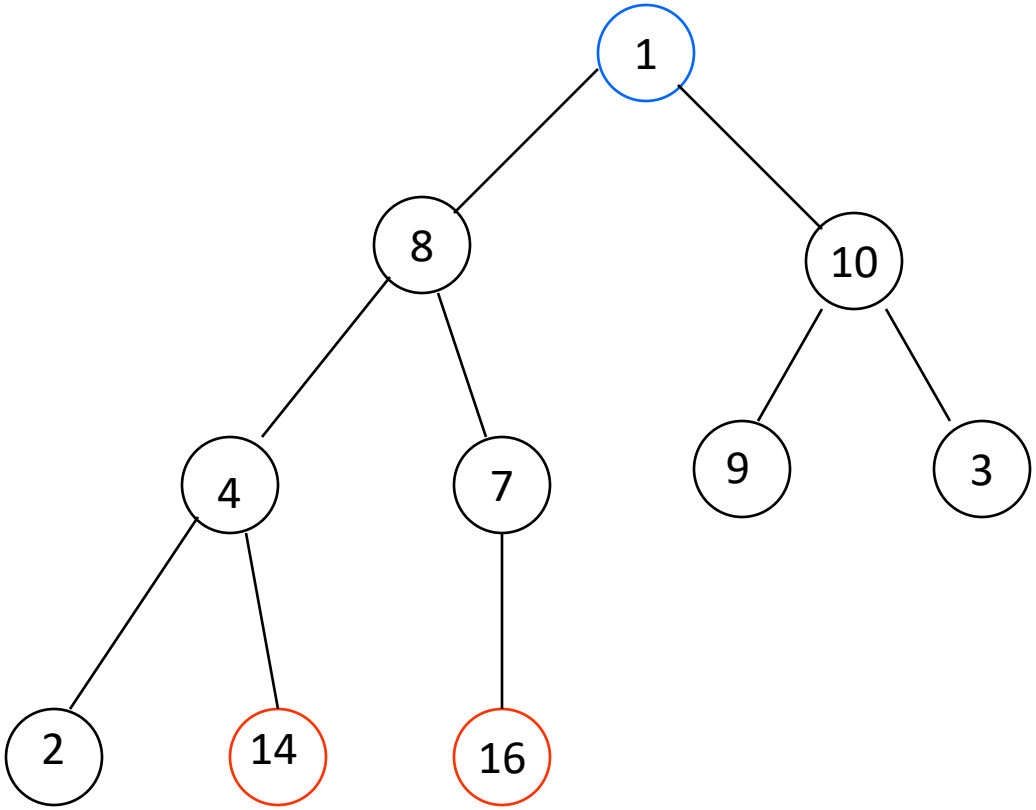
1, 14, 10, 8, 7, 9, 3, 2, 4, 16.


14
8
10
4
7
9
3
2
1
16

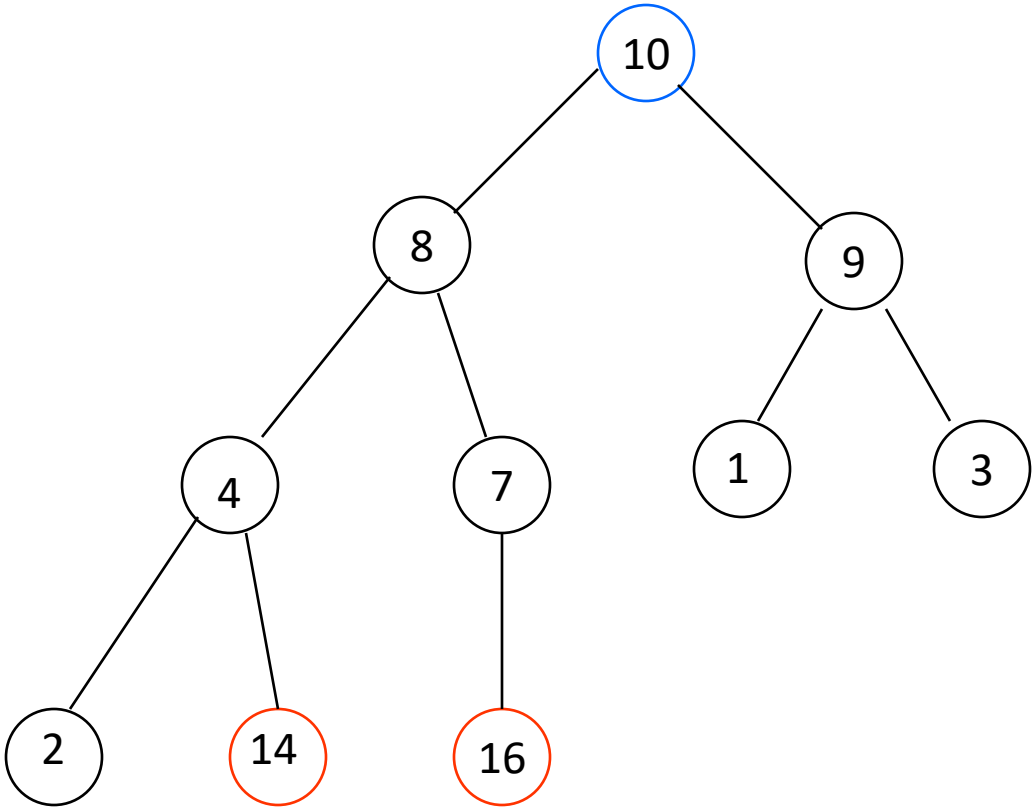


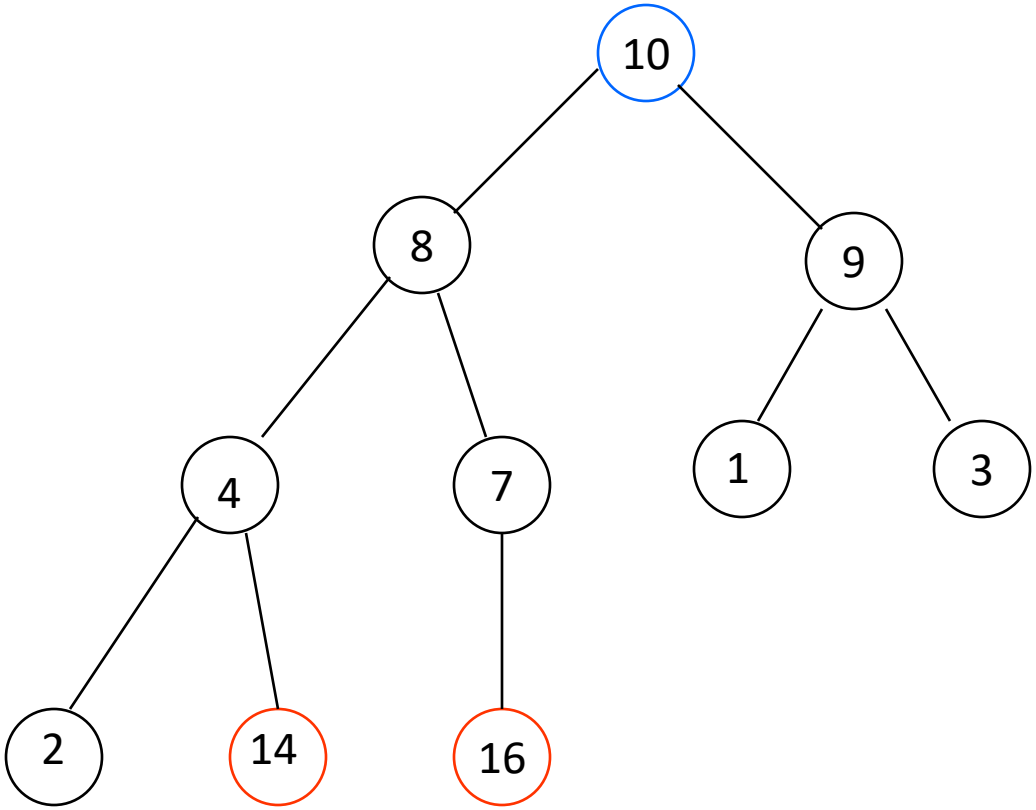
14, 8, 10, 4, 7, 9, 3, 2, 1, 16.


1
8
10
4
7
9
3
2
14
16

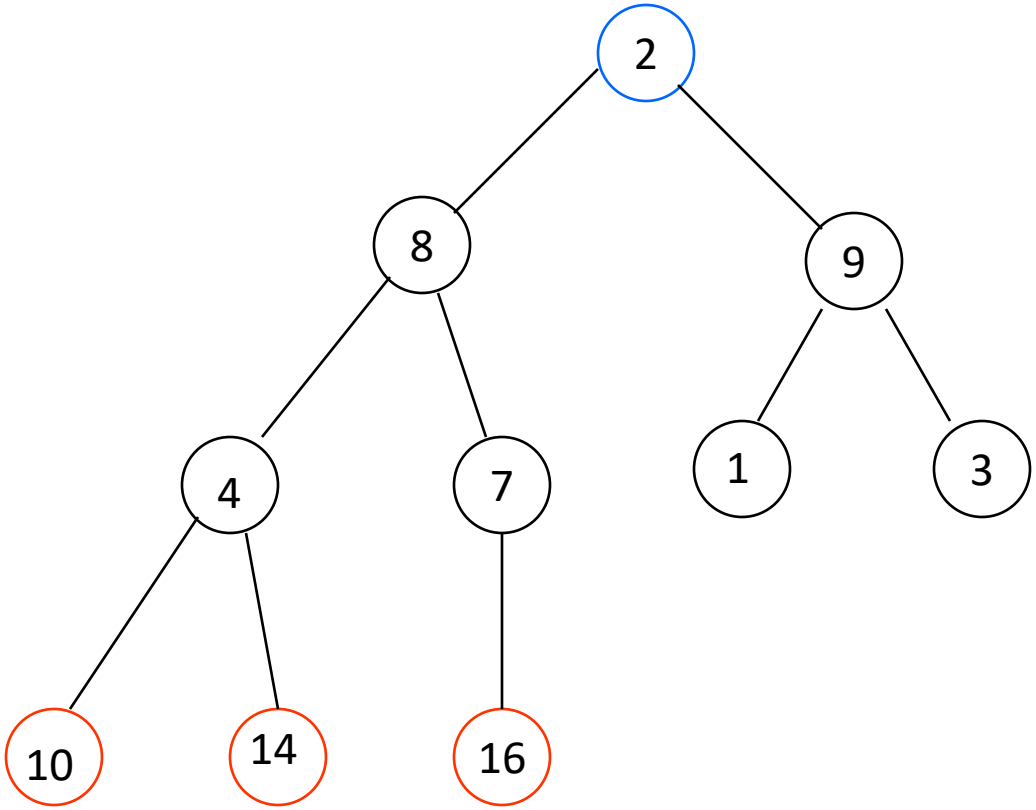


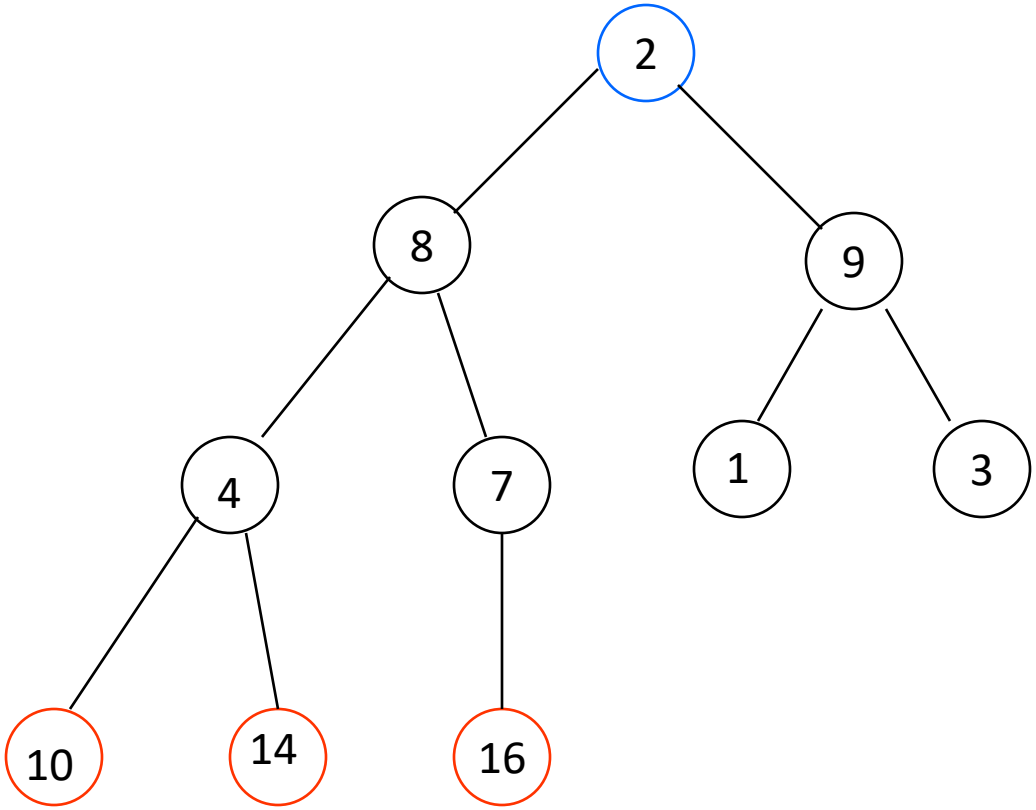
1, 8, 10, 4, 7, 9, 3, 2, 14, 16.


10
8
9
4
7
1
3
2
14
16

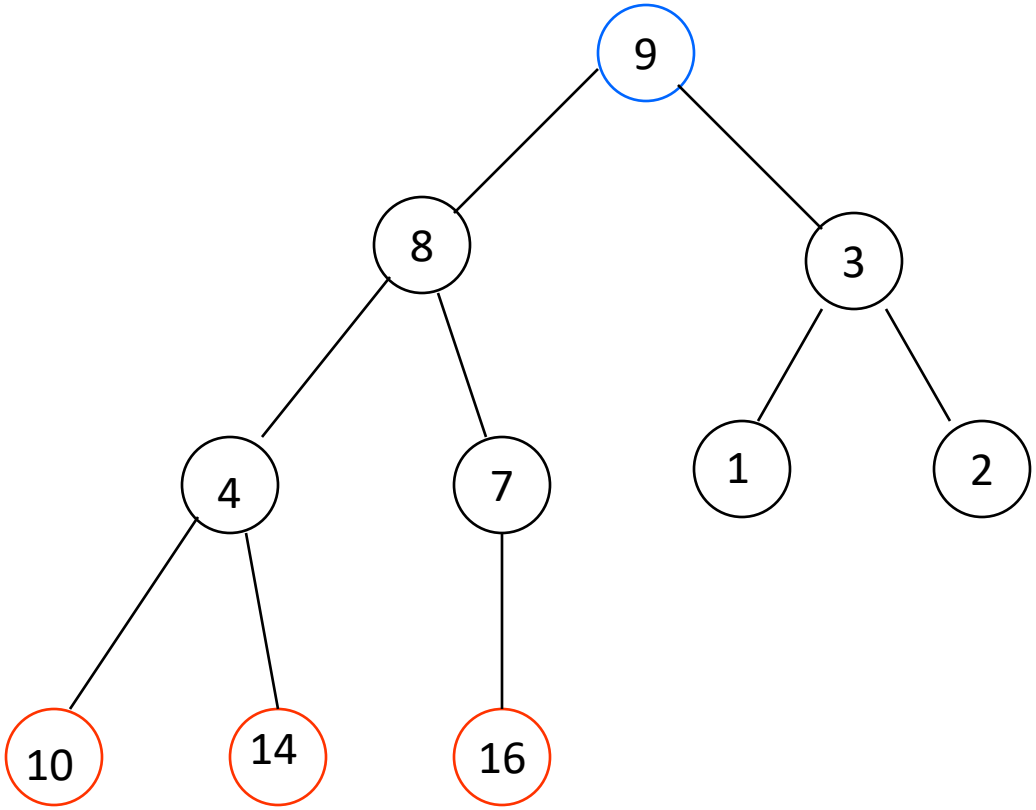


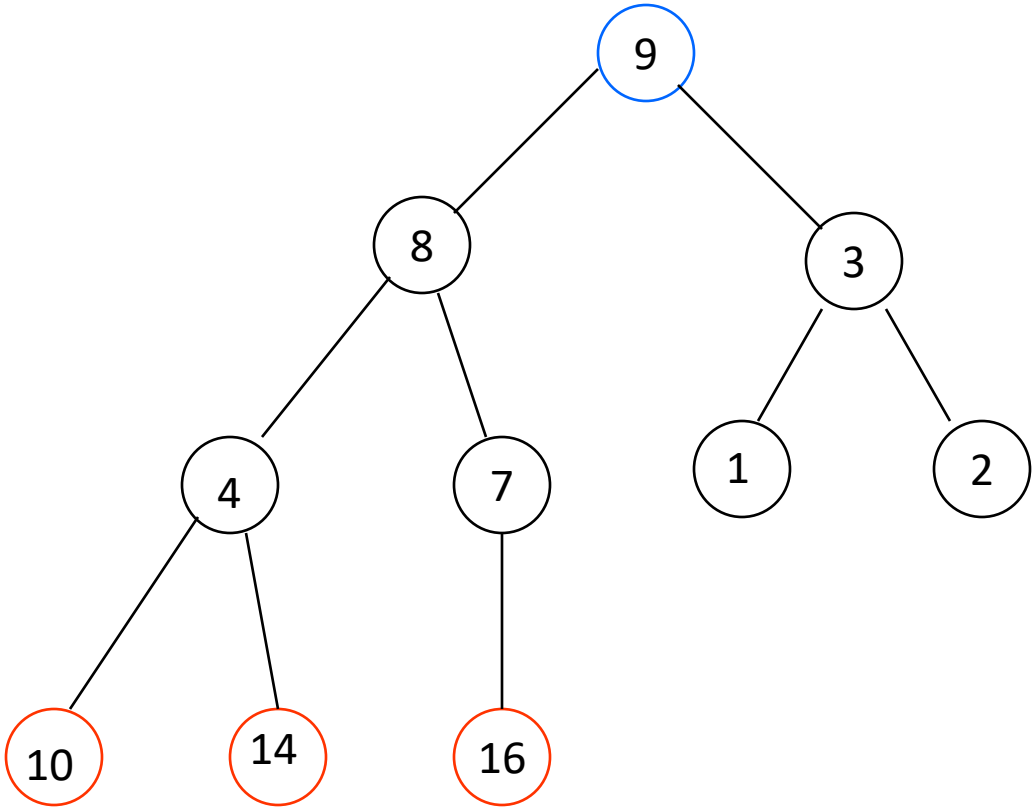
10, 8, 9, 4, 7, 1, 3, 2, 14, 16.


2
8
9
4
7
1
3
10
14
16

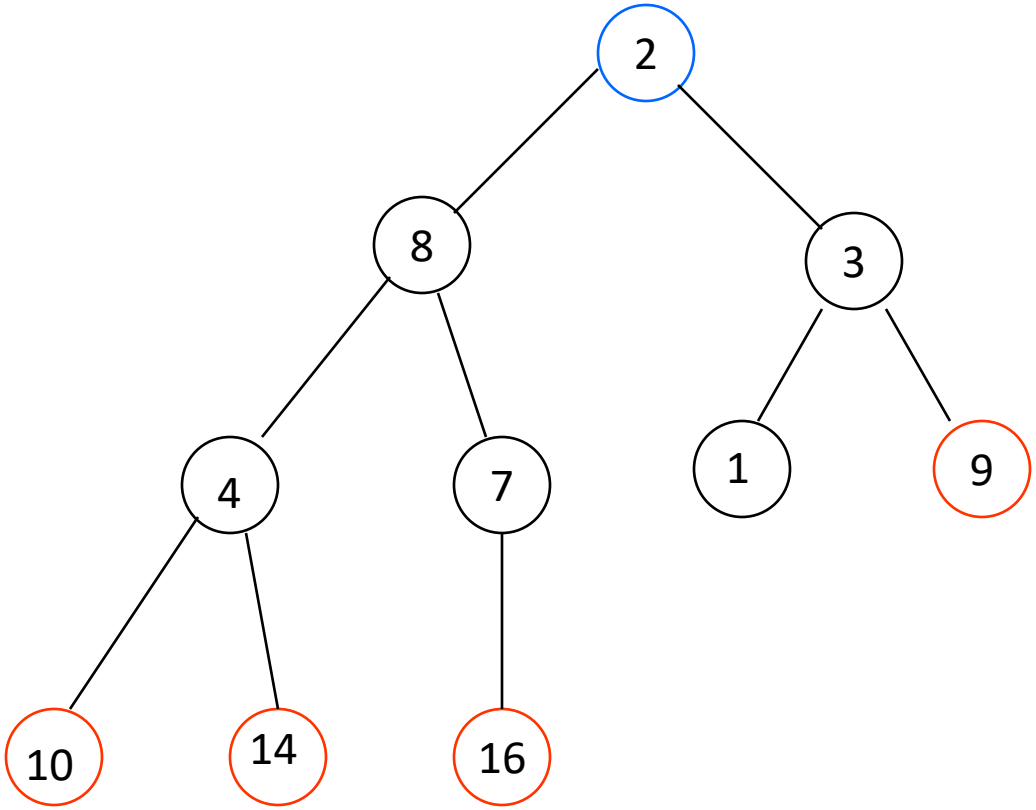


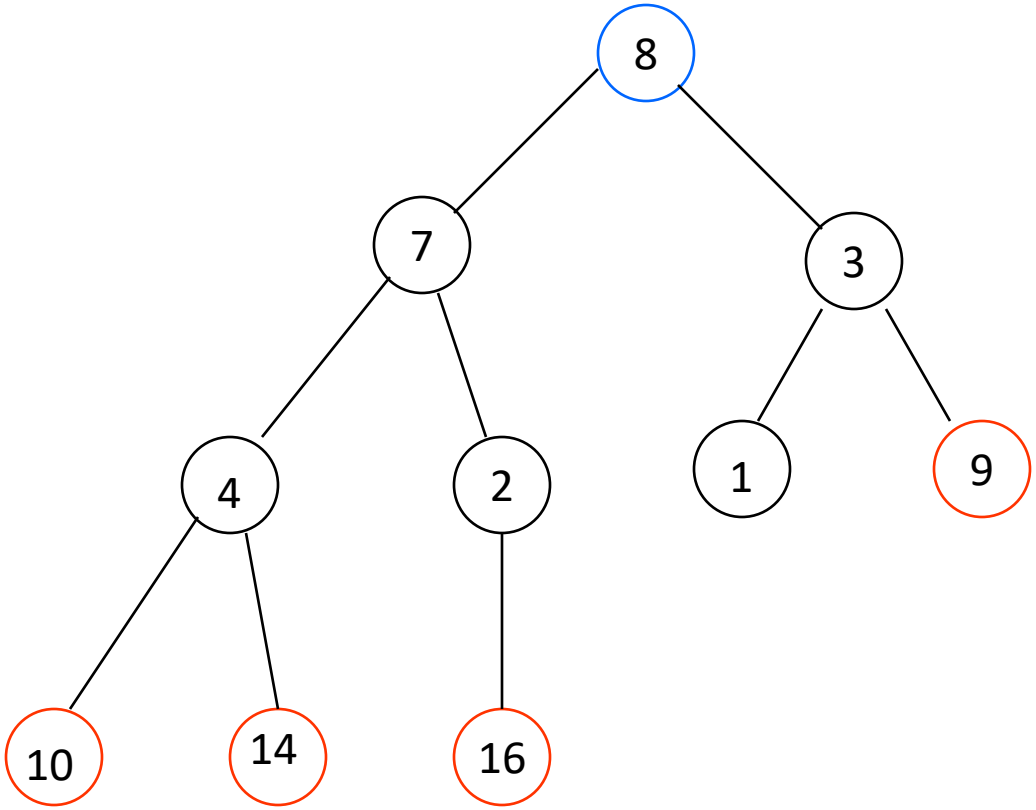
2, 8, 9, 4, 7, 1, 3, 10, 14, 16.

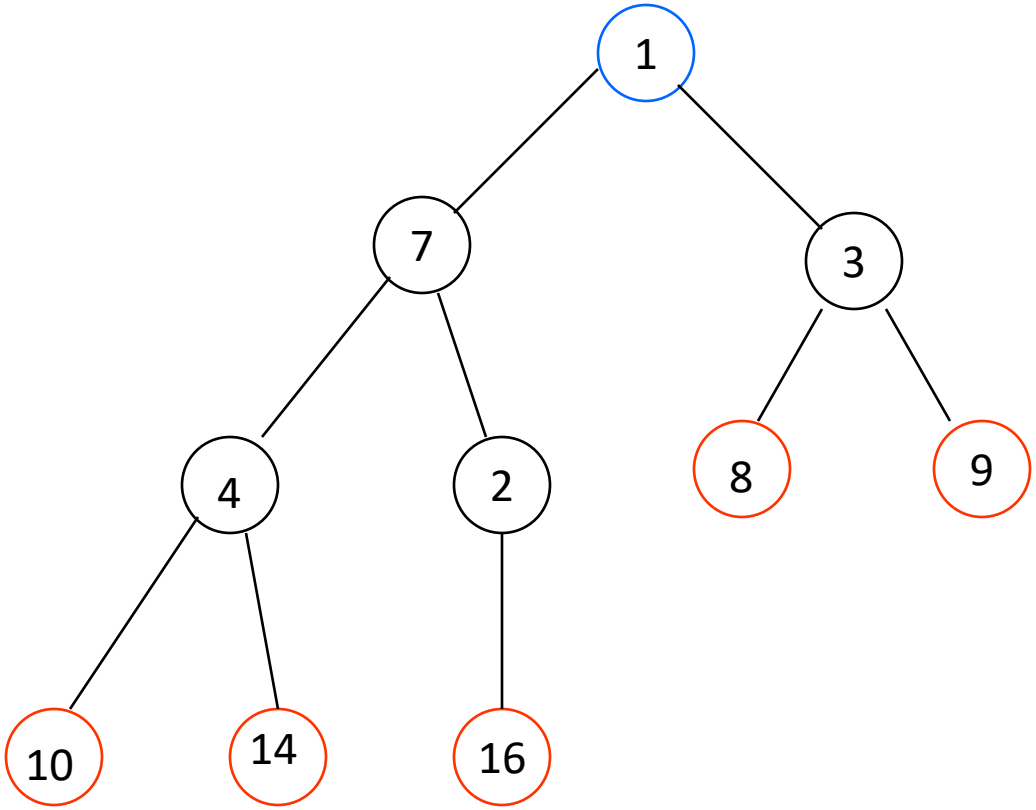

9
8
3
4
7
1
2
10
14
16

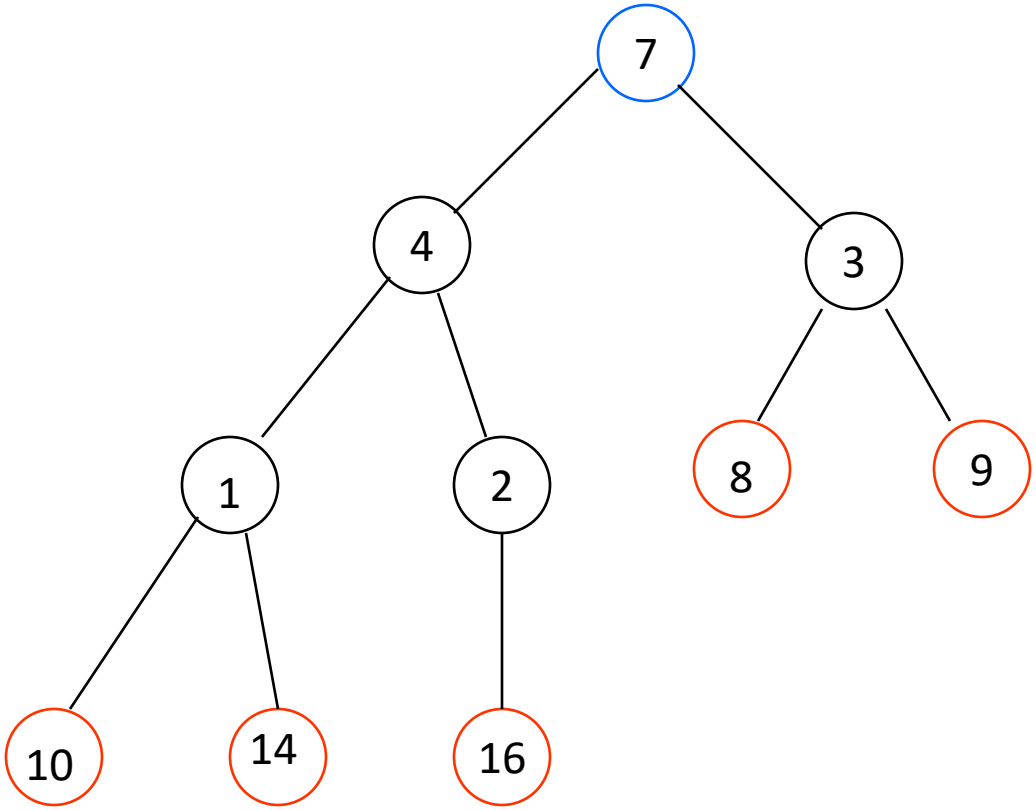


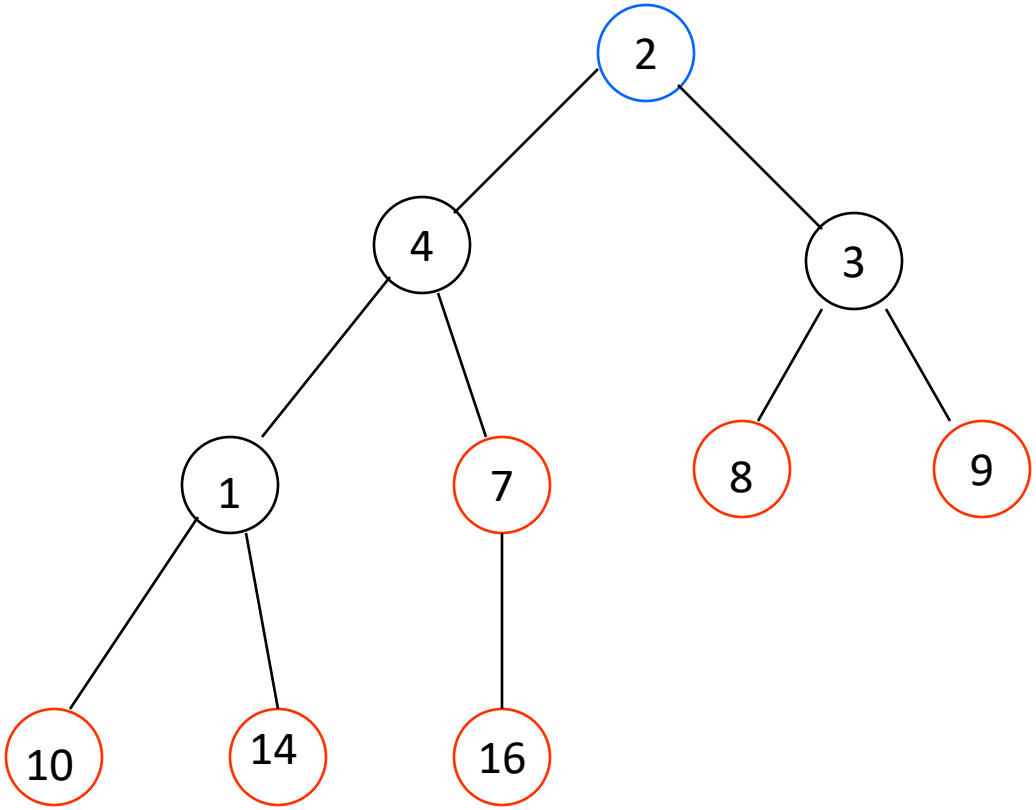
9, 8, 3, 4, 7, 1, 2, 10, 14, 16.

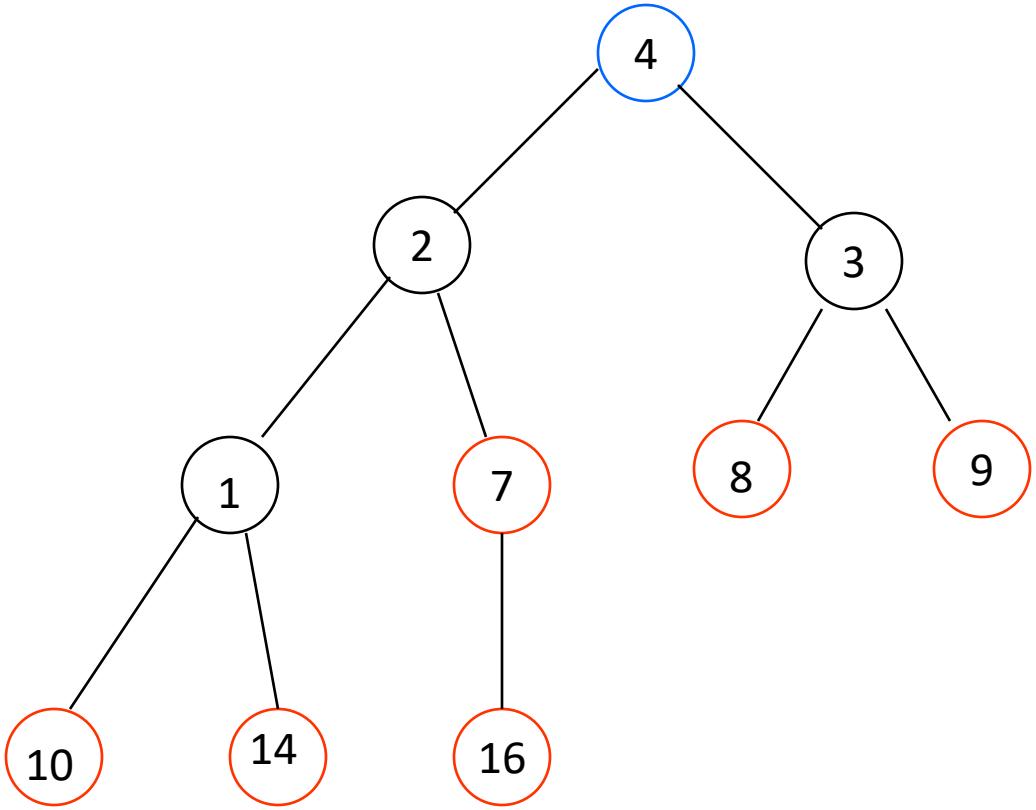

2
8
3
4
7
1
9
10
14
16

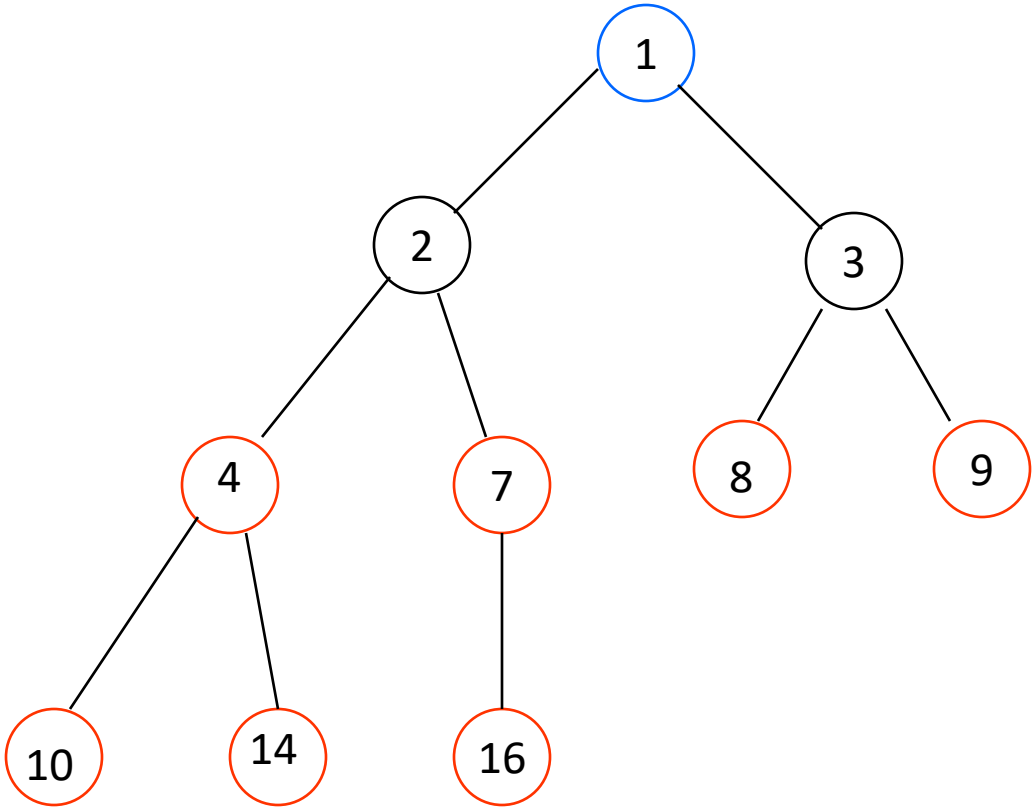

8
7
3
4
2
1
9
10
14
16

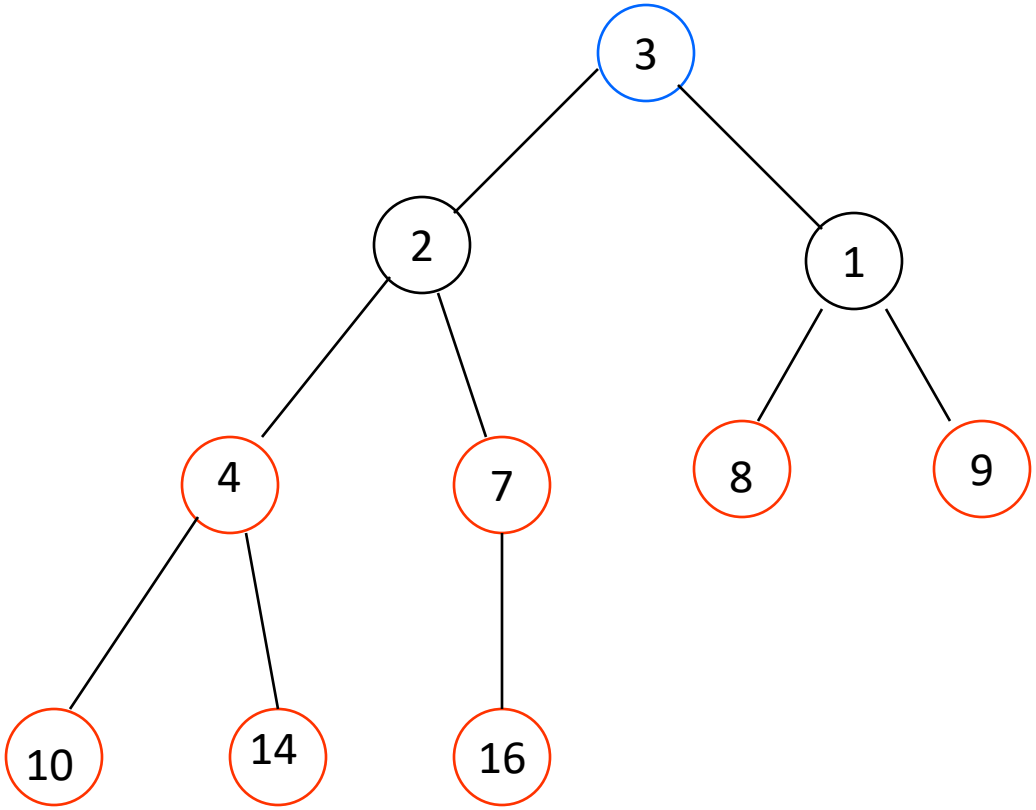

1
7
3
4
2
8
9
10
14
16

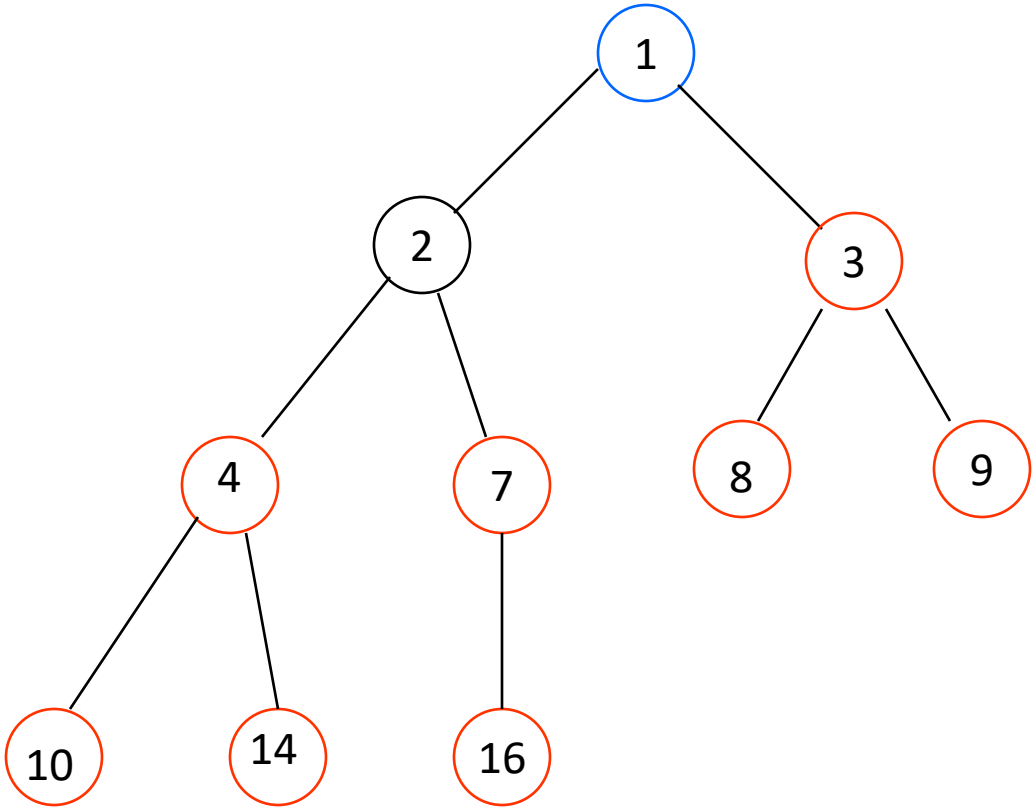

7
4
3
1
2
8
9
10
14
16

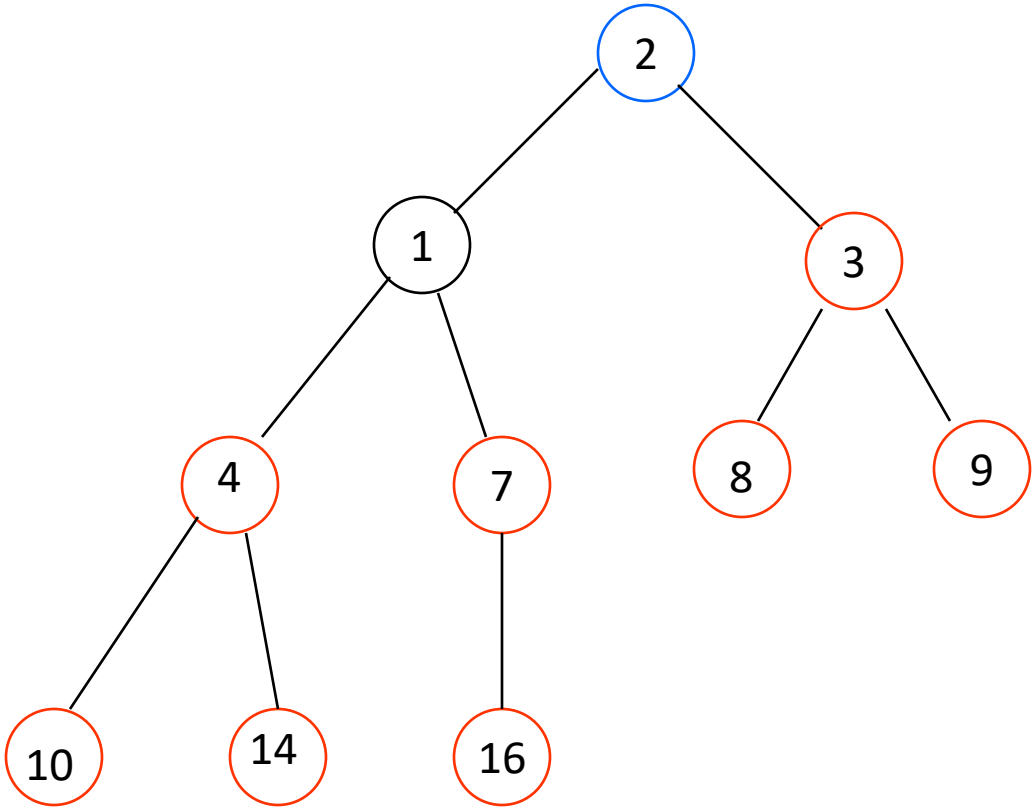

2
4
3
1
7
8
9
10
14
16

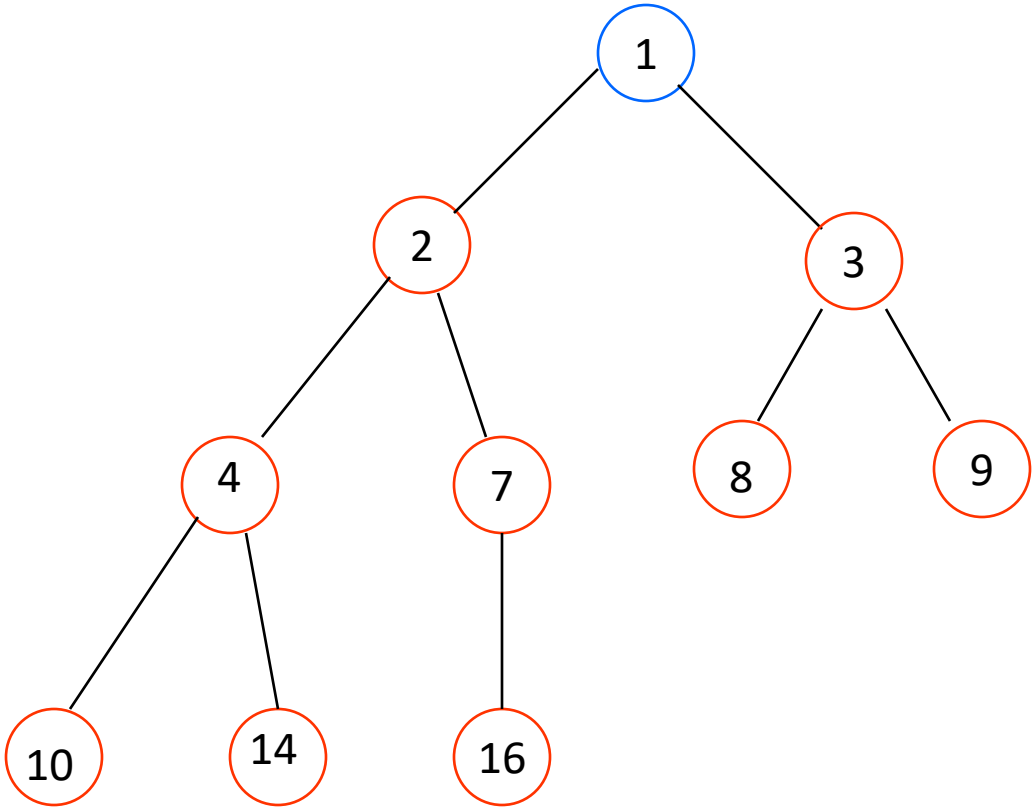

4
2
3
1
7
8
9
10
14
16


1
2
3
4
7
8
9
10
14
16


3
2
1
4
7
8
9
10
14
16


1
2
3
4
7
8
9
10
14
16


2
1
3
4
7
8
9
10
14
16


1
2
3
4
7
8
9
10
14
16

Running Time

Heapsort(A)

Buid-Max-Heap(A);

for $i \leftarrow length[A]$ downto 2 — O(n)

do begin

exchange $A[1] \leftrightarrow A[i]$;

$heap\text{-}size[A] \leftarrow heap\text{-}size[A] - 1$;

Max-Heapify(A,1); — O(lg n)

end-for

$O(n \lg n)$

Thank You!!!